

Deep Learning

[Notes by Piai Luca]

Academic Year 2024-2025

Contents

1	Feed Forward Neural Networks (FFNN)	1
1.1	Basic definitions of a NN	1
1.1.1	Artificial neurons	1
1.1.2	Single layer FFNN	2
1.1.3	Multi-layered structure	3
1.2	Input-output relation: the forward pass	4
1.3	Output layer and loss functions	5
1.3.1	Regression: MSE as loss function	5
1.3.2	Binary classification: sigmoid as output function	6
1.3.3	Binary classification: cross-entropy as loss function	7
1.3.4	Multi-class classification: softmax as output function	7
1.3.5	Multi-class classification: categorical cross-entropy as loss function	8
1.4	Backpropagation	9
1.4.1	Backpropagation derivation	9
1.5	Activation functions	12
1.5.1	Sigmoid Activation	12
1.5.2	Hyperbolic Tangent (tanh)	13
1.5.3	Linear Activation	14
1.5.4	Rectified Linear Unit (ReLU)	14
1.5.5	Leaky ReLU	14
1.5.6	Exponential Linear Unit (ELU)	15
1.6	Parameters initialization	15
1.7	Hyper-parameters	16
1.7.1	Learning rate	16
1.7.2	Mini-batch size	17
1.7.3	Number of layers and their size	17
1.7.4	Validation set for Hyper-parameters tuning	17

2	Convolutional Neural Networks	19
2.1	Limitations of the FFNN	19
2.2	2D convolution for finite-length signals	20
2.2.1	Convolution: 3 step procedure	20
2.2.2	Convolution implementation for CNNs	21
2.3	CNN structure	22
2.3.1	Layer overview	22
2.3.2	Output size vs filter size and stride	24
2.4	FFNN vs CNN	25
2.4.1	Local connectivity	25
2.4.2	Local weights	26
2.4.3	Translation equivariance	26
2.5	Pooling layers	27
2.6	Backpropagation for CNNs	28
2.6.1	Notation	28
2.6.2	Backpropagation derivation	28
2.6.3	Pooling during backpropagation	34
2.7	Alternative convolution approaches	34
2.7.1	Implementing Causality	34
2.7.2	Dilation	35
2.7.3	Temporal causal dilated convolution	35
2.7.4	Relation between dilation and receptive field	36
3	Automatic differentiation	39
3.1	The general problem	39
3.2	Symbolic differentiation	40
3.3	Numerical differentiation	40
3.4	Automatic differentiation	40
3.4.1	Forward mode	41
3.4.2	Backward mode	43
4	Overfitting and regularization in NN	47
4.1	Early stopping	47
4.2	Training on more data	47
4.3	L2 Regularization and mini-batch SGD	49
4.4	Dropout	50
4.4.1	Model ensembling	50

4.4.2	Dropout formally	50
4.5	Final considerations on reducing overfitting	53
5	Advanced optimization methods for NN	55
5.1	SGD	55
5.2	Mini-batch SGD	56
5.3	SGD with Momentum	58
5.3.1	Nesterov Momentum	60
5.4	Adaptive learning rate methods	61
5.4.1	Normalized gradient descent	61
5.4.2	Adaptive learning rates: AdaGrad	62
5.4.3	Root Mean-Square propagation	63
5.4.4	Adaptive moment estimation (Adam)	64
5.5	Choice of the optimizer	66
6	Batch Normalization	67
6.1	Covariate shift	67
6.2	BN main idea	68
6.3	BN layer	69
6.4	Learnable parameters	70
6.5	BN during testing	71
6.6	How to train BN layer	72
6.7	BN effects	74
6.8	BN results	75
7	Residual Networks	77
7.1	Top-N error for classification	77
7.2	Depth is a problem in NN?	77
7.3	Residual layer: original proposal [He2016]	78
7.3.1	Results on CIFAR-10	79
7.4	Residual layer: new proposal [He2016-b]	80
7.4.1	Backpropagating the gradient	80
7.4.2	Results on CIFAR-10	82
7.5	Residual network: effects	82
8	Recurrent Neural Networks	83
8.1	Why are RNN useful?	83
8.2	Vanilla RNN	83

8.2.1	Vanilla layer	84
8.2.2	Vanilla RNN problem	85
8.3	Long-Short Term Memory (LSTM)	85
8.3.1	LSTM cell	85
8.3.2	LSTM vs Vanilla RNN	87
8.3.3	Multiple layer LSTM	88
8.3.4	LSTM for sentimental analysis (NLP)	88
9	Autoencoders	91
9.1	Basic autoencoder structure	91
9.1.1	Problem of the basic structure	92
9.2	Regularized autoencoders	93
9.3	Denoising autoencoders	93
9.4	Variational autoencoders (VAE)	94
9.4.1	How to sample from a distribution	95
9.4.2	Latent representation (LR)	97
9.4.3	Learn $p(\mathbf{x})$	97
9.4.4	Deriving the loss function	100
9.4.5	Maximizing the ELBO	102
9.4.6	Loss function	103
9.4.7	Test time: generate new samples	105
10	Generative Adversarial Networks	107
10.1	VAE drawbacks	107
10.2	GAN approach: implicit density models	108
10.3	Adversarial framework formalized	109
10.3.1	Minmax game formulation	109
10.3.2	Expected outcome of the game	110
10.4	GAN training	113
10.4.1	Resistant to overfitting	114
10.4.2	Diminishing gradients	114
10.4.3	Non convergence: bouncing problem and label smoothing	115
10.4.4	Non convergence: mode collapse and minibatch discrimination	117
10.5	Conditional GANs	119
11	Continual Learning	121
11.1	Continual learning definition	121
11.2	Catastrophic forgetting	122

11.3 Rehearsal: replay-based method	123
11.4 Elastic weight consolidation (EWC)	123
11.4.1 Naive regularizer	124
11.4.2 EWC approach	124
11.4.3 Results	127
12 Contrastive learning	129
12.1 Contrastive Learning framework	130
12.2 Positive and negative keys sampling	131
12.2.1 Multisensory signals	131
12.2.2 Data transformations	132
12.2.3 Context-instance relationship	132
12.2.4 Sequential coherency and consistency	132
12.2.5 Natural clustering	133
12.2.6 Dissimilarity	133
12.3 Scoring functions for similarity	133
12.4 Contrastive losses	134
12.4.1 "Original" Contrastive loss	134
12.4.2 Triplet loss	134
12.4.3 NT-Xent	135
12.5 Transform head	136
12.6 SimCLR	137
12.6.1 Structure	137
12.6.2 Results	138

Chapter 1

Feed Forward Neural Networks (FFNN)

1.1 Basic definitions of a NN

The problem of the perceptron algorithm is that it learns only halfspaces and it fails if the data are non linearly separable. One possible solution to this problem is to use a feature mapping $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^m$ that is non linear in order to make the data linearly separable. Finding a very general feature mapping ϕ (e.g. kernel-based machine learning, random features) is a very difficult challenge, also this function can be computationally expensive to compute. Another possible solution is to use a neural network! We let the nn the hard part of learning the feature mapping.

1.1.1 Artificial neurons

Each neuron implements a smaller function that is based on the Perceptron, but instead of the *sign* function, a *non-linear activation function* $f(\cdot)$ is used. The input of a neuron j is defined as:

$$a_j := \sum_{i=1}^d w_{j,i} x_i + b_j = \langle \mathbf{w}_j, \mathbf{x} \rangle \quad (1.1)$$

where b_j is the bias associated to node j , $\mathbf{x} \in \mathbb{R}^d$, $\mathbf{w}_j \in \mathbb{R}^d$ are the weights associated to each arc that enters in node j . The output of the neuron j is

$$y_j := f(a_j) = f(\langle \mathbf{w}_j, \mathbf{x} \rangle) \quad (1.2)$$

where f is a non-linear function.

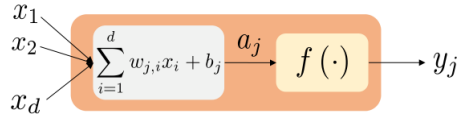


Figure 1.1: Artificial neuron.

1.1.2 Single layer FFNN

A single layer neural network is composed by many neurons in the only hidden layer. The input vector is fed to the hidden layer. We define:

$n_h :=$ number of neurons in the hidden layer

$n_{out} :=$ number of neurons in the output layer

$d :=$ input size $\mathbf{x} \in \mathbb{R}^d$

$\mathbf{w}_j :=$ vector of weights associated to the arcs entering in j

$$W^{in} \in \mathbb{R}^{n_h \times d} := \begin{bmatrix} \mathbf{w}_1^T \\ \vdots \\ \mathbf{w}_{n_h}^T \end{bmatrix}, \quad W^{out} \in \mathbb{R}^{n_{out} \times n_h} := \begin{bmatrix} \mathbf{w}_1^T \\ \vdots \\ \mathbf{w}_{n_{out}}^T \end{bmatrix}$$

$\mathbf{b}_j :=$ bias of node j

$$\mathbf{b}^{in} \in \mathbb{R}^{n_h} := \begin{bmatrix} b_1 \\ \vdots \\ b_{n_h} \end{bmatrix}, \quad \mathbf{b}^{out} \in \mathbb{R}^{n_{out}} := \begin{bmatrix} b_1 \\ \vdots \\ b_{n_{out}} \end{bmatrix}$$

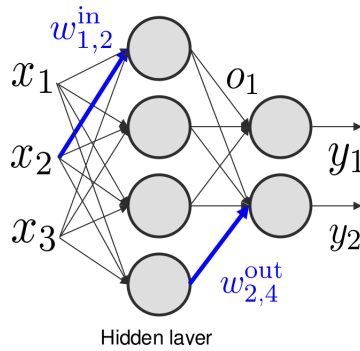


Figure 1.2: Single layer FFNN.

The vector containing all the input of the neurons is obtained as:

$$\mathbf{a} = W^{in}\mathbf{x} + \mathbf{b}^{in}.$$

The output vector of the hidden layer is obtained by applying the non-linear activation function f to the vector $\mathbf{a}$ element wise:

$$\mathbf{o} = f(\mathbf{a}).$$

Finally, the output vector of the network is obtained by multiplying the vector $\mathbf{o}$ to the matrix of weights W^{out} and then by applying the non-linear function g to the result:

$$\mathbf{y} = g(W^{out}\mathbf{o} + \mathbf{b}^{out}).$$

Note that in general the non-linear activation function f is usually different from the non-linear activation function g used for the nodes in the output layer.

From a theoretical point of view, a single-layer FFNN can approximate any function with arbitrary accuracy given a suitable activation function. That is, FFNN are **universal approximators** and this has been proven. The problem is that, in order to approximate a function, the number of neurons required it might be exponential in d (dimension of the input). The solution is to use a deep neural network: multiple layers with few neurons each.

1.1.3 Multi-layered structure

Some notational conventions that we're going to use:

$w_{j,i}^{(\ell)} :=$ weight on the arc that connects neuron i of layer
 $(\ell - 1)$ with neuron j of layer ℓ

$W^{(\ell)} :=$ matrix of weights of layer ℓ

$\mathbf{b} :=$ biases of layer ℓ

$a_j^{(\ell)} :=$ activation/input of neuron j of layer ℓ

$o_j^{(\ell)} := f(a_j^{(\ell)}) :=$ output of neuron j of layer ℓ

$\mathcal{L}(\mathcal{W}, \mathbf{x}) :=$ loss function of the FNN on input $\mathbf{x}$

$\mathcal{W} :=$ set of FFNN parameters (includes weights and biases)

$\mathcal{D} := \{(\mathbf{x}_1, t_1), \dots, (\mathbf{x}_N, t_N)\}$ labeled training data

$y_m :=$ output of the $m - th$ output unit

$\mathbf{y} :=$ output vector

$\ell = 1, \dots, L$ indices of the layers

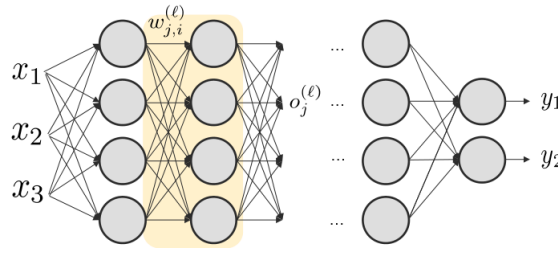


Figure 1.3: Multi-layered structure.

The notation for the matrix $W^{(\ell)}$ seems weird since the subscript j, i denote the weight of the arc from $i \rightarrow j$. It means, as we have already seen in the previous subsection, that each line in the matrix is a transposed vector $(\mathbf{w}_j)^T$. Using this notation allow us to use directly the matrix $W^{(\ell)}$, instead of its transposed.

$$\mathbf{a}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{o}^{(\ell-1)} + \mathbf{b}^{(\ell)}$$

The diagram shows the matrix notation for the forward pass. It features a large matrix $\mathbf{W}^{(\ell)}$ with rows and columns. A specific row is highlighted in yellow, containing the weights $w_{j,i-1}$, $w_{j,i}$, and $w_{j,i+1}$. A vector $\mathbf{a}_j^{(\ell)}$ is shown to the left of the matrix, with an arrow pointing to the highlighted row. The matrix is labeled with "Row idx" and "Col idx". To the right of the matrix is a vector $\mathbf{o}^{(\ell-1)}$ with a specific element o_i highlighted in yellow.

Figure 1.4: Notation: no need to transpose $W^{(\ell)}$.

1.2 Input-output relation: the forward pass

By propagating the input $\mathbf{x}$ from the input layer to the output layer, we're basically computing *multi-layer* perceptron (MLP). The pass from the input $\mathbf{x}$ to the output $\mathbf{y}$ is obtained with the so-called **forward pass**.

Forward pass

1. Feed $\mathbf{x}$ to the first layer

$$\mathbf{a}^{(1)} = \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}, \quad \mathbf{o}^{(1)} = f(\mathbf{a}^{(1)})$$

2. Use $\mathbf{o}^{(1)}$ as the input to the second layer

3. Repeat for all hidden layers

$$\mathbf{a}^{(\ell)} = W^{(\ell)} \mathbf{o}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{o}^{(\ell)} = f(\mathbf{a}^{(\ell)})$$

4. Get output vector as

$$\mathbf{a}^{(L)} = W^{(L)} \mathbf{o}^{(L-1)} + \mathbf{b}^{(L)}, \quad \mathbf{y} = g(\mathbf{a}^{(L)})$$

5. Store all the activations and output for later use (we will them for the backpropagation algorithm).

1.3 Output layer and loss functions

The loss function express the optimization objective of a NN training and it should reflect the learning task we want to solve. How to know if a loss function is good for our problem? A typical approach is to choose the loss as the **negative log-likelihood** (NLL) of parameters. By assuming that the training samples are i.i.d we can decompose the NLL as sum of single terms.

$$\mathcal{L}(\mathcal{D}, \mathcal{W}) = -\log p(\underbrace{\{\{t_n\}_{n=1}^N \mid \{\mathbf{x}_n\}_{n=1}^N; \mathcal{W}\}}_{\text{Likelihood function}}) = -\sum_{n=1}^N \log \underbrace{p(t_n \mid \mathbf{x}_n; \mathcal{W})}_{\text{Per-sample likelihood}} \quad (1.3)$$

1.3.1 Regression: MSE as loss function

Let $\mathcal{D} = \{(\mathbf{x}_1, \mathbf{t}_1), \dots, (\mathbf{x}_N, \mathbf{t}_N)\}$ be the labeled training set, where each label $\mathbf{t}_n \in \mathbb{R}^r$ (*multi-output regression*) and we assume that

$$\mathbf{t}_n = \mathbf{y}_n + \text{Gaussian noise}.$$

We have that the activation function g for the output layer is computed as $g(\mathbf{a}^{(L)}) = \mathbf{a}^{(L)}$ (identity). By using the NLL function and by assuming that the training samples are i.i.d, our objective is to maximize $p(\mathbf{t}_n | \mathbf{x}_n; \mathcal{W})$. Since we're considering a Gaussian noise, we consider a conditional Gaussian distribution for $\mathbf{t}_n$:

$$p(\mathbf{t}_n | \mathbf{x}_n; \mathcal{W}) = \mathcal{N}(\mathbf{t}_n; \mathbf{y}_n, \Sigma)$$

where $\mathbf{y}_n$ is the mean, Σ the covariance matrix.

We have that

$$\mathcal{N}(\mathbf{t}_n; \mathbf{y}_n, \Sigma) := \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left\{ -\frac{1}{2} (\mathbf{t}_n - \mathbf{y}_n)^T \Sigma^{-1} (\mathbf{t}_n - \mathbf{y}_n) \right\}$$

Taking $-\log$, leads to:

$$-\log \mathcal{N}(\mathbf{t}_n; \mathbf{y}_n, \Sigma) = \frac{1}{2} [\log |\Sigma| + d \log(2\pi) + (\mathbf{t}_n - \mathbf{y}_n)^T \Sigma^{-1} (\mathbf{t}_n - \mathbf{y}_n)]$$

Setting $\Sigma = I$ (identity), in the end we get

$$\mathcal{L}(\mathcal{D}, \mathcal{W}) = \frac{1}{2} \sum_{n=1}^N \|\mathbf{y}_n - \mathbf{t}_n\|^2 \quad (1.4)$$

that is, we get the **Squared Error** as objective function¹. So all of this was to say that, if we're using the MSE as objective function, than we're assuming a Gaussian noise in our measurements $\mathbf{t}_n$.

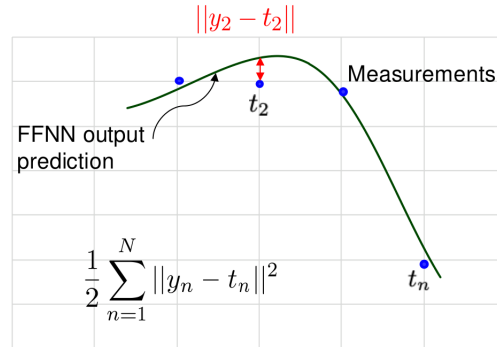


Figure 1.5: MSE.

1.3.2 Binary classification: sigmoid as output function

For binary classification we have $t_n \in \{0, 1\}$, so in the output layer we have just one neuron. Let us call C_1, C_2 the two classes. A typical choice for output function is the sigmoid function:

$$y_n = g(a_n^{(L)}) = \sigma(a_n^{(L)}) = \frac{1}{1 + e^{-a_n^{(L)}}} \quad (1.5)$$

¹The constant terms are not present since they're useless in the objective function.

The sigmoid function maps $\mathbb{R} \rightarrow [0, 1]$ and it's interpreted as $y_n = p(C_1 | \mathbf{x}_n)$, i.e., as a probability that the sample $\mathbf{x}_n$ belongs to C_1 .

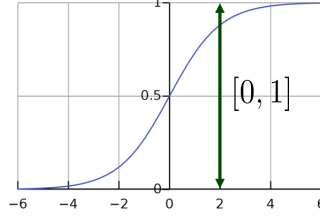


Figure 1.6: Sigmoid function

1.3.3 Binary classification: cross-entropy as loss function

Assume the samples in the training set are i.i.d.. The likelihood can be expressed as

$$p(t_n | \mathbf{x}_n, \mathcal{W}) = p(C_1 | \mathbf{x}_n)^{t_n} p(C_0 | \mathbf{x}_n)^{1-t_n}$$

so if $t_n = 1$ the second term is 0, vice versa if $t_n = 0$. By applying the $-\log$ we get

$$\begin{aligned} -\log p(t_n | \mathbf{x}_n, \mathcal{W}) &= -t_n \log p(C_1 | \mathbf{x}_n) - (1 - t_n) \log p(C_0 | \mathbf{x}_n) \\ &= -t_n \log y_n - (1 - t_n) \log (1 - y_n) \end{aligned}$$

since $y_n = p(C_1 | \mathbf{x}_n)$ and $1 - y_n = p(C_0 | \mathbf{x}_n)$.

In the end the total negative log-likelihood (on the entire dataset of N points) is

$$\begin{aligned} \mathcal{L}(\mathcal{D}, \mathcal{W}) &= - \sum_{n=1}^N \log p(t_n | \mathbf{x}_n; \mathcal{W}) \\ &= - \sum_{n=1}^N \underbrace{[t_n \log y_n + (1 - t_n) \log (1 - y_n)]}_{\text{binary cross-entropy function}} \end{aligned} \tag{1.6}$$

1.3.4 Multi-class classification: softmax as output function

In multi-class classification we have Q neurons in the output layer and the goal is to learn a mapping from an input vector to a set of Q labels. The labels are usually expressed using the **one-hot encoding**:

$$\mathbf{t}_n = [t_{n,1}, \dots, t_{n,q}]^T$$

if $\mathbf{x}_n$ belongs to class i then $t_{n,i} = 1$ and $t_{n,j} = 0 \forall j \neq i$.

The output should be a discrete probability distribution over the Q classes, so that $\mathbf{y}_n \in \mathbb{R}^Q$, where $y_{n,q} \in [0, 1] \forall q$ and $\sum_{q=1}^Q y_{n,q} = 1$. In other words we want that $y_{n,q} = p(t_{n,q} = 1 | \mathbf{x}_n)$. A common choice is the softmax function:

$$y_{n,q} = \frac{\exp(a_q^{(L)})}{\sum_{i=1}^Q \exp(a_i^{(L)})} \quad (1.7)$$

it's a soft (continuous) version of the max function.

1.3.5 Multi-class classification: categorical cross-entropy as loss function

The categorical cross-entropy function is a generalization of the binary cross-entropy function. The total error (on the entire training set) is defined as:

$$\mathcal{L}(\mathcal{D}, \mathcal{W}) = - \sum_{n=1}^N \sum_{q=1}^Q t_{n,q} \log y_{n,q} \quad (1.8)$$

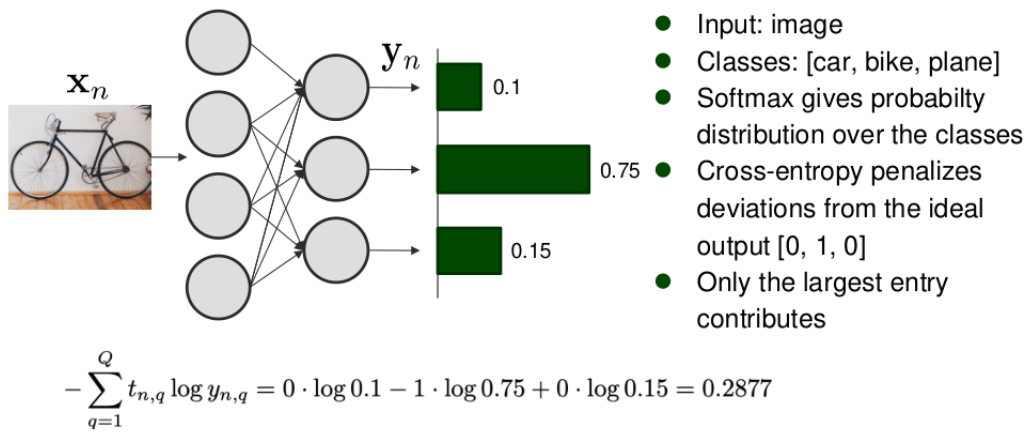


Figure 1.7: Cross entropy and softmax. In order to decrease the loss for $\mathbf{x}_n$ we need to increase the probability of class 2: the closest to 1, the lower the loss.

1.4 Backpropagation

One possible way to train a FFNN is to use the *stochastic gradient-descent* rule:

$$(w_{j,i}^{(\ell)})^{n+1} = (w_{j,i}^{(\ell)})^n - \eta \nabla_{w_{j,i}^{(\ell)}} \mathcal{L}(\mathcal{W}, \mathbf{x}) \quad (1.9)$$

The problems are: the gradient computation can be very computationally expensive; the number of weights can potentially be very large; the dependence of loss on each weight is complex (many layers). So we need an efficient algorithm that computes the update of the weights. This is the goal of the backpropagation algorithm.

The aim of the backpropagation is to compute the gradient of the loss function with respect to each weight/bias in the FNN:

$$\frac{\partial \mathcal{L}(\mathcal{W}, \mathbf{x})}{\partial w_{j,i}^{(\ell)}}, \frac{\partial \mathcal{L}(\mathcal{W}, \mathbf{x})}{\partial b_j^{(\ell)}}, \quad \forall i, j, \ell$$

To achieve this goal, the idea is to repeatedly use the *chain rule of derivatives*. This makes the computation efficient and recursively implementable.

Note that: the backpropagation is not a training algorithm, but rather an efficient way to compute the gradients of the loss function w.r.t. the network parameters. The training algorithm is the GD (or SGD, or the batch GD). Note also that the GD is an heuristic algorithm since the function that we're trying to minimize is (more often) non-convex. Therefore we don't know if the result found by the GD is actually a global optimum or a local one. One way of escaping local minima is to use the Stochastic GD or the batch GD, but we will see some techniques later in the course.

1.4.1 Backpropagation derivation

The first necessary step is to perform the forward pass of the input vector through the network. As a result we obtain $\mathbf{o}^{(\ell)}, \mathbf{a}^{(\ell)} \forall \ell = 1, \dots, L$ and the output $\mathbf{y}$. We will then use the results from this step to backpropagate the gradients from the last layer to the first.

Definition 1 (Chain rule). *Let define a composite function $y = F(x)$, $x = G(z)$. We have that:*

$$\frac{\partial y}{\partial z} = \frac{\partial F(G(z))}{\partial z} = \frac{\partial F}{\partial x} \frac{\partial x}{\partial z} = \frac{\partial F}{\partial x} \frac{\partial G}{\partial z}$$

By how the FFNN are constructed, we have as many nested functions as many layers in the network.

Consider the partial derivation of the loss function on the sample $\mathbf{x}$ with respect to the weight j, i in the layer ℓ :

$$\frac{\partial \mathcal{L}(\mathcal{W}, \mathbf{x})}{\partial w_{j,i}^{(\ell)}} \quad (1.10)$$

We know that the weight $w_{j,i}^{(\ell)}$ influences the activation $a_j^{(\ell)}$ since $a_j^{(\ell)} := \sum_n w_{j,n}^{(\ell)} o_n^{(\ell-1)} + b_j^{(\ell)}$. Proceeding with the forward pass, the activation $a_j^{(\ell)}$ will affect the output therefore also the loss. Therefore we can use the chain rule:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_{j,i}^{(\ell)}} &= \frac{\partial \mathcal{L}}{\partial a_j^{(\ell)}} \frac{\partial a_j^{(\ell)}}{\partial w_{j,i}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(\ell)}} o_i^{(\ell-1)} \\ \frac{\partial \mathcal{L}}{\partial b_j^{(\ell)}} &= \frac{\partial \mathcal{L}}{\partial a_j^{(\ell)}} \frac{\partial a_j^{(\ell)}}{\partial b_j^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(\ell)}} \cdot 1 \end{aligned} \quad (1.11)$$

where the last step is obtained just by looking at the definition of $a_j^{(\ell)}$.

Let define the **error message** as

$$\delta_j^{(\ell)} := \frac{\partial \mathcal{L}}{\partial a_j^{(\ell)}}, \quad \boldsymbol{\delta}^{(\ell)} = [\delta_1^{(\ell)}, \dots, \delta_j^{(\ell)}, \dots]^T. \quad (1.12)$$

By substituting the error message definition in the previous equation we get:

$$\frac{\partial \mathcal{L}}{\partial w_{j,i}^{(\ell)}} = \delta_j^{(\ell)} o_i^{(\ell-1)} \quad (1.13)$$

Note that $o_i^{(\ell-1)}$ (output of the parent neuron) is a value that we know after we compute the forward pass, so the unknown value is $\delta_j^{(\ell)}$ (error message).

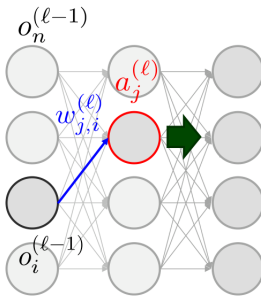


Figure 1.8

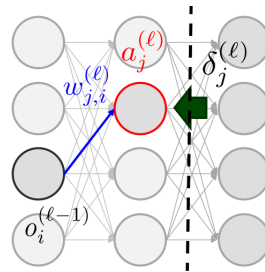


Figure 1.9

The activation $a^{(\ell)}$ affects all the neurons of layer $\ell + 1$. We can apply again the chain

rule but we need to consider all the neurons in the following layer:

$$\delta_j^{(\ell)} := \frac{\partial \mathcal{L}}{\partial a_j^{(\ell)}} = \sum_k \frac{\partial \mathcal{L}}{\partial a_k^{(\ell+1)}} \frac{\partial a_k^{(\ell+1)}}{\partial a_j^{(\ell)}} = \sum_k \delta_k^{(\ell+1)} \frac{\partial a_k^{(\ell+1)}}{\partial a_j^{(\ell)}} \quad (1.14)$$

Now we want to define a recursive expression. We need to study the term $\frac{\partial a_k^{(\ell+1)}}{\partial a_j^{(\ell)}}$.

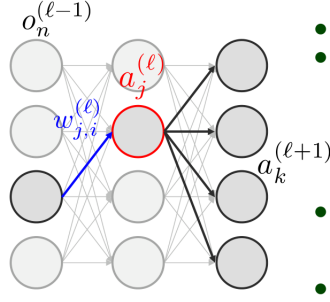


Figure 1.10

Recall that $a_k^{(\ell+1)} := \sum_m w_{k,m}^{(\ell+1)} o_m^{(\ell)} + b_k^{(\ell+1)}$ and that $o_m^\ell = f(a_j^{(\ell)})$. Therefore we can apply the chain rule again:

$$\frac{\partial a_k^{(\ell+1)}}{\partial a_j^{(\ell)}} = \frac{\partial a_k^{(\ell+1)}}{\partial o_j^{(\ell)}} \frac{\partial o_j^{(\ell)}}{\partial a_j^{(\ell)}} = w_{k,j}^{(\ell+1)} f'(a_j^{(\ell)}) \quad (1.15)$$

By substituting the above in the 1.14 we get the recursive relation that we were looking for:

$$\delta_j^{(\ell)} = f'(a_j^{(\ell)}) \sum_k \delta_k^{(\ell+1)} w_{k,j}^{(\ell+1)} \quad (1.16)$$

note that the derivative of the non linear function $f(\cdot)$ depends on the activation function that we choose to use.

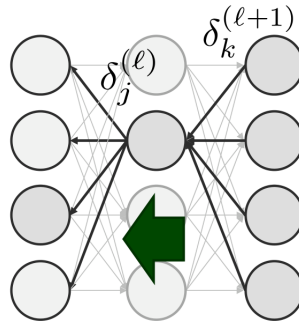


Figure 1.11

In order to compute the value of each error message $\delta^{(\ell)}$ for each layer of the network, we need a backward pass. This backward pass starts from the output layer, in this case the formula is the following (using the chain rule):

$$\delta_m^{(L)} := \frac{\partial \mathcal{L}}{\partial a_m^{(L)}} = \frac{\partial \mathcal{L}}{\partial y_m} \frac{\partial y_m}{\partial a_m^{(L)}} = \frac{\partial \mathcal{L}}{\partial y_m} g'(a_m^{(L)}). \quad (1.17)$$

where $y_m := g(a_m^{(L)})$. Note that the derivative of the non-linear function $g(\cdot)$ and the derivative of the loss function depend on the activation function and loss function that we choose.

Backpropagation algorithm

1. Execute a forward pass to obtain activations and outputs
2. Compute $\delta_m^{(L)} = \frac{\partial \mathcal{L}}{\partial y_m} g'(a_m^{(L)})$ for the specific loss and output function
3. Compute $\delta_j^{(\ell)} = f'(a_j^{(\ell)}) \sum_k \delta_k^{(\ell+1)} w_{k,j}^{(\ell+1)}$ with a backward pass
4. Obtain the gradients for all weights/biases

$$\frac{\partial L}{\partial w_{j,i}^{(\ell)}} = \delta_j^{(\ell)} o_i^{(\ell-1)} \quad \frac{\partial L}{\partial b_j^{(\ell)}} = \delta_j^{(\ell)}$$

1.5 Activation functions

We introduce the most common activation function used nowadays.

1.5.1 Sigmoid Activation

Very used in the past.

$$o_j^{(\ell)} := f(a_j^{(\ell)}) = \frac{1}{1 + \exp(-a_j^{(\ell)})} \quad (1.18)$$

Main features:

- Maps any real input to $[0, 1]$.
- Has a probabilistic interpretation and easy derivative (needed for the backpropagation algorithm):

$$f'(z) = f(z)(1 - f(z))$$

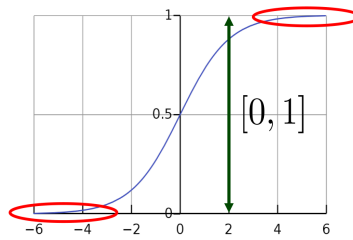


Figure 1.12: Sigmoid function and the flat regions.

Main problems:

- The derivative is **contractive** since its absolute value $\in (0, 1]$. Since the value is small, from 1.16 we can see that, if $f'(a_j^{(\ell+1)})$ is small then $\delta_j^{(\ell+1)}$ will be also small, but also $\delta_j^{(\ell)}$ will be small due to the recursive definition.
- Suffers from **vanishing gradients**: derivative becomes small for large or small inputs (flat regions). From 1.16 we can see that, if $f'(a_j^{(\ell)}) \rightarrow 0$ then $\delta_j^{(\ell)} \rightarrow 0$ therefore the error message backpropagates getting smaller and smaller (small update on the weights).
- Suffers from **bias drift**: outputs are not zero-centered, leading to bias drift in deep networks. Many layers sees a bias in its input and adds a bias itself. After stacking many layers this slows down the training, so it's better to use zero-centered output function.

1.5.2 Hyperbolic Tangent (tanh)

It's the scaled version of sigmoid with zero-centered output.

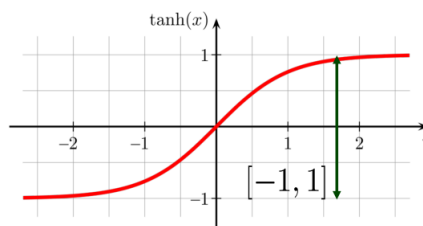


Figure 1.13: Tanh.

Main features:

- Avoids bias drift but still suffers from vanishing gradients.
- Derivative is contractive, leading to flat regions.

1.5.3 Linear Activation

Easiest derivative expression but useless in NN training since the composition of linear functions is still linear.

1.5.4 Rectified Linear Unit (ReLU)

Basis of modern neural networks.

$$\text{ReLU}(z) = \max(0, z) \quad (1.19)$$

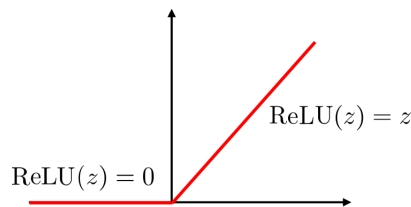


Figure 1.14: ReLU.

Main features:

- Non-linear, accelerates convergence, and has sparse outputs.
- Can be interpreted as "active" (when $z > 0$), or "inactive" (when $z \leq 0$).
- Mitigates vanishing gradients but can suffer from **dying ReLU** problem (flat gradient region): neurons can become inactive and never recover. It can happen when the bias value is too negative. We need to monitor the percentage of dead neurons during training.
- Output is not zero-centered, leading to **bias drift**.

1.5.5 Leaky ReLU

Addresses dying ReLU problem.

$$\text{LReLU}(z) = \begin{cases} \alpha z, & z < 0 \\ z, & z \geq 0 \end{cases} \quad (1.20)$$

Main features:

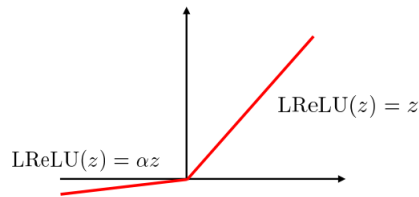


Figure 1.15: Leaky ReLU.

- Non-zero output for negative inputs, but loses sparsity (no inactive neurons, so no fast output).
- Can amplify noise for large negative values (increases the magnitude).

1.5.6 Exponential Linear Unit (ELU)

Tackles flatness and noise issues. It's a compromise between the ReLU and the Leaky ReLU.

$$\text{ELU}(z) = \begin{cases} \alpha(\exp z - 1), & z < 0 \\ z, & z \geq 0 \end{cases} \quad (1.21)$$

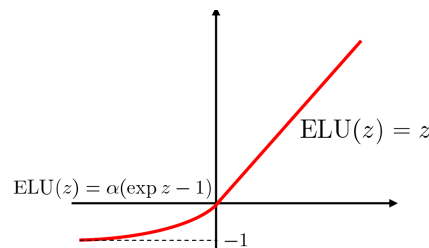


Figure 1.16: ELU.

Main features:

- Saturates for large negative inputs, making it robust to noise.
- More complex to compute and differentiate, potentially slower.
- Almost solves the dying ReLU.

1.6 Parameters initialization

Bad initialization of parameters can make learning extremely difficult (e.g. initialize the weights where the derivative of the loss is flat). Since it is hard to compute a good starting

point analytically, the initialization strategies are random and heuristic.

In general the weights should be initialized randomly to ensure **diversity**, also large values should be avoided since they lead to instability and numerical issues. Biases can be initialized for instance zero, or any other value.

Common choices:

- Zero-mean Gaussian distribution:

$$w_{j,i}^{(0)} \sim \mathcal{N}(0, \beta) \quad (1.22)$$

- Uniform distribution depending on the layer input size:

$$w_{j,i}^{(0)} \sim \mathcal{U}\left(-\frac{1}{\sqrt{d}}, \frac{1}{\sqrt{d}}\right) \quad (1.23)$$

- Glorot initialization:

$$w_{j,i}^{(0)} \sim \mathcal{U}\left(-\sqrt{\frac{6}{d+n}}, \sqrt{\frac{6}{d+n}}\right) \quad (1.24)$$

where $d := \text{input size}$, $n := \text{output size}$. This is currently the default in most DL frameworks.

1.7 Hyper-parameters

The FFNN hyper-parameters are not the NN parameters. The hyper-parameters regard only the structure of the NN. Examples: the learning rate η , the mini-batch size, the size of each layer, the number of layers, etc. We can choose the value of this hyper-parameters by using a *grid search*. In standard implementations they're not learned and their selection is usually non-trivial and requires experimentation.

1.7.1 Learning rate

If it is too large it may leads to overshooting the local minimum, if it is too small it may leads extremely slow convergence. Another choice is to use an adaptive learning rate. We will see later in the course how to define adaptive learning rates.

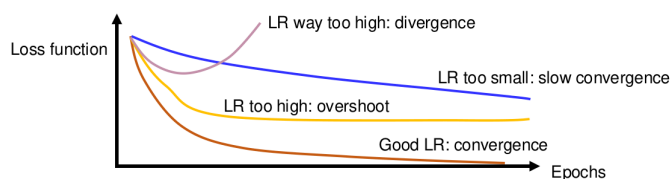


Figure 1.17: Learning rate.

1.7.2 Mini-batch size

If it is too large it may not fit into the memory of the machine, if it is too small the SGD algorithm become noisy and slow (e.g. if you use one single sample). Good practical rules: use the largest that can fit into memory; start small and increasing batch size as training progresses.

1.7.3 Number of layers and their size

If we add many layers and many neurons, than we have a huge number of parameters to learn, so the training is slow and difficult. Also it's very likely that the network will suffer from **overfitting**. If instead we use only a few number of layers and a small number of neurons, then there are too few parameters. Also in this case the training does not converge and it is likely that the network will suffer from **underfitting**.

1.7.4 Validation set for Hyper-parameters tuning

In order to tune the hyper-parameters we use a validation set. Usually we use a grid search and, if possible, we use the k-fold cross validation for model selection.

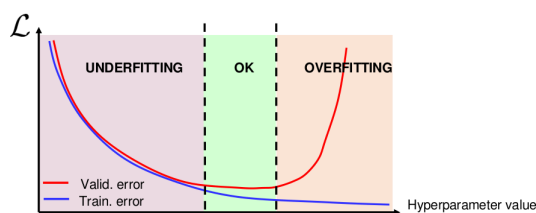


Figure 1.18: Tuning the hyper-parameters with a validation set.

Chapter 2

Convolutional Neural Networks

2.1 Limitations of the FFNN

FFNNs, in their basic formulation, have great limitations:

- **Excessively large number of parameters:** many weights to learn and much memory required to store all of them.
- **Structural information in input is lost:** if our input is a 2D image, than we can fed it to a FFNN by vectorizing it. However in this manner we lose the structural information of the image: pixel that are very close in the image can be far in 1D.

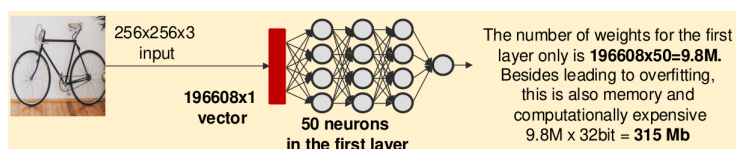


Figure 2.1: Problem 1.

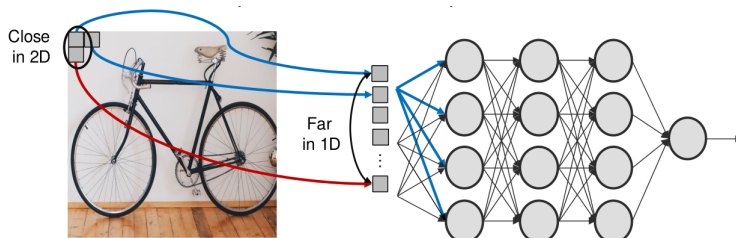


Figure 2.2: Problem 2.

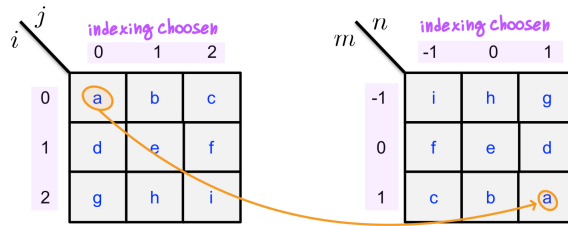
2.2 2D convolution for finite-length signals

The standard discrete convolution in 2D for finite-length signals is computed as follows:

$$(s * w)(i, j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} s(i-m, j-n)w(m, n) \quad (2.1)$$

- s is usually the input signal, in CNNs is a 2D image.
- w is usually the filter signal, in CNNs is called **kernel** (a $M \times N$ filter).
- $(s * w)$ is usually the output signal, in CNNs is the output 2D map.

Both the input image and the kernel are defined over a finite set of indices and zero outside of it. Note that the input image and the kernel can also be in 1D.



- Cells with the same letter **are multiplied together**

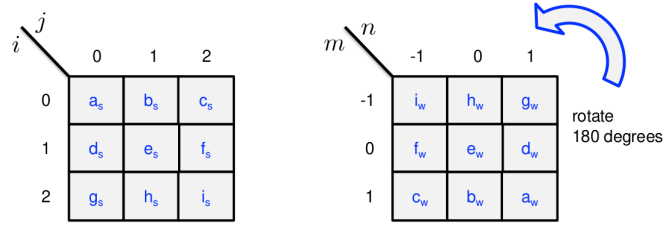
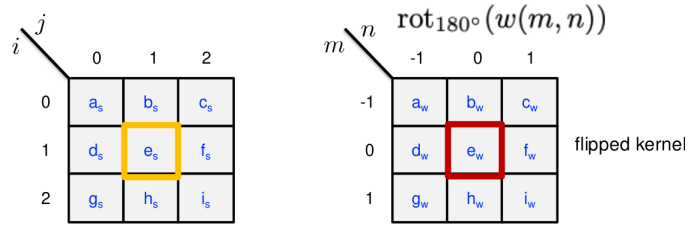
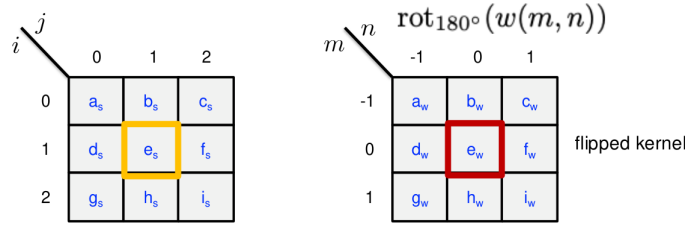
$$\begin{aligned} o(1, 1) = & \boxed{s(0, 0)w(1, 1)} + s(0, 1)w(1, 0) + s(0, 2)w(1, -1) + \\ & + s(1, 0)w(0, 1) + s(1, 1)w(0, 0) + s(1, 2)w(0, -1) + \\ & + s(2, 0)w(-1, 1) + s(2, 1)w(-1, 0) + s(2, 2)w(-1, -1) \end{aligned}$$

Figure 2.3: Convolution: computation of the element $o(1, 1) = (s * w)(1, 1)$ of the output map.

2.2.1 Convolution: 3 step procedure

From what we have discovered in the previous example, we can compute the convolution between the input image and the kernel, using a 3 step procedure. In the example below we consider again the computation of the element $o(1, 1) = (s * w)(1, 1)$ of the output map.

In order to compute all the other elements of the output map, we can apply the same approach. Note that, if we align the kernel cell $(0, 0)$ to the input cell $(0, 0)$, then the element of the kernel that do not perfectly overlap with the input are multiplied by 0. We will see this in more detail later on.


 Figure 2.4: Step 1: **kernel rotation**.

 Figure 2.5: Step 2: **kernel alignment**. The kernel cell with coordinates $(0, 0)$ overlaps the input cell with coordinates $(1, 1)$.

 Figure 2.6: Step 3: **compute the output**. $o(1, 1) = a_s a_w + b_s b_w + c_s c_w + \dots$

2.2.2 Convolution implementation for CNNs

Recall the definition of the 2D convolution for finite-length signals seen in a previous subsection:

$$(X_q * K_q)(i, j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} X_q(i - m, j - n) K_q(m, n) \quad (2.2)$$

where X_q is the input image and K_q is the kernel (we now use ML terminology). CNNs definition of convolution is different from what we have seen so far:

$$(X_q \otimes K_q)(i, j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} X_q(i + m, j + n) K_q(m, n). \quad (2.3)$$

The formula above is called **cross-correlation**, and this is the formula used in the CNNs. But why they implement cross-correlation instead of the real convolution? Because the

cross-correlation is faster and reduces the variability of the indices.

Relation between convolution and cross-correlation In the previous subsection, the Step 3 that we have performed to compute the value $o(1, 1)$ was actually a cross-correlation operation. So there exists a relation between the two operations:

$$(X_q * K_q)(i, j) = (X_q \otimes K'_q)(i, j), \quad K'_q(i, j) := \text{rot}_{180^\circ}(K_q(i, j)) \quad (2.4)$$

Observation 1. *Standard convolution is equivalent to the cross-correlation operation between the input image and a flipped kernel.*

Note two important things:

1. In the 2.3, the **rotation is not performed**: if you train a CNN by performing the rotation of the kernel and then you train another identical CNN but in this case you don't rotate the kernel, then at the end of the training you can see that the two CNNs have learned the same kernels. The only difference is that the first CNN has the rotated version of the kernels of the second CNN.
2. **Abuse of notation**: we say that the CNNs perform a convolution operation but in reality it is a cross-correlation operation. We will also use the symbol $*$ instead of $\otimes$.

2.3 CNN structure

2.3.1 Layer overview

CNN layer replace the matrix multiplication with the convolution.

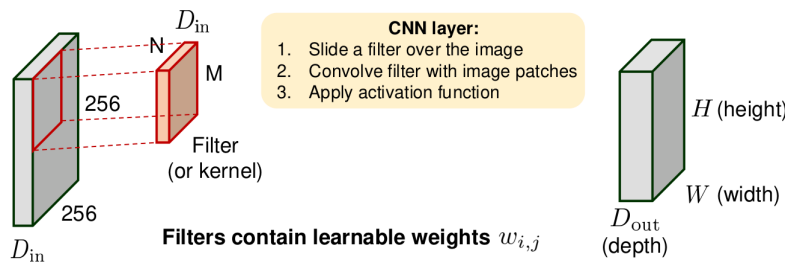


Figure 2.7: The third dimension of the input D_{in} must be equal to the third dimension of the kernel. The activation function is applied element-wise. In this case the number of parameters of the convolutional $N \times M \times D_{in}$ (without considering the bias).

The 2.3 is the definition of convolution in CNNs for 1 feature map, but in CNNs we need to move also along the depth dimension. Let us indicate $x_{i,j,q}$ an element of the input *tensor* (same apply for filter). The value q moves along the depth.

$$(X_q * K_q)(i, j) \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{i+m, j+n} w_{m,n,q} \quad (2.5)$$

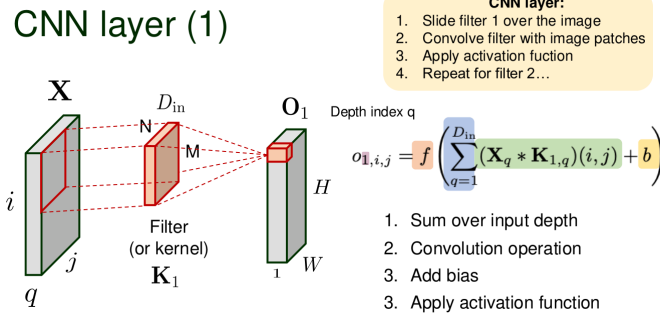


Figure 2.8: Application of one filter. If $D_{in} = 1$ then we just apply the convolution that we have introduced in the previous sections. Note that the output O_1 has depth 1 since we have applied only one convolution with the first filter.

In general, at every CNN layer, D_{out} filters are used (**output volume**). Each one generate a different **activation map**. How many filters can we use? It's hyperparameter that we need to fix that determinates the depth of the next layer. After we have computed the activation map, we apply a non-linear activation function element wise in order to obtain an output **feature map** (O_1 in the figure). In the end we get D_{out} feature maps that are fed as input to the next layer.

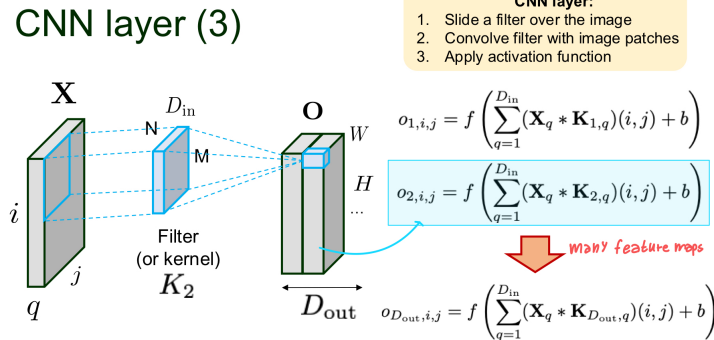


Figure 2.9: We get D_{out} feature maps (number of filters applied = depth of the output volume O).

It is a sequence of **convolutions** interspersed with **activation functions (ReLU)**

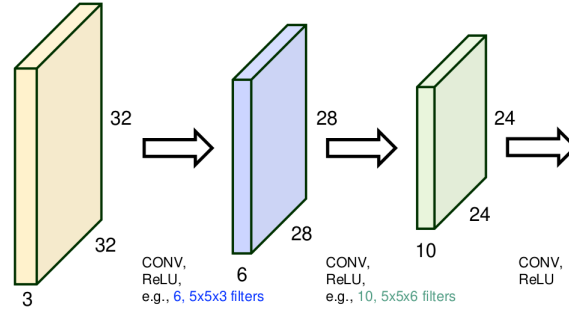


Figure 2.10: CNN volume output layer per layer.

Note that, after many layer, the output volume becomes smaller and smaller. This is a problem because it implies that we cannot create CNNs with many layers: after some layers the output volume reaches $1 \times 1 \times 1$. In the following section we will see how to deal with this.

2.3.2 Output size vs filter size and stride

Definition 2 (Feature map). *Result of convolving one filter of the layer with the whole input and applying the activation function (element-wise)*

Definition 3 (Stride). *Number of steps by which the filter is shifted (horizontally and vertically)*

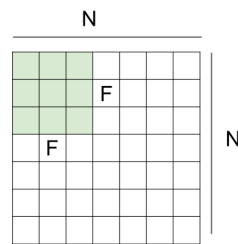


Figure 2.11: Input.

The output volume ($O \times O$) is computed as:

$$O = \frac{N - F}{\text{stride}} + 1. \quad (2.6)$$

The output volume O is usually smaller than the input volume and it also decreases with an increasing stride.

Definition 4 (Padding). *Add zeros at the edge of the input tensor so that filters fit an integer number of times.*

It is common to apply zero-padding to the borders.

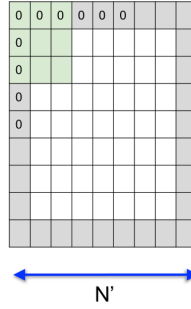


Figure 2.12: Padding applied to the input. Note that it is applied symmetrically on all side (each dimension increases by 2).

Why is the padding useful? By using it, the output size can be the same as the input size (without the padding). It is easy to see. Consider $\text{stride} = 1$, let $F \times F$ the size of the filter and add $(F - 1)/2$ zero-padding to the input of size N . Compute the output size:

$$N' = N + 2((F - 1)/2) = N + F - 1$$

$$O = \frac{N' - F}{\text{stride}} + 1 = N$$

so the output has size $N \times N$.

The final formula for computing the output dimension is the following one:

$$O = \frac{N - F + 2P}{S} + 1 \quad (2.7)$$

where N is the input dimension, P is the padding F is the filter size, S is the stride. For the 2D we need to apply the formula for each dimension separately.

2.4 FFNN vs CNN

2.4.1 Local connectivity

The usage of the convolution makes the connections with subsequent layer sparse and *local*. As a result, CNNs are much more efficient and fast than *dense* FFNN.

- FFNN: each neuron of a layer depends on all neurons of previous layer.
- CNN: each neuron of a layer does only depend on a small number of neurons from the previous layer.

In CNNs the patterns are learned in a hierarchical fashion: layer i learns local patterns of layer $i - 1$. By stacking multiple layers we can learn global patterns.

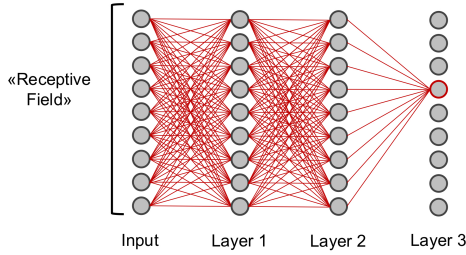


Figure 2.13: FFNNs receptive field.

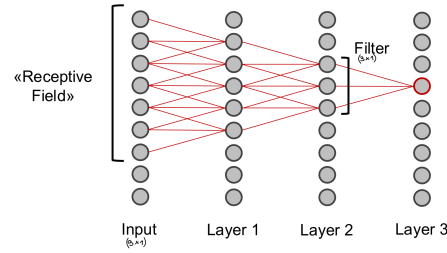


Figure 2.14: CNNs receptive field.

In a CNN, due to how the convolution is defined, many arcs do not exist. In practice we observe that, in a CNN, thanks to this particular structure, the neurons closer to the input only depend on **local features** that are typically small-scale simple patterns (like edges or simple shape). Instead, deeper neurons depend on **global features** that are more complex/large patterns (like eyes, arms, wheels).

2.4.2 Local weights

Since each filter is slid across the whole input keeping the same parameters, then it means that many arcs share the same weights. As we have already said, this is useful to recognize local patterns in the data, but also it's useful in order to reduce the number of total parameters that the CNN needs to learn. Reducing the number of parameters implies that our model is less likely to incur in overfitting, it needs less memory and also we can build more deeper networks that can learn more complex features.

2.4.3 Translation equivariance

Definition 5 (Equivariance). *Given two functions $\phi(\cdot)$ and $\xi(\cdot)$, and an input x . If it holds*

$$\phi(\xi(x)) = \xi(\phi(x)).$$

than ψ is equivariant w.r.t. ξ .

In our case, the $\phi(\cdot)$ is the *convolution function* and the $\xi(\cdot)$ is the *translation* of the input. In CNNs it holds that the convolution is equivariant w.r.t. translation. In other words it means that the features learned are independent from the position in the input: if we shift a certain pattern in the input, than the corresponding learned feature will be shifted too, but it will still be there unchanged.

This is **very useful**: makes CNNs robust to recognizing the same objects/patterns in different locations of images. This is not true for FFNN: if we want to classify images of cat, than the network can respond in different way if the cat is on the left corner or in the middle (for example).

Note that: in CNNs introduced so far, the convolution is *not* equivariant to rototranslations (we need to use more complex architectures).

2.5 Pooling layers

One way to reduce the dimensionality of the tensors between layer is to use a value of the stride $S > 1$. Another way for reducing the dimensionality is to use a pooling layer.

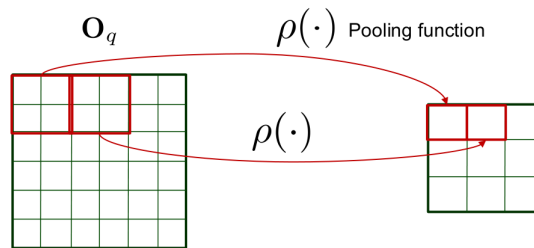


Figure 2.15: Pooling function.

Typical pooling functions are $\max(\cdot)$, $\text{mean}(\cdot)$, the region has a size (say) $R \times R$ (hyper-parameter) with a stride of R . Note that the pooling is just a function, there aren't any parameters to learn.

Note that: pooling introduces **invariance** to small translations. It means that a pattern in the input does not change location in the output if a small translation is applied. This is useful if we're considering for instance a classification task, in which we don't care about the position of the object, we only want to know if a certain object is in the image or not.

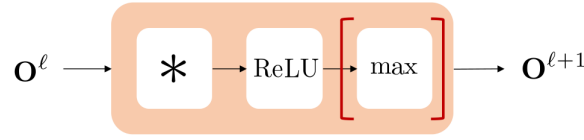


Figure 2.16: Convolution (linear) $\rightarrow$ Activation function (non-linear) $\rightarrow$ Pooling (dimensionality reduction) - optional.

2.6 Backpropagation for CNNs

2.6.1 Notation

- Layers indexed by $\ell = 1, 2, \dots, L$
- Input feature map $H \times W$ indices i, j
- Filter $M \times N$ indices m, n (all filters have same dim for simplicity)
- Generic weight between layers $\ell - 1, \ell$ is $w_{m,n}^\ell$, bias b^ℓ
- Activation (conv. of previous output and filter) (we use only one kernel for simplicity)

$$a_{i,j}^\ell = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} o_{i+m,j+n}^{\ell-1} w_{m,n}^\ell + b^\ell$$

- Output

$$o_{i,j}^\ell = f(a_{i,j}^\ell)$$

2.6.2 Backpropagation derivation

We want derive an expression for the computation of the gradient of the loss function w.r.t. a weight, like we have done for the FFNN. In CNNs usually the last layer is a feedforward layer: convolutional layers learn the features instead the FF layer is used at the output to classify/regress. To achieve this, the output of the last convolutional layer is *flattened*.

For simplicity, we fix the value of the stride $S = 1$, the padding $P = 0$ (so the feature map is always smaller than the input), the depth of the input D_{in} (= depth of the kernel) and the depth of the output $D_{out} = 1$ (so we use only one kernel).

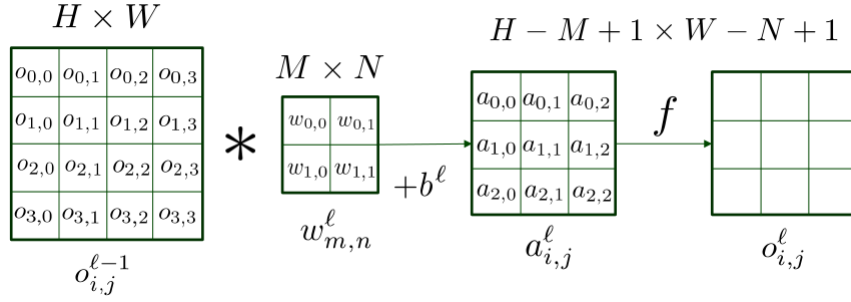


Figure 2.17: Convolutional layer structure: convolution + bias and non-linear activation function.

As we have done for the derivation of the backpropagation for FFNNs, the first information that we need is provided by the **forward pass**: we have to propagate the input from the first layer of the CNN to the last one.

Our aim is to compute the gradient of the loss w.r.t. weights $\frac{\partial \mathcal{L}(\mathcal{W}, \mathbf{X})}{\partial w_{m,n}^{(\ell)}}$. So we need to understand how, one single weight, influences the total loss function.

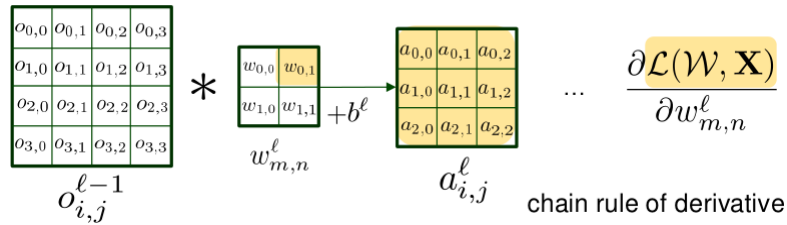


Figure 2.18: How the weight $w_{m,n}^{(\ell)}$ influences the total loss. In FFNN one weight affected all the activations $a_{i,j}^{(\ell)}$, in CNN no.

Therefore we can use the chain rule by considering only the affected activation:

$$\frac{\partial \mathcal{L}(\mathcal{W}, \mathbf{X})}{\partial w_{m,n}^\ell} = \sum_{i=0}^{H-M} \sum_{j=0}^{W-N} \frac{\partial \mathcal{L}(M, \mathbf{X})}{\partial a_{i,j}^\ell} \frac{\partial a_{i,j}^\ell}{\partial w_{m,n}^\ell} \quad (2.8)$$

We can define the error signal as:

$$\delta_{i,j}^{(\ell)} := \frac{\partial \mathcal{L}(\mathcal{W}, \mathbf{X})}{\partial a_{i,j}^\ell} \quad (2.9)$$

and by substituting in the previous equation we get:

$$\frac{\partial L(\mathcal{W}, \mathbf{X})}{\partial w_{m,n}^\ell} = \sum_{i=0}^{H-M} \sum_{j=0}^{W-N} \delta_{i,j}^{(\ell)} \frac{\partial a_{i,j}^\ell}{\partial w_{m,n}^\ell} \quad (2.10)$$

We have that $a_{i,j}^{(\ell)} := \sum_{m'=0}^{M-1} \sum_{n'=0}^{N-1} o_{i+m',j+n'}^{\ell-1} w_{m',n'}^\ell + b^\ell$ therefore it follows:

$$\frac{\partial a_{i,j}^\ell}{\partial w_{m,n}^\ell} = o_{i+m,j+n}$$

since only the output with the same indices as $w_{m,n}$ survives (i.e., when $m' = m$ and $n' = n$).

By substituting the above in the 2.10 we get the following:

$$\frac{\partial \mathcal{L}(\mathcal{W}, \mathbf{X})}{\partial w_{m,n}^\ell} = \sum_{i=0}^{H-M} \sum_{j=0}^{W-N} \delta_{i,j}^\ell o_{i+m,j+n}^{\ell-1} \quad (2.11)$$

We can clearly see that the above is actually a convolution ("cross-correlation"), where $\delta_{i,j}^\ell$ plays the role of the input and $o_{i+m,j+n}^{\ell-1}$ is the filter.

$$\frac{\partial \mathcal{L}(\mathcal{W}, \mathbf{X})}{\partial w_{m,n}^\ell} = \boldsymbol{\delta}^\ell * \mathbf{O}_{m,n}^{\ell-1} \quad (2.12)$$

where in this case $\boldsymbol{\delta}^\ell$ is a matrix (same size of the yellow block in the previous picture).

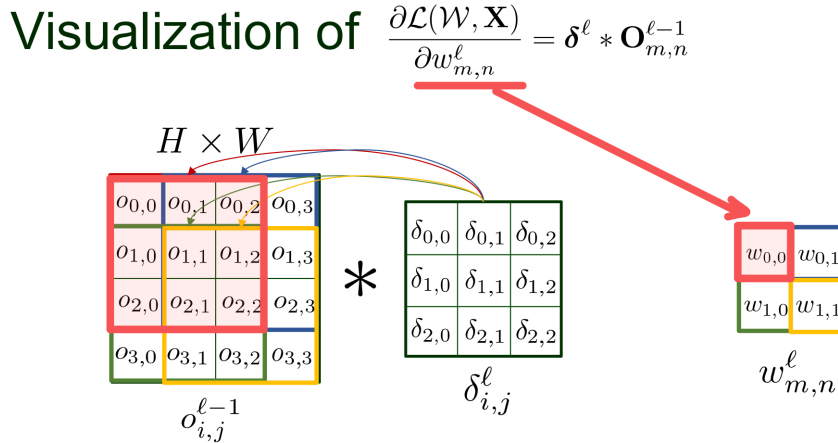


Figure 2.19: The matrix $\mathbf{O}_{m,n}^{\ell-1}$ depends on the weight $w_{m,n}^\ell$.

The rule is the following: the top left corner of the matrix $\boldsymbol{\delta}^\ell$ (the filter) must overlap the element (m,n) of the matrix $\mathbf{O}_{m,n}^{\ell-1}$. Then we compute the convolution as we know.

Now we only need to find a recursive expression that allow us to compute the error signal starting from the last layer of the network. This is more complex than for FFNNs since CNNs are not fully connected.

We know that $\delta_{i,j}^{(\ell)} := \frac{\partial \mathcal{L}(\mathcal{W}, \mathbf{X})}{\partial a_{i,j}^\ell}$. We need to understand what values influence $a_{i,j}^\ell$ (its dependencies).

Tracking dependencies (1)

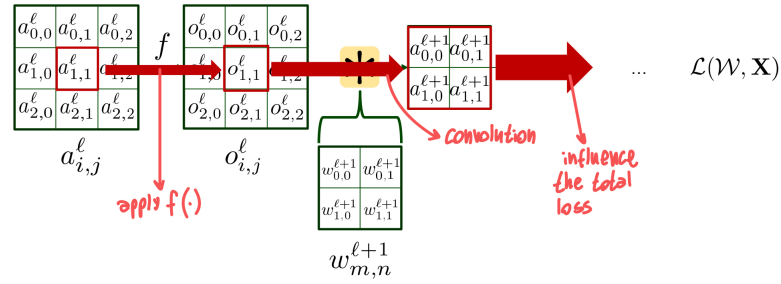


Figure 2.20

The relation between $a_{i,j}^\ell$ and $o_{i,j}^\ell$ is just the non-linear function $f(\cdot)$. So let's focus on the dependencies of $o_{i,j}^\ell$.

- Let's track the dependencies on $o_{1,1}^\ell$ from $\ell + 1$ to ℓ

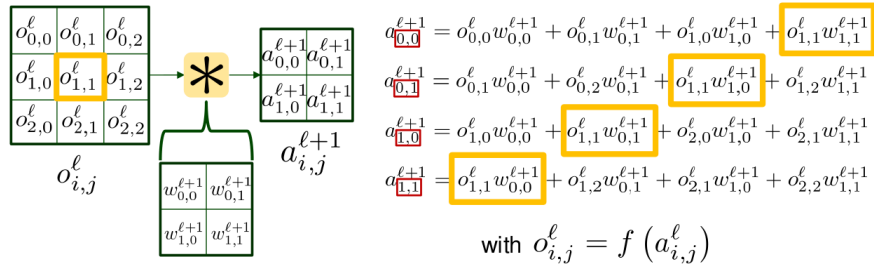


Figure 2.21: Note: we're omitting the bias.

Consider $o_{1,1}^\ell$, from the picture we can see that the activations influenced are:

$$a_{1-n,1-m}^{\ell+1}, \quad n \in \{0, \dots, N-1\}, \quad m \in \{0, \dots, M-1\}$$

• Let's track the dependencies on $o_{1,1}^\ell$ from $\ell + 1$ to ℓ

$$\begin{aligned}
 a_{0,0}^{\ell+1} &= o_{0,0}^\ell w_{0,0}^{\ell+1} + o_{0,1}^\ell w_{0,1}^{\ell+1} + o_{1,0}^\ell w_{1,0}^{\ell+1} + o_{1,1}^\ell w_{1,1}^{\ell+1} \\
 a_{0,1}^{\ell+1} &= o_{0,1}^\ell w_{0,0}^{\ell+1} + o_{0,2}^\ell w_{0,1}^{\ell+1} + o_{1,1}^\ell w_{1,0}^{\ell+1} + o_{1,2}^\ell w_{1,1}^{\ell+1} \\
 a_{1,0}^{\ell+1} &= o_{1,0}^\ell w_{0,0}^{\ell+1} + o_{1,1}^\ell w_{0,1}^{\ell+1} + o_{2,0}^\ell w_{1,0}^{\ell+1} + o_{2,1}^\ell w_{1,1}^{\ell+1} \\
 a_{1,1}^{\ell+1} &= o_{1,1}^\ell w_{0,0}^{\ell+1} + o_{1,2}^\ell w_{0,1}^{\ell+1} + o_{2,1}^\ell w_{1,0}^{\ell+1} + o_{2,2}^\ell w_{1,1}^{\ell+1}
 \end{aligned}$$

with $o_{i,j}^\ell = f(a_{i,j}^\ell)$

$o_{1,1}^\ell = f(a_{1,1}^\ell)$

Influences (is connected with) the following variables:

$a_{0,0}^{\ell+1}, a_{0,1}^{\ell+1}$

$a_{1,0}^{\ell+1}, a_{1,1}^{\ell+1}$

Figure 2.22: Note: we're omitting the bias.

In general, we can define a rule for the indices. The activations influenced by $o_{i,j}^\ell$ are

$$a_{i-m,j-n}^{\ell+1}, \quad m \in \{0, 1\}, \quad n \in \{0, 1\} \quad (2.13)$$

and in order for the resulting indices to valid it must hold that:

$$i - m \in \{0, \dots, H - M + 1\}, \quad j - n \in \{0, \dots, W - N + 1\} \quad (2.14)$$

that, is the indices must be inside the coordinates of the activation map.

We have found which values are influenced by $a_{i,j}^\ell$, so we can finally use the chain rule.

$$\begin{aligned}
 \delta_{i,j}^\ell &:= \frac{\partial \mathcal{L}(\mathcal{W}, X)}{\partial a_{i,j}^\ell} = \sum_m \sum_n \frac{\partial \mathcal{L}(\mathcal{W}, X)}{\partial a_{i-m,j-n}^{\ell+1}} \frac{\partial a_{i-m,j-n}^{\ell+1}}{\partial a_{i,j}^\ell} \\
 &= \sum_m \sum_n \delta_{i-m,j-n}^{\ell+1} \frac{\partial a_{i-m,j-n}^{\ell+1}}{\partial a_{i,j}^\ell}
 \end{aligned} \quad (2.15)$$

note that the constraint on the indices must be verified.

We are almost done. We need to compute $\frac{\partial a_{i-m,j-n}^{\ell+1}}{\partial a_{i,j}^\ell}$.

We know that $a_{i-m,j-n}^{\ell+1} = \sum_{m'=0}^{M-1} \sum_{n'=0}^{N-1} w_{m',n'}^{\ell+1} f(a_{i-m+m',j-n+n'}^\ell) + b^{\ell+1}$.

The only element that survives in the derivative is the one with $m' = m$ and $n' = n$ because in that case we have that $(i - m + m', j - n + n')$ matches (i, j) of the considered activation.

Therefore:

$$\frac{\partial a_{i-m,j-n}^{\ell+1}}{\partial a_{i,j}^{\ell}} = w_{m,n}^{\ell+1} f'(a_{i,j}^{\ell})$$

So in the end:

$$\begin{aligned} \delta_{i,j}^{\ell} &:= \frac{\partial \mathcal{L}(\mathcal{W}, X)}{\partial a_{i,j}^{\ell}} = \sum_m \sum_n \delta_{i-m,j-n}^{\ell+1} w_{m,n}^{\ell+1} f'(a_{i,j}^{\ell}) \quad \text{Real convolution!} \\ &= f'(a_{i,j}^{\ell}) \boldsymbol{\delta}^{\ell+1} * \text{rot}_{180^\circ}(\mathbf{K}^{\ell+1}) \end{aligned} \quad (2.16)$$

For the biases the derivation is easy. We have one bias for each kernel. Bias b^{ℓ} influences all the activation values $a_{i,j}^{\ell}$.

$$\begin{aligned} \frac{\partial \mathcal{L}(\mathcal{W}, \mathbf{X})}{\partial b^{\ell}} &= \sum_{i=0}^{H-M} \sum_{j=0}^{W-N} \frac{\partial \mathcal{L}(\mathcal{W}, \mathbf{X})}{\partial a_{i,j}^{\ell}} \frac{\partial a_{i,j}^{\ell}}{\partial b^{\ell}} \\ &= \sum_{i=0}^{H-M} \sum_{j=0}^{W-N} \delta_{i,j}^{\ell} \underbrace{\frac{\partial a_{i,j}^{\ell}}{\partial b^{\ell}}}_{=1} \\ &= \sum_{i=0}^{H-M} \sum_{j=0}^{W-N} \delta_{i,j}^{\ell} \end{aligned} \quad (2.17)$$

Backpropagation for CNNs:

1. Apply a forward pass to compute all the $a_{i,j}^{\ell}, o_{i,j}^{\ell}$
2. Last layer is usually dense, compute error signal as for FFNNs (check FFNN derivation) $\boldsymbol{\delta}^L$.

Note that we need to perform a *reshape* of the vector (from vector to matrix) in order to use it in the CNN layers.

3. Recursively compute the error signal matrices

$$\delta_{i,j}^{\ell} = f'(a_{i,j}^{\ell}) \boldsymbol{\delta}^{\ell+1} * \text{rot}_{180^\circ}(\mathbf{K}^{\ell+1})$$

4. Compute the gradients using the error signal matrices

$$\frac{\partial \mathcal{L}(\mathcal{W}, \mathbf{X})}{\partial w_{m,n}^{\ell}} = \boldsymbol{\delta}^{\ell} * O_{m,n}^{\ell-1}, \quad \frac{\partial \mathcal{L}(\mathcal{W}, \mathbf{X})}{\partial b^{\ell}} = \sum_{i=0}^{H-M} \sum_{j=0}^{W-N} \delta_{i,j}^{\ell}$$

2.6.3 Pooling during backpropagation

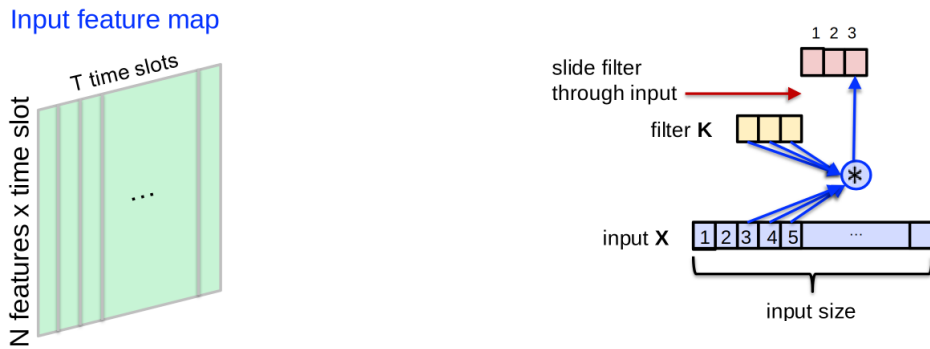
The pooling operates on patches of the input. We need to distinguish two cases:

- **Max pooling:** the error signal is assigned to the max value in the patch, as it is responsible for the change in the output (the other values do not affect the total loss, only the maximum does).
- **Average pooling:** the error signal is assigned to all the input units in the patch after dividing it by $R \times R$ with R being the dimension of the (square) pooling patch (since all the values affect proportionally the total loss).

2.7 Alternative convolution approaches

2.7.1 Implementing Causality

Consider the problem in which the input are N feature vectors, one vector per time slot $(t_1, t_2, \dots, t_T)$, for a total of T time slots. Consider a 1D kernel of 3 elements ($F = 3$).



We can see from the figure above that, by computing the 1D convolution, we do not take into account the **causality**: the output at time 3 depends on input entries at time 3, 4, 5. Instead what we want is the following: each value of a specific entry in the output must depends on all the previous entries in the input. The value should not depend on the values of the input that follow (into the feature).

How can we achieve this behaviour? We can add zero padding on the left side of the input sequence.

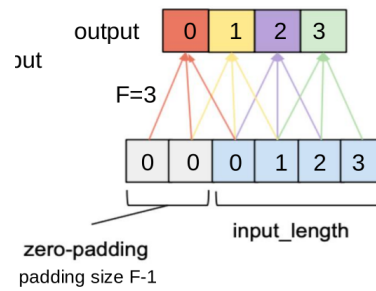


Figure 2.23: With zero-padding on the left-side of the input sequence we meet causality.

2.7.2 Dilation

Dilation in the context of a convolutional layer refers to the distance between the elements of the input sequence that are used to compute one entry of the output sequence.

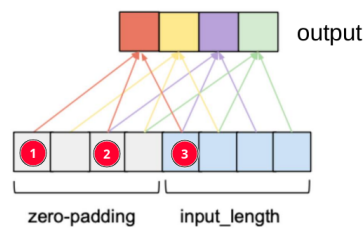


Figure 2.24: Example of a 2-dilated layer (dilation rate = 2) with input length = 4, kernel size = 3. Zero padding is added for causality.

2.7.3 Temporal causal dilated convolution

It consists in two features:

- Add padding on left side of input sequence to achieve causality and preserve the size at output.
- Pick a filter and convolve it with the input.
 - Independently convolve the same filter with each input feature vector.
 - Shared weights: the same filter weights applied to multiple input vectors.

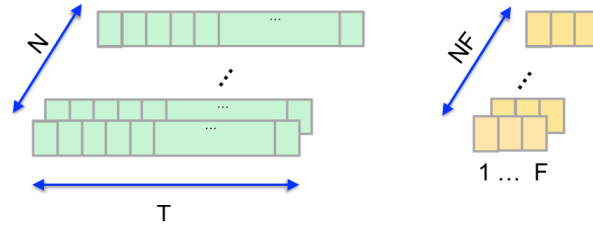


Figure 2.25: The input feature map and the kernels. (NF is the number of filters).

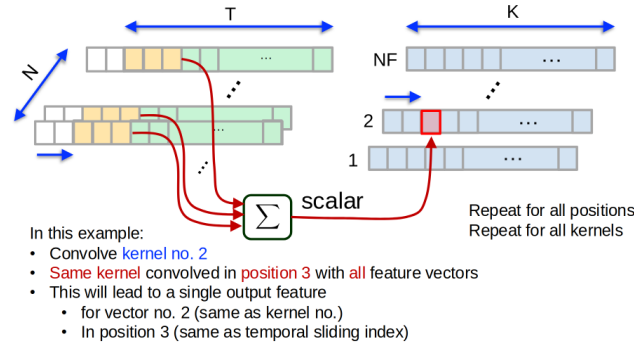


Figure 2.26: Temporal causal convolution. In output we get a vector for each filter applied.

2.7.4 Relation between dilation and receptive filed

Why the dilation is useful? Without the dilation, if the filter size is small, then we get a relative small receptive field. Instead with the dilation we can get a large receptive fields without adding any new layer. This is a desired result in many real world applications.

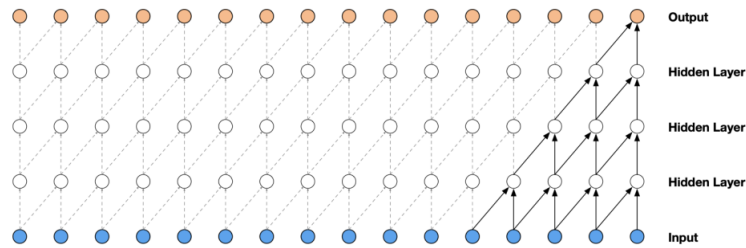


Figure 2.27: Receptive field: 3 convolutional layers, causal 1D convolution with a filter size = 2. Without the dilation, each output feature depends on only 5 subsequent input features.

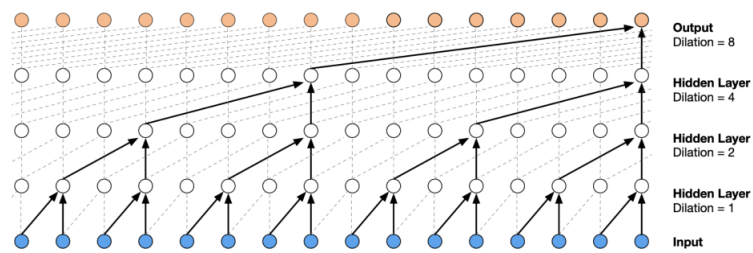


Figure 2.28: Receptive field and dilation: each output feature depends on all input feature. The value of the dilation used for each layer is computed in order to achieve this result.

Chapter 3

Automatic differentiation

In the two previous chapters we have seen a very effective way for computing the derivative of the loss function w.r.t. a weight of the network. This algorithm is called backpropagation. We have seen that the solution provided by the backpropagation can be applied to both FFNNs and CNNs. In general the procedure can be applied to any neural network structure. But why can't we just compute directly the partial derivative?

3.1 The general problem

The input of our problem are: the input data vector $\mathbf{x} \in \mathbb{R}^d$ and the set of weights $\mathcal{W}$. We're interested in computing the derivative of the loss function with respect to each weight of the network, so the loss actually is a function of the weight $\mathcal{L} : \mathbb{R}^{|\mathcal{W}|} \rightarrow \mathbb{R}^m$ (the weights are the variables). The value m is the number of nodes in output, in this general case we assume that we use a loss function for each different output neuron (general case). Therefore in the end we are interested in computing the derivative of each loss function w.r.t. each possible weight. In other words we're interested in computing the Jacobian:

$$J_{\mathcal{L}} = \begin{bmatrix} \frac{\partial \mathcal{L}_1}{\partial w_1} & \dots & \frac{\partial \mathcal{L}_1}{\partial w_{|\mathcal{W}|}} \\ \vdots & \ddots & \vdots \\ \frac{\partial \mathcal{L}_m}{\partial w_1} & \dots & \frac{\partial \mathcal{L}_m}{\partial w_{|\mathcal{W}|}} \end{bmatrix} \quad (3.1)$$

the dimensionality is $m \times |\mathcal{W}|$. If the case of a single loss function, the Jacobian becomes the gradient vector:

$$\nabla \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial w_1}, \dots, \frac{\partial \mathcal{L}}{\partial w_{|\mathcal{W}|}} \right)^T \quad (3.2)$$

3.2 Symbolic differentiation

Derivatives are expressed using symbolic equations that are propagated using algebra and calculus rules. (See example in the slides).

In the end we get a pretty well defined close formula.

Problems:

- The size of the expression becomes very large quickly.
- There are redundant terms.
- The formula is closed-form: this excludes for loops, if statement, recursion, etc (e.g. the pooling layer is expressed with an if statement).

3.3 Numerical differentiation

Consider the incremental ration approximation:

$$\frac{\partial \mathcal{L}(w)}{\partial w} \approx \frac{\mathcal{L}(w + \varepsilon) - \mathcal{L}(w)}{\varepsilon} + O(\varepsilon) \quad (3.3)$$

where $O(\varepsilon)$ is the round-off error and ε must be small.

Problems:

- Lowering ε we get a better precision but this leads to numerical instability.
- Requires two evaluation of the loss per weight.

For a better precision we can use the center difference approach:

$$\frac{\partial \mathcal{L}(w)}{\partial w} \approx \frac{\mathcal{L}(w + \varepsilon) - \mathcal{L}(w - \varepsilon)}{2\varepsilon} + O(\varepsilon^2) \quad (3.4)$$

But we have the same problem as above. The main problem of the two proposed approximated approach is that in NN small errors are propagated, so this leads to bad approximation.

3.4 Automatic differentiation

It avoids the drawbacks of both the previous approaches: no round-off error, the gradients are exact; the computation is extremely efficient. There exist two possible mode: **forward**

mode and **backward mode**.

3.4.1 Forward mode

Let us consider a toy example. Consider the following function:

$$f(x_1, x_2) = x_1 x_2 + e^{x_1 x_2} - \sin x_2$$

which can be represented with a *computational directed graph* (the representation is not unique).

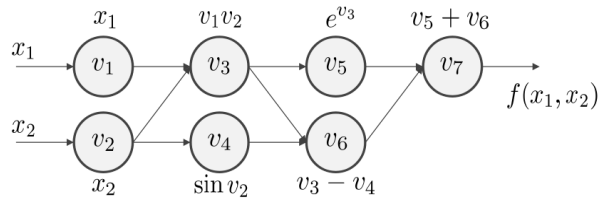


Figure 3.1: Representation of the function as a graph. Note that $v_7 = f(x_1, x_2)$.

We call each intermediate variable $v_1, \dots, v_7$ **primal variable**.

$$v_1 = x_1$$

$$v_2 = x_2$$

$$v_3 = v_1 v_2$$

$$v_4 = \sin v_2$$

$$v_5 = e^{v_3}$$

$$v_6 = v_3 - v_4$$

$$f(x_1, x_2) =: v_7 = v_5 + v_6$$

We use them to simplify the computation of the derivatives. Consider that we want to evaluate $\partial f / \partial x_1$. Let us define the **tangent variables** as:

$$\dot{v}_i := \frac{\partial v_i}{\partial x_1}$$

The tangent variables are recursively related:

$$\dot{v}_i := \frac{\partial v_i}{\partial x_1} = \sum_{j \in \text{pa}(i)} \frac{\partial v_i}{\partial v_j} \frac{\partial v_j}{\partial x_1} = \sum_{j \in \text{pa}(i)} \frac{\partial v_i}{\partial v_j} \dot{v}_j \quad (3.5)$$

$$\dot{v}_i = \frac{\partial v_i}{\partial x_1} = \sum_{j \in \text{pa}(i)} \frac{\partial v_i}{\partial v_j} \frac{\partial v_j}{\partial x_1}$$

Set of **parents** of node i
in the computational graph

Figure 3.2: Tangent variable are recursively related: by looking at the dependencies and using the chain rule.

Since (look at the graph) $j < i$, this suggest a forward procedure computed through the computational graph.

By using the equation above, starting from the input notes v_1, v_2 , we get the following results:

$$\begin{aligned} \dot{v}_1 &= 1 & (\text{Forward start}) \\ \dot{v}_2 &= 0 \\ \dot{v}_3 &= v_1 \dot{v}_2 + \dot{v}_1 v_2 = x_2 \\ \dot{v}_4 &= \dot{v}_2 \cos v_2 = 0 \\ \dot{v}_5 &= \dot{v}_3 e^{v_3} = x_2 e^{x_1 x_2} \\ \dot{v}_6 &= \dot{v}_3 - \dot{v}_4 = x_2 \\ \frac{\partial f}{\partial x_1} &=: \frac{\partial v_7}{\partial x_1} =: \dot{v}_7 = \dot{v}_5 + \dot{v}_6 = x_2 e^{x_1 x_2} + x_2 & (\text{Forward end}) \end{aligned}$$

With a single forward pass we get the full *evaluation trace* $(v_i, \dot{v}_i) \forall i$.

In the example above we have considered a case in which the computational graph has only one output. If we have for instance 2 outputs, than the tangent variables can be reused for each output node. This means that with a single forward pass, we compute the partial derivatives for each loss function f w.r.t. a single weight.

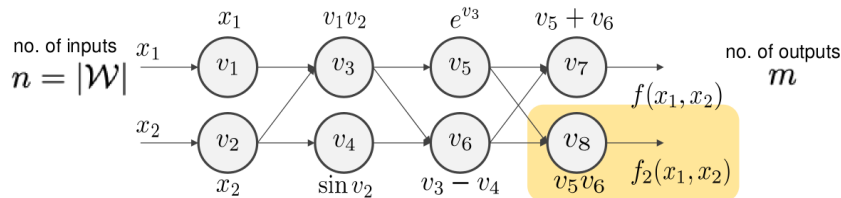


Figure 3.3: Computational graph with two outputs. With a single forward pass we compute $\partial f / \partial x_1$ and (reusing the tangent variables) $\partial f_2 / \partial x_1$.

Problem: using the forward mode, we need a full evaluation trace in order to compute the partial derivative $\partial f / \partial x_1$. In NNs we have $f = \mathcal{L} : \mathbb{R}^{|\mathcal{W}|} \rightarrow \mathbb{R}^m$, therefore, in order to compute all the entries of the Jacobian, we need to perform $|\mathcal{W}|$ forward passes (one for each weight of the network). Why this is a problem? In most of the cases the number of weights is much bigger than the number of loss functions used. However, if we're in the case in which $|\mathcal{W}| \ll m$, then the forward pass is computationally efficient.

$$\mathbf{J}_{\mathcal{L}} = \begin{bmatrix} \frac{\partial \mathcal{L}_1}{\partial w_1} & \cdots & \frac{\partial \mathcal{L}_1}{\partial w_{|\mathcal{W}|}} \\ \vdots & & \vdots \\ \frac{\partial \mathcal{L}_m}{\partial w_1} & \cdots & \frac{\partial \mathcal{L}_m}{\partial w_{|\mathcal{W}|}} \end{bmatrix} \quad \mathcal{L} : \mathbb{R}^{|\mathcal{W}|} \rightarrow \mathbb{R}^m$$

Full Jacobian requires $|\mathcal{W}|$ passes through the NN

Figure 3.4: A single forward pass produces a column of the Jacobian.

3.4.2 Backward mode

Let us consider the example used for the forward mode. In this case, we don't define the tangent variables, instead we define the so-called **adjoint variables**:

$$\bar{v}_i := \frac{\partial f}{\partial v_i}$$

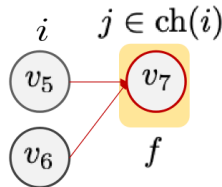


Figure 3.5: The adjoint variables are recursively related.

Similar to what we have seen for the tangent variables, also the adjoint variables are recursively related. By looking at the dependencies and using the chain rule, we can define the recursive expression:

$$\bar{v}_i := \frac{\partial f}{\partial v_i} = \sum_{j \in \text{ch}(i)} \frac{\partial f}{\partial v_j} \frac{\partial v_j}{\partial v_i} = \sum_{j \in \text{ch}(i)} \bar{v}_j \frac{\partial v_j}{\partial v_i} \quad (3.6)$$

Since (by looking at the picture above) $j < i$, the recursion move backward.

By using the equation above, starting from the output node v_7 , we get the following results:

$$\bar{v}_7 := \frac{\partial f}{\partial v_7} := \frac{\partial v_7}{\partial v_7} = 1 \quad (\text{Backward start})$$

$$\bar{v}_6 = \bar{v}_7 = 1$$

$$\bar{v}_5 = \bar{v}_7 = 1$$

$$\bar{v}_4 = -\bar{v}_6 = -1$$

$$\bar{v}_3 = \bar{v}_5 v_5 + \bar{v}_6 = e^{x_1 x_2} + 1$$

$$\frac{\partial f}{\partial x_2} := \frac{\partial f}{\partial v_2} := \bar{v}_2 = \bar{v}_3 v_1 + \bar{v}_4 \cos v_2 = x_1 + x_1 e^{x_1 x_2} - \cos x_2 \quad (\text{Backward end})$$

$$\frac{\partial f}{\partial x_1} := \frac{\partial f}{\partial v_1} := \bar{v}_1 = \bar{v}_3 v_2 = x_2 + x_2 e^{x_1 x_2} \quad (\text{Backward end})$$

Note that in the expression above, we need to compute the value of the primal variables v_1, v_2, v_5 . How can we do that? We need a *forward pass*¹ and we need to store the values in the memory.

Advantages: with a single forward pass and a single backward pass we get the full evaluation trace $(v_i, \bar{v}_i) \forall i$. By using the backward mode, with just one forward pass and one backward pass, we compute the partial derivatives $\partial f / \partial x_i$ for each input x_i of the computational graph. In NNs we have $f = \mathcal{L} : \mathbb{R}^{|\mathcal{W}|} \rightarrow \mathbb{R}^m$, therefore the backward mode let us compute the partial derivative of one loss function w.r.t. each weight in $\mathcal{W}$. In order to compute the full Jacobian, we need to perform m passes, one for each loss function.

$$\mathbf{J}_{\mathcal{L}} = \begin{bmatrix} \frac{\partial \mathcal{L}_1}{\partial w_1} & \cdots & \frac{\partial \mathcal{L}_1}{\partial w_{|\mathcal{W}|}} \\ \vdots & & \vdots \\ \frac{\partial \mathcal{L}_m}{\partial w_1} & \cdots & \frac{\partial \mathcal{L}_m}{\partial w_{|\mathcal{W}|}} \end{bmatrix} \quad \mathcal{L} : \mathbb{R}^{|\mathcal{W}|} \rightarrow \mathbb{R}^m$$

Full Jacobian requires m backward passes through the NN

Figure 3.6: With one forward pass and one backward pass we get full column of the Jacobian. This is way much more efficient w.r.t forward mode for training NNs, since usually $|\mathcal{W}| \gg m$.

¹Note that this is NOT related to the automatic differentiation (forward mode). In this case we need to compute the values of the primal variables v_i by fixing the values of the inputs of the computational graph. You can compute this values by looking at the graph.

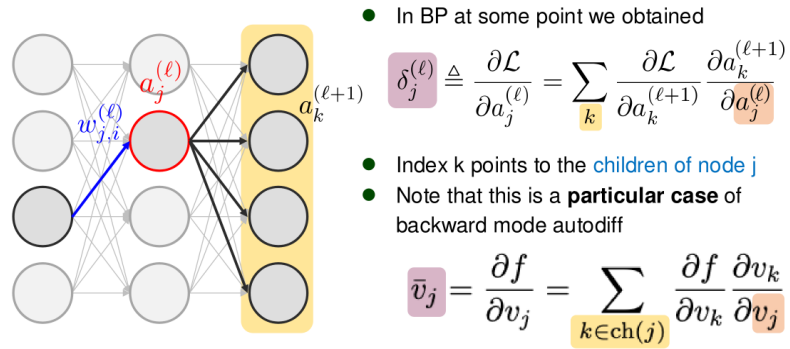


Figure 3.7: Link with the backpropagation algorithm for FFNN. This procedure also applies to CNN with pooling layer: for instance, for the max pooling, all the activations that do not survive, have a set of children that is empty.

Chapter 4

Overfitting and regularization in NN

Neural networks are prone to overfitting. The more parameters, the more likely overfitting appears, so we need to introduce some practical tricks to avoid this problem.

4.1 Early stopping

This approach is extremely easy to implement and to understand. During NN training it is common to compute the error function at every epoch (full mini-batch SGD pass over the training set). At each epoch we compute and plot the total error as follows:

$$\frac{1}{MK} \sum_{m=1}^M \overbrace{\sum_{k=1}^K \mathcal{L}(\mathbf{x}_{m,k}, t_{m,k})}^{\text{avg loss over a mini-batch}}$$

where M is the number of mini-batch, K is the size of each mini-batch (so MK is the size of the training). Also, at each epoch, we also compute the validation error.

The idea of early stopping is the following one: interrupt training as soon as the validation loss does not decrease anymore. That is, before we entering the overfitting region we stop the training. This is a very simple trick that should always be used when training a NN.

4.2 Training on more data

Best way to avoid overfitting? Just use more data for the training! The bigger the NN, the more data we need (remember the VC-dim of NNs and the fundamental theorem of statistical learning). Problem: getting new "real" data is sometimes impossible or really expensive. So what can we do? We can generate artificial data from the available one!

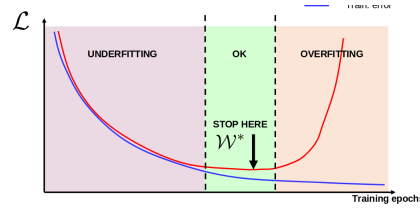


Figure 4.1: Early stopping: stop before when the validation error at epoch t is greater than the one at iteration $t - 1$.

The new data should belong to the same distribution.

In general, generating more realistic data is not trivial. The easiest cases are classification problems: we just have to apply some transformations on the sample that we have.

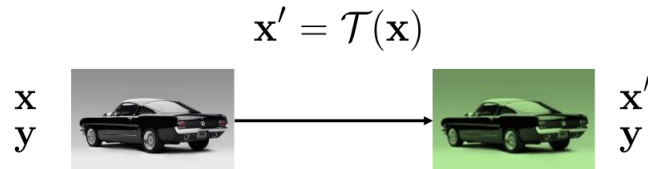


Figure 4.2: We apply a transformation to the image: green filter. The label y of the image remains unchanged (is still a car).

We can apply many different transformation: translation, rotation squeeze, change colors, blur, etc. We can use all this transformations to enlarge our dataset. Note that the artificial images are not like the "real" images since they are correlated to the images they were generated from (they're not independent).

But if instead we're dealing with speech signals, then we cannot use the same transformation used for the image above. Changing colors is futile (signals don't have colors); if we translate the signal we change its meaning. To summarize: we need to define some sort of transformations based on the knowledge of the input data domain. Such transformations sometimes are very difficult to find.

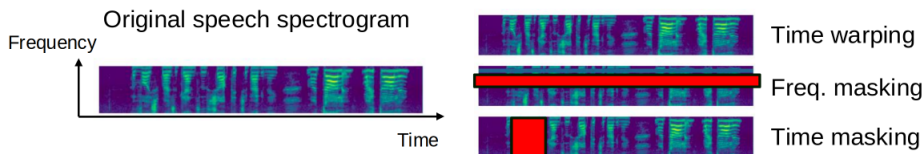


Figure 4.3: Data augmentation for speech signals with some meaningful transformation.

4.3 L2 Regularization and mini-batch SGD

The objective function that we want to minimize becomes:

$$\mathcal{L}(\mathcal{D}, \mathcal{W}) + \lambda \sum_{w_i \in \mathcal{W}} w_i^2 \quad (4.1)$$

where $\|\mathbf{w}\|_2 := \sum_{w_i \in \mathcal{W}} w_i^2 = \mathbf{w}^T \mathbf{w}$ is the called Tikhonov regularization (or L2 regularization). This is usually used in combination with SGD on minibatches.

$$\begin{aligned} w_j^{(n+1)} &= w_j^{(n)} - \frac{\eta^{(n)}}{K} \sum_{k=1}^K \nabla \left[\mathcal{L}(\mathbf{x}_k, t_k) + \lambda \sum_{w_i \in \mathcal{W}} w_i^2 \right] \\ \text{Apply linearity of the gradient} \quad &= w_j^{(n)} - \eta^{(n)} \lambda \nabla \sum_{w_i \in \mathcal{W}} w_i^2 - \frac{\eta^{(n)}}{K} \sum_{k=1}^K \nabla \mathcal{L}(\mathbf{x}_k, t_k) \\ \text{Gradient acts on } w_j \text{ only} \quad &= \underbrace{(1 - 2\eta^{(n)} \lambda) w_j^{(n)}}_{\text{coefficient is } < 1} - \frac{\eta^{(n)}}{K} \sum_{k=1}^K \nabla \mathcal{L}(\mathbf{x}_k, t_k) \end{aligned}$$

Figure 4.4: L2 regularization and the update in mini-batch SGD. At each step the weights are reduced (since the coefficient is < 1).

Using regularization has the effect of reducing the weights that are less important, only the most important ones get a bigger value. The bigger the value of λ , the bigger the number of weights that assume a small value. Therefore using L2 regularization is useful to reduce the number of parameters $\Rightarrow$ the NN becomes smaller and simpler $\Rightarrow$ we reduce overfitting. The selection of suitable λ is performed using a validation set (model selection).

Sometimes it is used L1 regularization, in which the objective function is:

$$\mathcal{L}(\mathcal{D}, \mathcal{W}) + \lambda \sum_{w_i \in \mathcal{W}} |w_i|. \quad (4.2)$$

Typically it is used in some selected layers. In practice we observe that, with L1 regularization, the network tends to increase the number of weights fixed to zero (the less important one). Instead, with L2 regularization, the less important weights get a small value but not necessarily zero: in total we have less number of zeros.

4.4 Dropout

4.4.1 Model ensembling

Dropout is related the concept of model ensembling. The idea of model ensembling is simple: we have many different models that are trained on the same input data. Then we build a bigger model that passes the input to each small model and its output is obtained by combining somehow the output of each small model (e.g. use the mean). Combining multiple models reduces overfitting.

Problem: it requires to train many different models and this is not suitable with NNs because they require much time and computation.

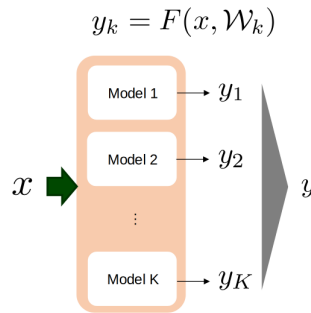


Figure 4.5: Model ensembling.

Consider the case in which we have infinite computation and we define one model for each possible value of the weight set $\mathcal{W}$ (infinitely many). Then the Bayesian combination tell us the optimal way of combining the models:

$$y = \int F(x, \mathcal{W}) p(\mathcal{W}|\mathcal{D}) d\mathcal{W} \quad (4.3)$$

in a sense is the average of the output of the infinitely many models, weighted by a probability (that we don't have). Dropout tries to approximate this output by using a single model.

4.4.2 Dropout formally

The main idea of the dropout is to randomly remove (drop) some of the neurons at every epoch during the training of a NN. Each neuron (the output is excluded) is dropped with a probability of $1 - p$ during training. Instead, during test, each neuron is active and the weights are scaled by p . Scaling by p maintains the expected values of the output of each

layer the same as in the training phase: it's like we're using the "average" network during test.

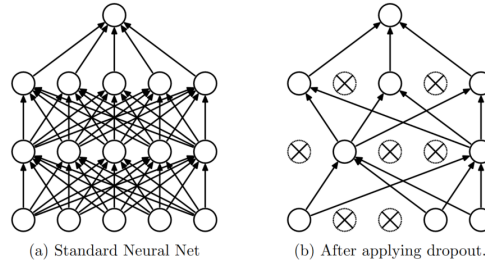


Figure 4.6: Dropout idea during training.

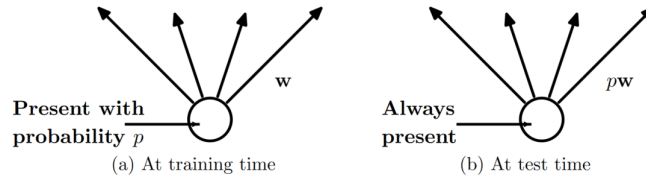


Figure 4.7: Neuron pov in training and test: each neuron is killed with probability $1 - p$ during training.

How dropout approximate the ideal Bayesian ensembling? When we kill some neurons during training, we're considering a different model from the one with all the neurons alive. To be more precise, if we remove even just one weight of the network, then we're considering a different model. Therefore in total, with dropout, we're considering $2^{|W|}$ different networks (with shared weights). Then the final model, the one that we use during test, we consider all the neurons with the weights scaled: this approximates the ensemble average (it is like combining different NNs to get one big NN).

We need to modify the output equation of layer in a NN. In a FFNN the output is:

$$\mathbf{o}^\ell = f(\mathbf{W}^\ell \mathbf{o}^{\ell-1} + \mathbf{b}^\ell) \quad (4.4)$$

To formalize dropout we need to introduce a vector

$$\mathbf{r} \in \{0, 1\}^{N^{\ell-1}}, \quad r_i \sim \text{Bernoulli}(p) \quad (4.5)$$

where $N^{\ell-1}$ is the number of neurons in the layer $\ell - 1$, and p is the probability of being

present for a neuron in the NN. Therefore a layer with dropout can be written as follows:

$$\mathbf{o}^\ell = f(\mathbf{W}^\ell(\mathbf{o}^{\ell-1} \odot \mathbf{r}) + \mathbf{b}^\ell) \quad (4.6)$$

that is, each output of layer $\ell - 1$ is considered with a probability p , the other ones are neglected. The same applies to CNNs, we need to consider a matrix R of Bernoulli random variables.

The question now is: how's dropout related to regularization? This is not evident from what we have said in this section. We will try to answer this question in the next subsection. However experimental results show that dropout reduce overfitting.

In practice, what we see is that, without dropout, we have **the problem of co-adaptation**: each neuron is strictly related to the neurons that are connected with it. In other words, each neuron is not "self sufficient" since relies on others to extract features: neurons may change in a way that they fix up the mistakes of the other neurons. As a consequence the network is more prone to overfit data.

By using dropout, the co-adaptation is mitigated: each neuron is more "self sufficient" since the presence of each neuron is highly unreliable. Dropout forces the existing neuron to learn more general and "average" feature. Experimental results shows that, by using dropout, layers in a CNN learn more interpretable features and in general the network gets better result in terms of overfitting.

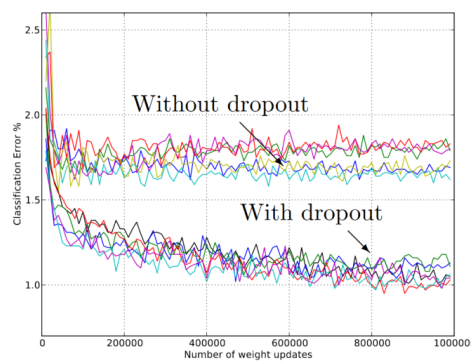


Figure 4.8: Dropout effect in practice: test error (MNIST).

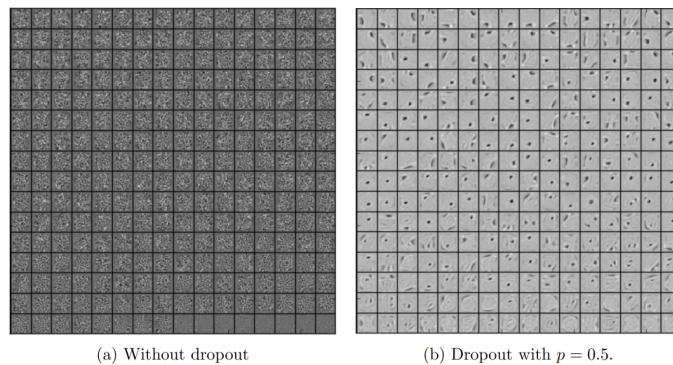


Figure 4.9: We can see that, without dropout, the features learned are not meaningful: neurons have co-adapted to produce good results. With dropout we can see that the hidden units seem to detect edges, strokes and spots in different parts of the image. This shows that dropout does break up co-adaptations, which is probably the main reason why it leads to lower generalization errors.

4.5 Final considerations on reducing overfitting

Which trick should we use to avoid overfitting? Early stopping should always be used, data augmentation should be considered especially for classification tasks since it is easy to apply, L2 and dropout should always be applied since they can strongly reduce overfitting. To summarize: we use all of them. Using these tricks allow us to implement very big and deep neural networks with relative small overfitting.

Chapter 5

Advanced optimization methods for NN

The base algorithm used for training NNs is the Gradient Descent. The weights are updated at each iteration of the algorithm. The update rule:

$$\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} + \mathbf{w}_s \quad (5.1)$$

where $\mathbf{w}_s$ is the descent step. The algorithm stops when a certain condition is satisfied. Different conditions can be used:

- Maximum time T reached.
- Difference between the gradient amplitude in iteration $i + 1$ and iteration i is lower than ε .
- Absolute difference between the total loss in iteration $i + 1$ and iteration i is lower than ε . This is the most used condition.

Each algorithm discussed in this chapter differs from the others in the computation of the descend step.

5.1 SGD

At each iteration i we sample one point with index $p(i)$ from the training set with size n . The update of the weights is computed as:

$$\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} - \eta^{(i)} \nabla \mathcal{L}(\mathbf{x}_{p(i)}, \mathbf{t}_{p(i)}; \mathbf{w}^{(i)}) \quad (5.2)$$

$p(i)$ is uniformly distributed in $\{1, \dots, n\}$, therefore we have that:

$$\begin{aligned}
 E[\nabla \mathcal{L}(\mathbf{x}_j, \mathbf{t}_j; \mathbf{w})] &= \sum_{h=1}^n \overbrace{P[j=h]}^{=1/n \text{ (uniform distribution)}} \nabla \mathcal{L}(\mathbf{x}_h, \mathbf{t}_h; \mathbf{w}) \\
 &= \sum_{h=1}^n \frac{1}{n} \nabla \mathcal{L}(\mathbf{x}_h, \mathbf{t}_h; \mathbf{w}) \\
 &= \frac{1}{n} \nabla \left(\sum_{h=1}^n \mathcal{L}(\mathbf{x}_h, \mathbf{t}_h; \mathbf{w}) \right) := \nabla \mathcal{L}(\mathcal{D}, \mathcal{W})
 \end{aligned} \tag{5.3}$$

In other words, the SGD, on average, follows the same direction followed by the GD.

5.2 Mini-batch SGD

The training set of size n is divided in n_b mini-batches with K samples each. The descend step is computed considering the gradient over the mini-batch that we're considering:

$$\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} - \frac{\eta^{(i)}}{K} \sum_{j=1}^K \nabla \mathcal{L}(\mathbf{x}_{p(j)}, \mathbf{t}_{p(j)}; \mathbf{w}^{(i)}) \tag{5.4}$$

One can prove (similarly to what we have done for the SGD), that, on average, the mini-batch SGD behaves the same as the GD. That is, on average it follows the same direction. The main difference with SGD is that, by considering $K > 1$ samples for the update of the weights, the mini-batch SGD has a lower variance. In other words the *noise of gradients* is reduced.

Let $\nabla \mathcal{L}_m$ be the gradient over a mini-batch. The variance is:

$$\begin{aligned}
 \text{Var}(\nabla \mathcal{L}_m) &= \text{Var} \left(\frac{1}{K} \sum_{j=1}^K \nabla \mathcal{L}(\mathbf{x}_{p(j)}, \mathbf{t}_{p(j)}; \mathbf{w}) \right) \\
 &= \frac{1}{K^2} \text{Var} \left(\sum_{j=1}^K \nabla \mathcal{L}(\mathbf{x}_{p(j)}, \mathbf{t}_{p(j)}; \mathbf{w}) \right) \quad (\text{sum of } N \text{ random variables}) \\
 &= \frac{1}{K^2} K \text{Var}(\nabla \mathcal{L}(\mathbf{x}_1, \mathbf{t}_1; \mathbf{w})) = \frac{1}{K} \text{Var}(\nabla \mathcal{L}(\mathbf{x}_1, \mathbf{t}_1; \mathbf{w}))
 \end{aligned}$$

The higher the K , the lower the variance $\Rightarrow$ the noise of gradients is reduced. The minimum noise is achieved when $K = n$, in this case we're actually using the GD: one "mini-batch" that contains the entire training set.

SGD vs mini-batch SGD The updates in SGD are noisy w.r.t. the true gradient and this leads to erratic behaviour in the search (wrong directions). Since the loss function of a NN is more often highly non-convex with many local minima, SGD's noisy updates are very useful since they introduce randomness into the optimization process that helps escaping local minima. This is the reason why SGD is better than GD: GD always gets stuck in flat regions.

The main problem of the SGD is that it is too slow: taking many wrong/noisy directions slow up the optimization process. Mini-batch SGD take into account this problem and tries to mitigate the wrong directions.

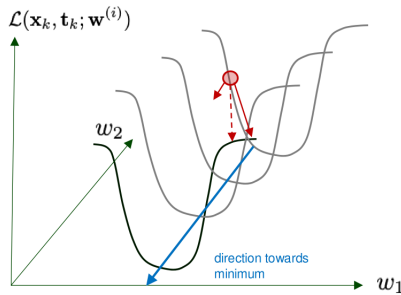


Figure 5.1: The red dotted line is the direction of the maximum decrease (direction followed by GD).

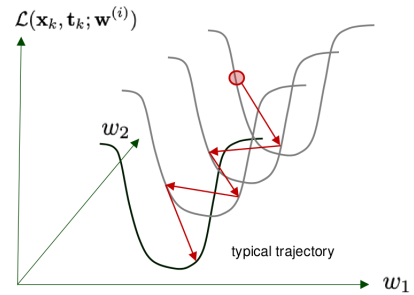


Figure 5.2: By using noisy gradient updates, the algorithm tends to follow a zig-zag trajectory which slow down the convergence towards the minimum.

How can we improve the SGD algorithm? Since we know that on average the algorithm is following the true gradient, then an idea is to trust the directions followed many times for the previous updates. This idea is implemented by the **momentum** trick.

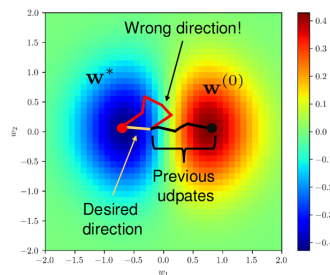


Figure 5.3: Trust the directions of previous updates.

5.3 SGD with Momentum

Imagine a ball rolling on a landscape: the slope makes it descend but constant slopes in the same direction give it **momentum**. If enough momentum is gained, a large force is needed to steer the ball towards a different direction.

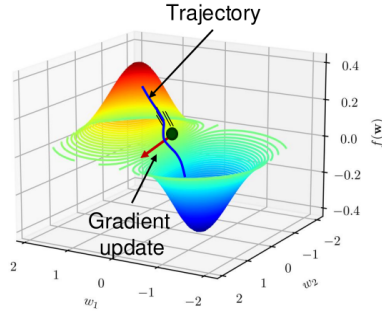


Figure 5.4: The ball represents the current values of the weights. The blue line is the trajectory followed, the red line is a noisy gradient update.

With this trick, momentary changes in the update direction are compensated. How can we implement such trick in the SGD algorithm?

Let the current parameters vector $\mathbf{w}^{(i)}$ be the position of the ball and let $\mathbf{v}^{(i)}$ be the current velocity of the ball (we assume unit mass, therefore the momentum $p = mv$ is $p = v$). The gravity force (with slope) is the gradient of the loss (computed for a mini-batch):

$$\mathbf{g}^{(i)} := \frac{1}{K} \sum_{k=1}^K \nabla \mathcal{L}(\mathbf{x}_k, \mathbf{t}_k; \mathbf{w}^{(i)}) := \text{The acting force} \quad (5.5)$$

The updating rule is defined as:

$$\underbrace{\mathbf{v}^{(i+1)}}_{\text{New momentum}} = \underbrace{\alpha \mathbf{v}^{(i)}}_{\text{Prev momentum}} - \eta^{(i)} \mathbf{g}^{(i)} \quad (5.6)$$

$$\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} + \mathbf{v}^{(i+1)} \quad (5.7)$$

$\alpha \in [0, 1]$.

The current momentum accumulates previous gradients in an implicit weighted sum.

$$\begin{aligned}
\mathbf{v}^{(1)} &:= -\eta^{(0)} \mathbf{g}^{(0)} \quad (\mathbf{v}^{(0)} = \mathbf{0}) \\
\mathbf{v}^{(2)} &:= \alpha \mathbf{v}^{(1)} - \eta^{(1)} \mathbf{g}^{(1)} = \alpha(-\eta^{(0)} \mathbf{g}^{(0)}) - \eta^{(1)} \mathbf{g}^{(1)} \\
\mathbf{v}^{(3)} &:= \alpha \mathbf{v}^{(2)} - \eta^{(2)} \mathbf{g}^{(2)} = \alpha^2(-\eta^{(0)} \mathbf{g}^{(0)}) - \alpha \eta^{(1)} \mathbf{g}^{(1)} - \eta^{(2)} \mathbf{g}^{(2)} \\
&\dots \\
\mathbf{v}^{(k)} &= \alpha^{k-1}(-\eta^{(0)} \mathbf{g}^{(0)}) + \alpha^{k-2}(-\eta^{(1)} \mathbf{g}^{(1)}) + \dots + \alpha(-\eta^{(k-1)} \mathbf{g}^{(k-1)}) - \eta^{(k)} \mathbf{g}^{(k)}
\end{aligned}$$

What's the meaning of the value α ? As we can see from the computation above: the larger the alpha, the more previous gradients affect the current update on the weights (**stronger momentum**). With this implementation, quick changes in direction have less influences in the update. As a result the final trajectory is smoother than the one followed by SGD.

Another feature of the momentum: long-term trends in the gradient direction bring acceleration. This is useful in order to escape local minima.

To summarize the momentum features:

1. **Accelerates convergence:** wrong directions are mitigated.
2. Makes it easier to **escape from local minima**.

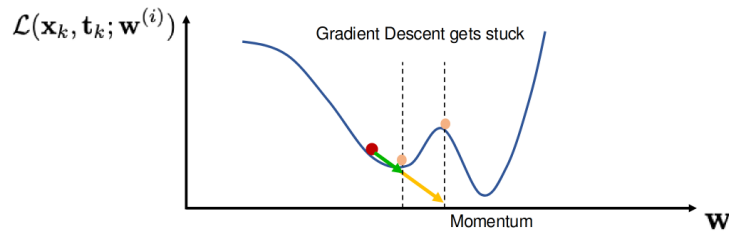


Figure 5.5: Escape from local minima with momentum.

Link to physics If we consider $\alpha = 1$, then the update of the momentum can be rewritten as:

$$\begin{aligned}
\mathbf{v}^{(i+1)} - \mathbf{v}^{(i)} &= -\eta^{(i)} \mathbf{g}^{(i)} \\
m \frac{\partial \mathbf{v}}{\partial t} &= \mathbf{F}
\end{aligned}$$

where:

- $\mathbf{v}^{(i+1)} - \mathbf{v}^{(i)}$ is the discrete version of $\frac{\partial v}{\partial t}$ (the denominator is the time step which is 1): the variation of the velocity between two updates.
- $-\eta^{(i)} \mathbf{g}^{(i)}$ is the force.
- $m = 1$ unit mass.

5.3.1 Nesterov Momentum

Is a variant of the momentum. The main problem of the momentum introduced so far is that it lacks of stability and it suffers from the **overshooting problem**.

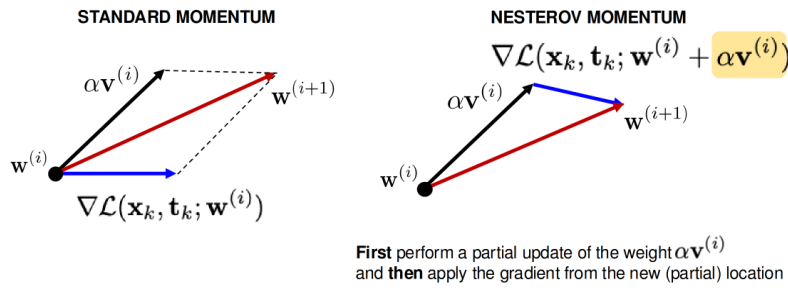


Figure 5.6: Nesterov momentum idea.

In the classical momentum the update is:

$$\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} + \overbrace{\alpha \mathbf{v}^{(i)} - \eta^{(i)} \mathbf{g}^{(i)}}^{:= \mathbf{v}^{(i+1)}}$$

In the Nesterov momentum the idea is the following:

1. First we perform a partial update $\mathbf{w}^{(i)} + \alpha \mathbf{v}^{(i)}$.
2. Then we compute the gradient from the new partial location.

This has the effect of **anticipating** the possible update.

The revised update equations:

$$\mathbf{g}_N^{(i)} := \frac{1}{K} \sum_{k=1}^K \nabla \mathcal{L}(\mathbf{x}_k, \mathbf{t}_k; \mathbf{w}^{(i)} + \alpha \mathbf{v}^{(i)}) \quad (5.8)$$

:= Acting force (on the partial location)

$$\mathbf{v}^{(i+1)} = \alpha \mathbf{v}^{(i)} - \eta^{(i)} \mathbf{g}_N^{(i)} \quad (5.9)$$

$$\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} + \mathbf{v}^{(i+1)} \quad (5.10)$$

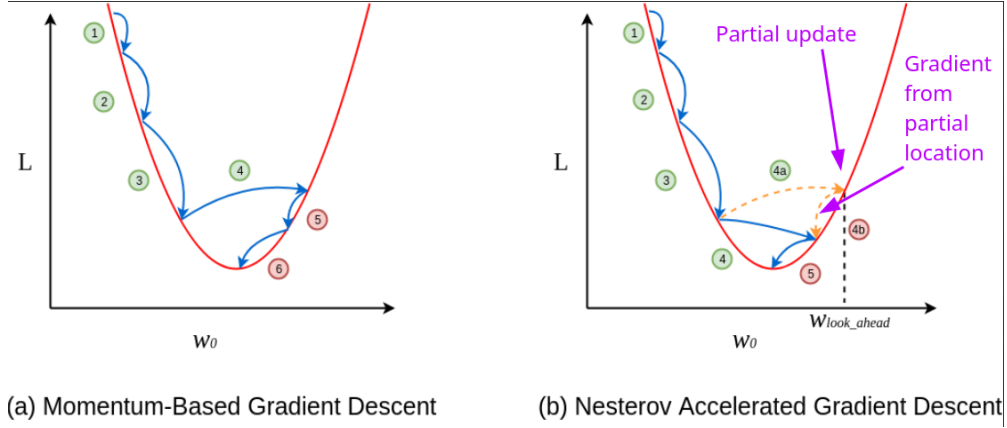


Figure 5.7: Nesterov example: overshooting mitigation leads to fast convergence.

Pros: using Nesterov momentum brings additional stability and mitigates overshooting.

5.4 Adaptive learning rate methods

5.4.1 Normalized gradient descent

Another problem of the GD algorithm is that the gradient vanishes near stationary points: steps become smaller and learning gets slower.

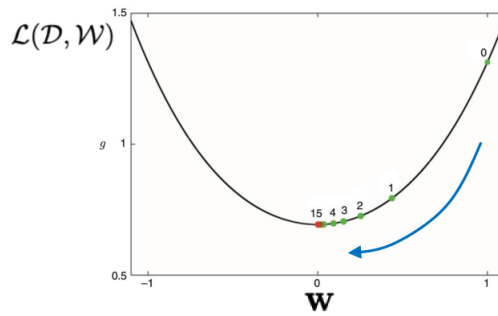


Figure 5.8: Gradient vanishes near stationary points.

To solve this problem we can normalize the gradient at each step:

$$\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} - \eta \frac{\mathbf{g}^{(i)}}{\|\mathbf{g}^{(i)}\|_2} \quad (5.11)$$

In this way we ignore the magnitude of the gradient and the update is constant at each iteration since it holds:

$$\|\mathbf{w}^{(i+1)} - \mathbf{w}^{(i)}\|_2 = \left\| \eta \frac{\mathbf{g}^{(i)}}{\|\mathbf{g}^{(i)}\|_2} \right\|_2 = \eta.$$

The update rule can be rewritten as:

$$\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} - \eta^{(i)} \mathbf{g}^{(i)}, \quad \eta^{(i)} := \frac{\eta}{\|\mathbf{g}^{(i)}\|_2 + \varepsilon} \quad (5.12)$$

where η must be a small value (big value leads to overshooting), instead $\varepsilon \approx 10^{-7}$ is used to avoid division by zero.

By using the dynamically adjusting learning step $\eta^{(i)}$ we improve convergence by moving faster across flat regions and quickly converging towards the minimum.

5.4.2 Adaptive learning rates: AdaGrad

Using the same learning rate for all parameters is not recommended because some weights may require slower or faster updates than others, depending on the current loss landscape.

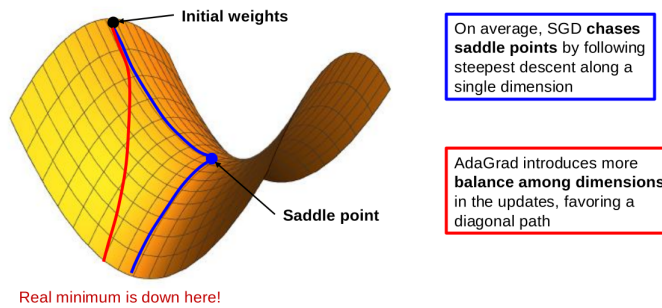


Figure 5.9: AdaGrad intuition.

By using the Adaptive Gradient, each weight has its own learning rate and this allows tuning the update speed along each direction of the weight landscape.

Let $\mathbf{g}^{(i)}$ be the gradient computed on a mini-batch at iteration i . Define

$$\alpha_m^{(i)} := \sum_{j=1}^i (g_m^{(j)})^2 \quad (5.13)$$

as the accumulated gradients for weight m . Then the weight specific learning rate is

computed as:

$$\eta_m^{(i)} := \frac{\eta}{\sqrt{\alpha_m^{(i)} + \varepsilon}} \quad (5.14)$$

where

- η is the initial learning rate.
- ε is a small constant to avoid the division by zero.

The updating rule becomes:

$$\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} - \boldsymbol{\eta}^{(i)} \odot \mathbf{g}^{(i)} \quad (5.15)$$

where $\boldsymbol{\eta}^{(i)}$ contains the learning rate associated to each weight.

Features:

- **Pro:** allows **exploring dimensions that are seldom updated** $\rightarrow$ weights that are updated many times have a lower learning rate, therefore less explored directions have more importance.
- **Cons:** **vanishing learning rates** $\rightarrow$ by looking at the definition of $\eta_m^{(i)}$ above, we can see that the learning rate becomes smaller and smaller as time evolves. Soon or later all learning rates go to zero and this may prematurely stop the training process.

5.4.3 Root Mean-Square propagation

RMSprop solves the problem of vanishing learning rates in AdaGrad. Idea: use an **exponentially decaying** moving average.

Let

$$r_m^{(i)} := \beta r_m^{(i-1)} + (1 - \beta)(g_m^{(i)})^2, \quad \beta \in [0, 1] \quad (5.16)$$

be the moving average for weight m . Then the weight specific learning rate is computed as:

$$\eta_m^{(i)} := \frac{\eta}{\sqrt{r_m^{(i)} + \varepsilon}}. \quad (5.17)$$

The role of β is very similar to the one played by the parameter α introduced for the momentum trick (see Section 5.3): the bigger the value of β (usually 0.9) the more the

older values influence the current value.

To see it more clearly, at iteration i we have:

$$r_m^{(i)} = \sum_{k=1}^i \beta^{i-k} (1 - \beta) (g_m^{(k)})^2 \quad (5.18)$$

so we can see that the coefficients decay exponentially fast.

The question now is: after how many steps can we neglect past terms? The coefficient is $\beta^j(1 - \beta)$ but we can neglect the common factor $(1 - \beta)$ and consider only β^j . If β^j is too small, then we can neglect the contribution in the summation above:

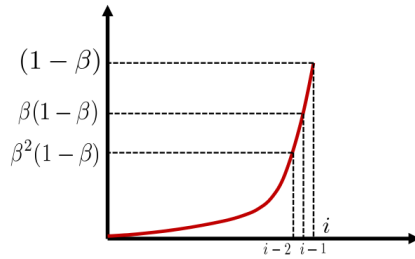
$$\text{if } \beta^j < T \Rightarrow \text{neglect the term}$$

where T is the threshold (e.g. 0.01, 0.001). So we can solve the inequality above for j , we get

$$j \log \beta < \log T \Leftrightarrow j > \frac{\log T}{\log \beta}$$

(since $\log \beta < 0 \Rightarrow$ flip sign). If we consider $\beta \approx 1$, we can approx the log function as the identity function. By using the Taylor expansion of $\log \beta$ we get

$$j \approx \frac{\log T}{\beta - 1} \quad (5.19)$$



5.4.4 Adaptive moment estimation (Adam)

Is the most widely used optimizer in NN. It is based on some of the techniques introduces in this chapter:

- Mini-batch SGD with momentum.
- RMSprop.

Experimental results show that it outperforms other optimizers in many tasks.

The RMSprop is called **second moment** and is computed as we have seen in the previous subsection:

$$\psi_m^{(i)} := \beta_2 \psi_m^{(i-1)} + (1 - \beta_2)(g_m^{(i)})^2, \quad \beta_2 \in [0, 1]. \quad (5.20)$$

Instead, the momentum is called **first moment** and is computed similarly to what we have seen in Section 5.3:

$$\xi_m^{(i)} = \beta_1 \xi_m^{(i-1)} + (1 - \beta_1)g_m^{(i)}, \quad \beta_1 \in [0, 1]. \quad (5.21)$$

The two moments are **biased** because they are initialized to zero: the actual moments are systematically underestimated. This is not a problem after many iterations because of the exponential decay, but it is a problem the first iterations. To compensate the bias on the first iterations, adam uses **bias correction**:

$$\hat{\xi}_m^{(i)} = \frac{\xi_m^{(i)}}{1 - \beta_1^i}, \quad \hat{\psi}_m^{(i)} = \frac{\psi_m^{(i)}}{1 - \beta_2^i} \quad (5.22)$$

In the end we can define the updating rule of the weights based on the two moments:

$$w_m^{(i)} = w_m^{(i-1)} - \eta \frac{\hat{\xi}_m^{(i)}}{\sqrt{\hat{\psi}_m^{(i)} + \varepsilon}}$$

Smoothed gradient
(moving average)
gives momentum

RMSprop-like
adaptive step length
based on past history

Figure 5.10: Update for each weight in Adam optimizer.

The ratio $\hat{\xi}_m^{(i)} / \hat{\psi}_m^{(i)}$ can be interpreted as a Signal-to-Noise Ratio:

- If the ratio is low $\Rightarrow$ the current gradient estimate is noisy $\Rightarrow$ we take a small step towards this direction.
- If the ration is high $\Rightarrow$ the current gradient is more accurate w.r.t. previous steps $\Rightarrow$ we can take a bigger step towards this direction.

The Nesterov momentum can be introduced in Adam optimizer. In this case we talk about NAdam optimizer.

5.5 Choice of the optimizer

Adam is the most widely used one because it performs well across a wide range of datasets and models, however, its effectiveness has been questioned by many papers. What we can do is to treat the choice of the optimizer as an hyperparameter (using a validation set).

Chapter 6

Batch Normalization

6.1 Covariate shift

General idea What's covariate shift? Imagine that you're training a **single FFNN**. Consider a classification problem in which we have two classes: car and bike. Suppose that in our training set, the cars are all black. In general we say that our dataset is biased.

After the training, our one layer FFNN has learned a decision boundary. The problem is that the test set contains image with any colors: the test set doesn't have the same bias as the training set. In this situation our FFNN has bad performance since it has an high generalization error. This happens because the **input distribution has changed** and the learned weights are not optimized for the full input distribution. Therefore we may have to retrain the network.

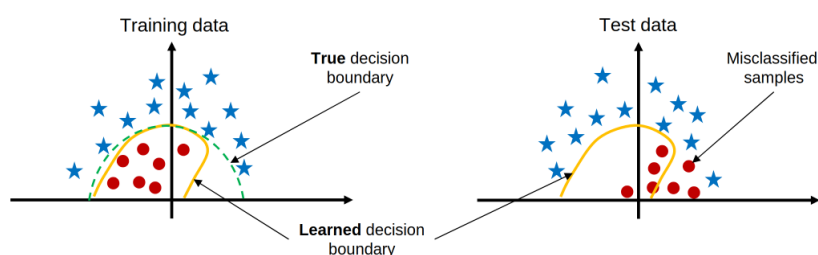


Figure 6.1: Covariate shift represented graphically for a one layer FFNN.

Covariate shift in FFNN In the general case this problem happens in every layer of the neural network¹. Indeed the input of each layer is the output of the previous one and

¹Remember that a FFNN is also called multilayer perceptron: layer $i + 1$ applies a decision boundary on the output of layer i .

by changing the value of the weights (during training) the **output distribution** changes continuously (iteration by iteration). During training each layer has to learn the weights and also it needs to compensate for this shift. **This is a problem** because makes the convergence slower and the training more difficult.

In deep neural network this problem is called **internal covariate shift** and is a big concern in NNs with many hidden layers: as we go deeper the dependence on previous inputs increases and consequently the covariate shift gets worse and worse.

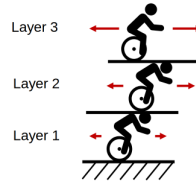


Figure 6.2: Idea of covariance shift in deep neural networks.

6.2 BN main idea

Batch normalization aims at controlling the covariate shift (mitigates the effect). The main idea: standardize the input distribution and rescale it with tunable parameters².

Batch normalization is a neural network layer placed between any two layers in the NN.



Figure 6.3: Batch normalization layer.

Consider the activation vector $\mathbf{a}$ of a generic layer of the neural network and let $\mathbf{a}'$ be the output of the batch normalization layer.

We first apply the batch normalization:

$$\mathbf{a}' = BN_{\alpha, \beta}(\mathbf{a}) \quad (6.1)$$

²**Note:** in practice, it is not clear whether this is really what it does or if this is the reason why it improves neural network training.

then the output vector of the layer is obtained by using the activation function on the normalized vector $\mathbf{a}'$:

$$\mathbf{o} = f(\mathbf{a}'). \quad (6.2)$$

This order is not mandatory, we can also do the opposite: first we apply the activation function to $\mathbf{a}$, then we apply batch normalization to $\mathbf{o}$ to get $\mathbf{o}'$. Rule of thumb: try both and choose the best.

Thanks to batch normalization, each layer receives an input with a **controlled distribution**. As a consequence, each layer can learn almost independently from the others.

6.3 BN layer

Let us consider a neural network trained with mini-batch SGD of dimension m . Let us consider a generic layer of the network. We denote the k -th mini-batch of activations as

$$B_k = \{\mathbf{a}_1, \dots, \mathbf{a}_m\}$$

one activation vector for each sample in the mini-batch. Since the output of the batch normalization layer is computed independently on each activation component $a_{i,j}$ (where i referred to vector $\mathbf{a}_i$ in B_k , j is the j -th component of $\mathbf{a}_i$), we can simplify the notation: we consider that each $\mathbf{a}_i$ is actually a scalar (a vector with only one element) that we indicate with a_i . Our goal now is to define the output of batch normalization which is indicated as a'_i .

$$a_i \rightarrow BN_{\gamma,\beta}(a_i) \rightarrow a'_i$$

To do that we have to:

1. Compute the mini-batch mean:

$$\mu_k := \frac{1}{m} \sum_{i=1}^m a_i \quad (6.3)$$

2. Compute the mini-batch variance:

$$\sigma_k^2 := \frac{1}{m} \sum_{i=1}^m (a_i - \mu_k)^2 \quad (6.4)$$

3. Standardize the input a_i :

$$\hat{a}_i := \frac{a_i - \mu_k}{\sqrt{\sigma_k^2 + \varepsilon}} \quad (6.5)$$

($\varepsilon \approx 10^{-8}$ is added in the implementation for numerical stability).

4. Rescale (γ) and shift (β) the activations using learnable parameters:

$$a'_i := \gamma \hat{a}_i + \beta \quad (6.6)$$

5. Apply activation function and move to next layer:

$$o_i := f(a'_i) \quad (6.7)$$

The learnable parameters γ, β are updated iteration by iteration by using the SGD updates. They give flexibility to set the mean and variance to any preferred value that improves the learning process. Note that they can also be used to "disable" the batch normalization (sometimes if we do not apply batch normalization we get better results), the learner has this degree of freedom.

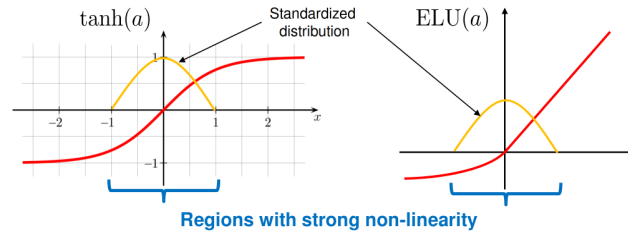


Figure 6.4: Tuning the mean and the variance is useful to fully exploit the non-linearity in the input. For the ELU we want to have a relative small variance since around zero the function has a strong non-linearity; instead the tanh function is linear for small values around zero so it is better to have a larger variance.

6.4 Learnable parameters

In the previous section, each activation vector $\mathbf{a}_i$ had only one element. In the general case each activation vector in B_k has many elements (say n): $\mathbf{a}_i = [a_{i,1}, \dots, a_{i,n}]^T \forall \mathbf{a}_i \in B_k$. The batch normalization procedure must be applied for each activation component.

1. Consider the first element of all activation vectors in B_k :

$$a_{1,1}, a_{2,1}, \dots, a_{m,1}$$

2. Apply batch normalization procedure to get the associated values

$$a'_{1,1}, a'_{2,1}, \dots, a'_{m,1}$$

Then we can apply the activation function to each of them and move to next layer.

3. Now we need to consider the second element of all activation vectors in B_k and we need to apply the same procedure.

Now from the scheme above we can understand that we use a different pair γ, β for each activation component.

In FFNN:

Therefore, by adding a single batch normalization layer we introduce ($2 \times$ No. of activation components) learnable parameters to our FFNN.

In CNN:

We have a single pair γ, β for each input feature map. Therefore we introduce ($2 \times D_{in}$) learnable parameters to our CNN.

6.5 BN during testing

In the previous sections we have discussed how to implement the batch normalization during training. What about the testing? We need to apply a similar rescaling. We use the mean and the variance over the whole training dataset in order to get better estimates!

Since reprocessing the whole training dataset is too complex, we maintain running mean and variance estimates during training. Each training iteration includes one pass over all mini-batches.

Running mean estimate:

$$\bar{\mu}^{(i)} = \alpha \bar{\mu}^{(i-1)} + (1 - \alpha) \mu_k \quad (6.8)$$

Running variance estimate:

$$(\bar{\sigma}^2)^{(i)} = \alpha (\bar{\sigma}^2)^{(i-1)} + (1 - \alpha) \sigma_k^2 \quad (6.9)$$

where i is the current iteration and μ_k, σ_k^2 are the mini-batch k estimates, $\alpha \in (0, 1)$.

Then the batch normalization input-output relation during testing time is computed using the running mean estimate and the running variance estimate:

$$\alpha'_i = \gamma \frac{a_i - \bar{\mu}^{(N)}}{\sqrt{(\bar{\sigma}^2)^{(N)} + \varepsilon}} + \beta \quad (6.10)$$

where N is the last training iteration and the values γ, β are learned during training.

6.6 How to train BN layer

We need to compute the backpropagation equations to:

- **Task 1:** Obtain the gradients of the loss wrt γ, β .
- **Task 2:** Obtain the gradients $\partial \mathcal{L} / \partial a_i$ to be backpropagated.

We want to apply the autodifferentiation and to do that we need to build the computational graph for the batch normalization layer and apply the backward-mode.

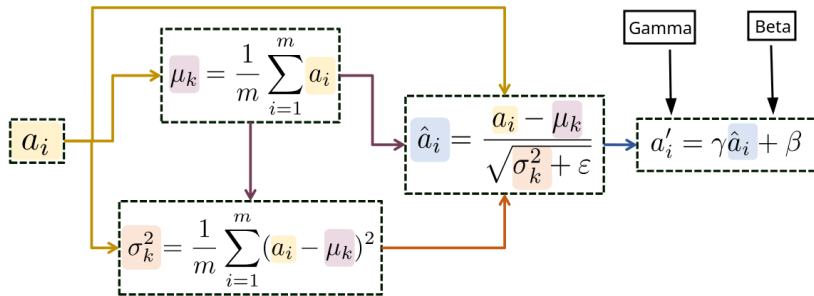


Figure 6.5: Computational graph for batch normalization.

Task 1 Starting from the last node of the computational graph we have:

$$\frac{\partial \mathcal{L}}{\partial a'_i} := \delta'_i$$

this is the error signal coming from the next layer, we assume that we have this value.

Proceeding with the backward-mode, we consider the node β and we have:

$$\frac{\partial \mathcal{L}}{\partial \beta} = \sum_{i=1}^m \frac{\partial \mathcal{L}}{\partial a'_i} \overset{=1}{\frac{\partial a'_i}{\partial \beta}} = \sum_{i=1}^m \frac{\partial \mathcal{L}}{\partial a'_i} \quad (6.11)$$

where we have considered in the sum all the children of β (all the values influenced by β , so all the a_i in our batch).

Now we consider the node γ :

$$\frac{\partial \mathcal{L}}{\partial \gamma} = \sum_{i=1}^m \frac{\partial \mathcal{L}}{\partial a'_i} \frac{\partial a'_i}{\partial \gamma} = \sum_{i=1}^m \frac{\partial \mathcal{L}}{\partial a'_i} \hat{a}_i \quad (6.12)$$

We now have the equation needed to update the values of γ , β in the GD.

Task 2 We use the backward-mode, we start from the last node a'_i .

$$\frac{\partial \mathcal{L}}{\partial a'_i} := \delta'_i$$

error signal coming from the next layer, we assume that we have this value.

Now consider node $\hat{a}_i$:

$$\frac{\partial \mathcal{L}}{\partial \hat{a}_i} = \frac{\partial \mathcal{L}}{\partial a'_i} \frac{\partial a'_i}{\partial \hat{a}_i} = \frac{\partial \mathcal{L}}{\partial a'_i} \gamma$$

note that here we don't have a sum because $\hat{a}_i$ influences only a'_i (only one value, not all the values a_i in our mini-batch).

Now we consider the node μ_k , we get:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mu_k} &= \sum_{i=1}^m \frac{\partial \mathcal{L}}{\partial \hat{a}_i} \frac{\partial \hat{a}_i}{\partial \mu_k} + \frac{\partial \mathcal{L}}{\partial \sigma_k^2} \frac{\partial \sigma_k^2}{\partial \mu_k} = \\ &= \sum_{i=1}^m \frac{\partial \mathcal{L}}{\partial \hat{a}_i} \frac{-1}{\sqrt{\sigma_k^2 + \varepsilon}} + \frac{\partial \mathcal{L}}{\partial \sigma_k^2} \frac{1}{m} \sum_{i=1}^m -2(a_i - \mu_k) \end{aligned}$$

We're missing value $\frac{\partial \mathcal{L}}{\partial \sigma_k^2}$ for now. We compute it in the next step. Above note that μ_k influences all values $\hat{a}_i$ and the variance

Now consider the node σ_k^2 :

$$\frac{\partial \mathcal{L}}{\partial \sigma_k^2} = \sum_{i=1}^m \frac{\partial \mathcal{L}}{\partial \hat{a}_i} \frac{\partial \hat{a}_i}{\partial \sigma_k^2} = \dots = \sum_{i=1}^m \frac{\partial \mathcal{L}}{\partial \hat{a}_i} (a_i - \mu_k) \frac{-1}{2} (\sigma_k^2 + \varepsilon)^{-3/2}$$

Finally we can consider the node a_i :

$$\begin{aligned} \delta_i &:= \frac{\partial \mathcal{L}}{\partial a_i} = \frac{\partial \mathcal{L}}{\partial \hat{a}_i} \frac{\partial \hat{a}_i}{\partial a_i} + \frac{\partial \mathcal{L}}{\partial \mu_k} \frac{\partial \mu_k}{\partial a_i} + \frac{\partial \mathcal{L}}{\partial \sigma_k^2} \frac{\partial \sigma_k^2}{\partial a_i} \\ &= \frac{\partial \mathcal{L}}{\partial \hat{a}_i} \frac{1}{\sqrt{\sigma_k^2 + \varepsilon}} + \frac{\partial \mathcal{L}}{\partial \mu_k} \frac{1}{m} + \frac{\partial \mathcal{L}}{\partial \sigma_k^2} \frac{2(a_i - \mu_k)}{m} \end{aligned} \quad (6.13)$$

the remaining partial derivatives was computed in the previous steps.

6.7 BN effects

Batch normalization produces 3 main effects.

1. **Eliminates the internal covariate shift** (we're not actually sure about that): the distributions of the activations are standardized, rescaled and shifted.
2. **Regularizes the model**: this is because BN operates on each mini-batch. The updates of the parameters γ, β depend on the full mini-batch. Since each data point can be seen in different mini-batches, it is harder for the model to overfit because the batchnorm output for the same sample changes every time (by how BN is defined).
3. **Enables higher learning rate**: The values of the activations influence the magnitude of the gradient. Since the activations can change a lot iteration by iteration, this makes tricky the choice of a good LR. Since BN is invariant to a rescaling of the activations it allows us setting higher LR.

Invariant to rescaling:

$$BN_{\gamma, \beta}(\mathbf{a}) = BN_{\gamma, \beta}(W\mathbf{o}) = BN_{\gamma, \beta}(\xi W\mathbf{o})$$

where ξ is a scalar. This can be proved easily:

Proof. Let μ_k, σ_k^2 be the mean and the variance computed for a set of activations $\{a_1, \dots, a_m\}$ and let $\mu'_k, (\sigma_k^2)'$ be the mean and the variance computed for the set of activations $\{\xi a_1, \dots, \xi a_m\}$, where ξ is a scalar. We have:

$$\begin{aligned}\mu'_k &= \frac{1}{m} \sum_{i=1}^m \xi a_i = \xi \mu_k \\ (\sigma_k^2)' &= \frac{1}{m} \sum_{i=1}^m (\xi a_i - \xi \mu_k)^2 = \xi^2 \sigma_k^2\end{aligned}$$

In the end the standardization is:

$$\hat{a}'_i = \frac{\xi a_i - \xi \mu}{\sqrt{\xi^2 \sigma_k^2}} = \hat{a}_i$$

(Ignore ε which is added in the implementation of BN). □

6.8 BN results

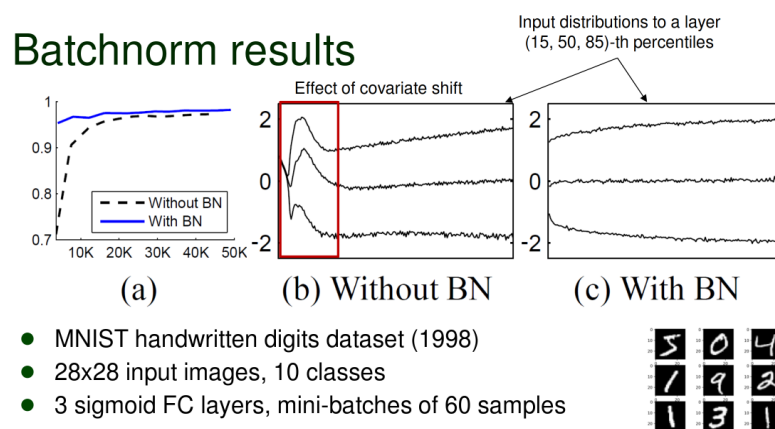


Figure 6.6: (a) BN leads to better accuracy and it is faster. (b,c) The normalized input values (pre-activation values) fed into a particular layer over training iterations. (b) In the first iterations the network is trying to make sense of the data: it wastes a lot of time in figuring out what is the correct distribution. (c) With BN we don't have any drifts in the beginning.

Despite its empirical effectiveness, it's still unclear why batch normalization actually works. We're not sure that BN works because it reduces internal covariate shift. Indeed some recent works have shown that sometimes BN does not mitigate covariate shift at all.

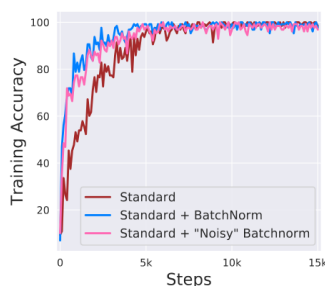


Figure 6.7: The standard + noisy batchnorm is obtained by adding some random noise at the output of each BN layer, in order to change the distribution input of the next layer. Even if the distributions are not controlled, we get a very similar result to the one achieved by the network standard + batchnorm: the performance are not significantly affected.

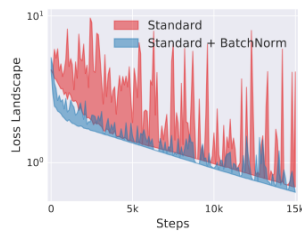


Figure 6.8: At a particular training step, we measure the variation (shaded region) in loss. The variation is computed as the difference in the loss when we move in the direction of the gradient (compute the gradient and evaluate the loss change when moving in that direction). The bigger the difference, the steepest the loss in that direction. With batchnorm the function is more regular: we get small improvements and the increase of the loss is very small. With a standard NN we can see a lot of spikes: we get big improvements but also very steep increase in the loss.

Batchnorm, from what we know, improve the performance because it smooths out the loss function (see figure above).

Chapter 7

Residual Networks

7.1 Top-N error for classification

Consider a classification task with K classes. Consider a NN with softmax as output function: in output we get a vector of probabilities that sums to one. Now let N be some number (lower than K). The prediction for some input sample is considered correct if the true label is among the **top-N** predictions (in the N highest probabilities).

Image	True label	Predicted label	Top 3 predicted labels
1	dog	dog	dog, cat, giraffe
2	cat	dog	dog, bug, cat
3	dog	bug	bug, giraffe, mouse
4	giraffe	giraffe	giraffe, cat, mouse
5	mouse	mouse	mouse, dog, cat

Figure 7.1: Example: Top-1 score: 3/5, Top-3 score: 4/5.

7.2 Depth is a problem in NN?

Why we want to implement deeper and deeper neural networks? Empirical results show that, the more deep the neural network, the more complex and abstracts are the features learned. The main problem of deep neural network is the **vanishing gradients**. In practice we observe that, the deeper the network, the more likely the gradients vanish.

In a previous chapter we have already talked about this problem. We have mentioned that it can be ameliorated with good initialization of the weights, with batch normalization and by selecting a good non-linear activation function (like the ReLU) that avoids contractive derivatives.

But this is not enough!

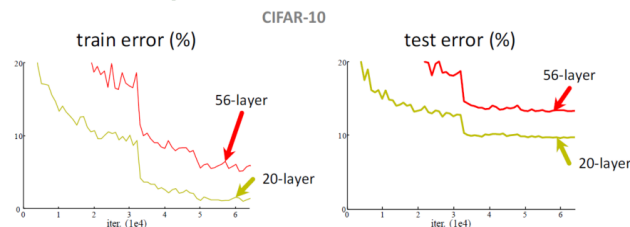


Figure 7.2: Deeper should lead to lower training error.

Consider a NN with 20 layers and another one with 56 layers. If the 20-layer network gets better results, then it means that adding layers is somehow counterproductive. But if this is the case, the 56-layer network should get the same performance as the 20-layer network: the first 20 layer can be copied from the other network, the last 36 can be set as identities. Therefore the two networks should get the same training error, but this is not the case as we can see from the picture above. So the problem must be related to the training. Residual networks aim to solve this problem.

7.3 Residual layer: original proposal [He2016]

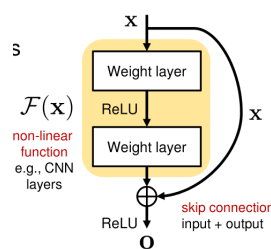


Figure 7.3: Residual unit structure.

$\mathcal{F}(\mathbf{x})$ is the non-linear function that represents the layers in our network (the yellow part in the image above). The output is computed as the sum between the input $\mathbf{x}$ and the value returned by $\mathcal{F}(\mathbf{x})$. The input $\mathbf{x}$ is propagated through the next layer thanks to the skip connection.

We can stack multiple residual units.

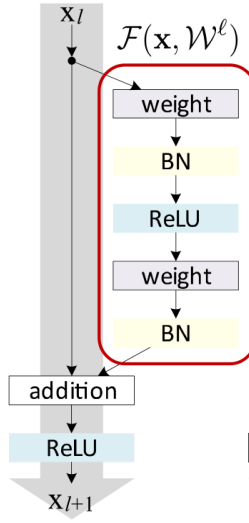


Figure 7.4: Residual unit structure (general case).

Each residual unit can be expressed as follows:

$$\mathbf{x}^{\ell+1} = f(\mathcal{F}(\mathbf{x}^{\ell}, \mathcal{W}^{\ell}) + h(\mathbf{x}^{\ell})) \quad (7.1)$$

In the original paper $f(\cdot) = \text{ReLU}(\cdot)$ and $h(\mathbf{x}^{\ell}) = \mathbf{x}^{\ell}$ (identity).

7.3.1 Results on CIFAR-10

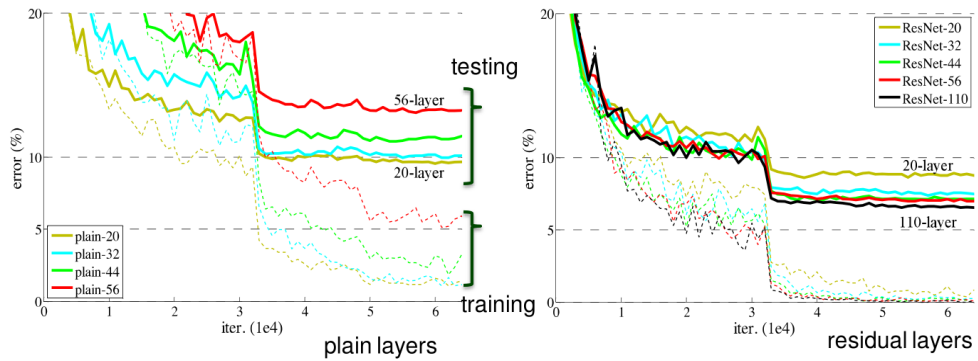


Figure 7.5: ResNets can go deeper and still improve their performance. The performances between (a) and (b) are flipped.

7.4 Residual layer: new proposal [He2016-b]

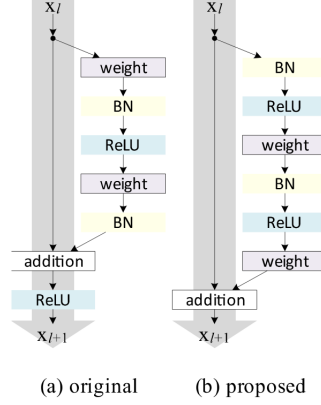


Figure 7.6: New proposal.

The main difference is that f is the identity function and not the ReLU function.

$$\mathbf{x}^{\ell+1} = \mathcal{F}(\mathbf{x}^{\ell}, \mathcal{W}^{\ell}) + \mathbf{x}^{\ell} \quad (7.2)$$

Now if we stack-up L residual layers, we can exploit the formula above and we get:

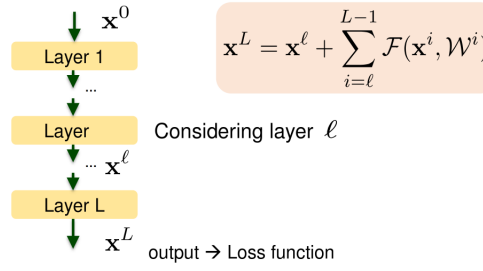
$$\mathbf{x}^L = \mathbf{x}^0 + \sum_{i=0}^{L-1} \mathcal{F}(\mathbf{x}^i, \mathcal{W}^i) \quad (7.3)$$

where $\mathbf{x}^0$ is the input of the first residual layer, $\mathbf{x}^L$ is the output of the last residual layer. We can see that there isn't any non-linear function between the input and the output. That is, the **information propagates directly through the layers**.

If you consider $\mathbf{x}^0$ as the input of the network and $\mathbf{x}^L$ the output, then $\mathbf{x}^L - \mathbf{x}^0 = \sum_{i=0}^{L-1} \mathcal{F}(\mathbf{x}^i, \mathcal{W}^i)$, that is, we're trying to learn the **residual** (the difference between the output and the input) and not the full non-linear map. This is why they are called residual networks.

7.4.1 Backpropagating the gradient

Consider that we start from layer ℓ and we stuck up residual layers until layer L . The value of $\mathbf{x}^{\ell}$ influences the value of the loss function since it is related to $\mathbf{x}^L$.



The output (starting from layer ℓ) is computed as

$$\mathbf{x}^L = \mathbf{x}^\ell + \sum_{i=\ell}^{L-1} \mathcal{F}(\mathbf{x}^i, \mathcal{W}^i)$$

We have that:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial x^\ell} &= \frac{\partial \mathcal{L}}{\partial x^L} \frac{\partial x^L}{\partial x^\ell} \\ &= \frac{\partial \mathcal{L}}{\partial x^L} \left(1 + \underbrace{\frac{\partial}{\partial x^\ell} \sum_{i=\ell}^{L-1} \mathcal{F}(x^i, W^i)}_{(C)} \right) \\ &= \underbrace{\frac{\partial \mathcal{L}}{\partial x^L}}_{(A)} + \underbrace{\frac{\partial \mathcal{L}}{\partial x^L} \frac{\partial}{\partial x^\ell} \sum_{i=\ell}^{L-1} \mathcal{F}(x^i, W^i)}_{(B)} \end{aligned}$$

where:

- $\frac{\partial \mathcal{L}}{\partial x^\ell}$ is analogous to the error message (even though $\mathbf{x}$ is not exactly the activation $\mathbf{a}$).
- (B) is the "standard" term that we have in NN without residual units: the error message is back propagated starting from the last layer.
- (A) is the new term that appears because of the residual units.
- (C) we have no control on this term since the function $\mathcal{F}$ is learned. This value can go to zero for whatever reason.

If (C) is close to zero and we remove term (A), then the "error message" of layer ℓ will be very small. This causes the vanishing gradient problem. With the term (A), the "error message" of the last layer reaches all the previous layer ℓ : **the information flows**

directly from last layer to the previous ones. This implies that the gradient for a layer does not vanish even when the weights are arbitrarily small.

7.4.2 Results on CIFAR-10

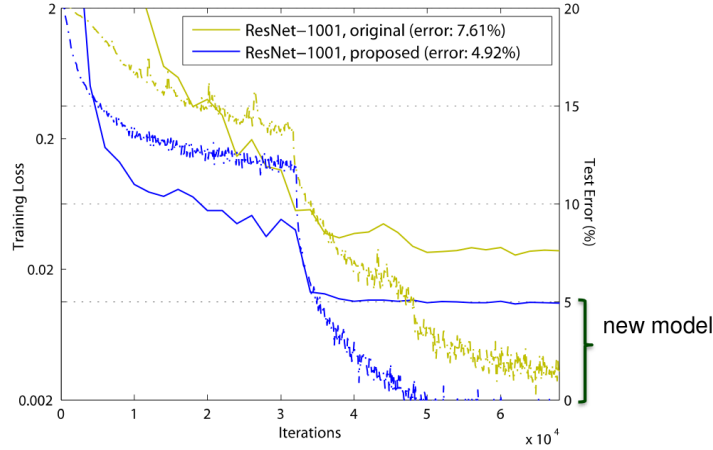


Figure 7.7: Comparison between the original (yellow) and the new proposal (blue).

7.5 Residual network: effects

Residual units changes the architecture of neural networks and empirical results show that, by using this trick, we can implement extremely deep neural networks without warring to much about the vanishing gradients problem.

The **key feature** is the **skip connections** that allows to learn a residual instead of the full linear mapping and also the gradient is directly backpropagated from the last layer to all the others. Skip connections are part of almost all state of the art NN architectures.

Chapter 8

Recurrent Neural Networks

8.1 Why are RNN useful?

So far we have only considered NNs in which the information flows in only one direction: loops are not allowed. This is not a limitation if our input samples are assumed to be i.i.d.. However, if we deal with sequential data, then we need to modify the architecture of the network in order to implement the notion of **correlation** among training samples. Correlation can be either temporal or positional:

- **Temporal correlation:** video (sequence of correlated images).
- **Positional correlation:** text made of natural language.

In those two cases, the samples are not i.i.d., they're are correlated.

We assumed that each sample depends on a number of past inputs:

$$\mathbf{x}_n \sim p(\mathbf{x}_n | \mathbf{x}_{n-1}, \mathbf{x}_{n-2}, \dots) \quad (8.1)$$

8.2 Vanilla RNN

The output of a RNN layer is a function of:

- The current input vector $\mathbf{x}_k$.
- The **internal neuron state** $\mathbf{h}_{k-1}$ (hidden state).

The internal neuron state stores the information about the previous inputs. Somehow the informations of all previous inputs are stored in the internal state. Therefore we do not

have to memorize all the previous inputs (too much memory required). Formally

$$\mathbf{h}_k = f(\mathbf{h}_{k-1}, \mathbf{x}_k; \mathcal{W}) \quad (8.2)$$

where f has the usual structure of a linear operation followed by an element-wise non-linearity. The new hidden state is a function of the previous hidden state and the current input. The two vectors are related by some learnable weights.

8.2.1 Vanilla layer

Input-output relations for a single layer:

$$\mathbf{a}_k = W\mathbf{h}_{k-1} + U\mathbf{x}_k + \mathbf{b} \quad (8.3)$$

$$\mathbf{h}_k = \tanh(\mathbf{a}_k) \quad (8.4)$$

where W and U are two learnable matrices of weights.

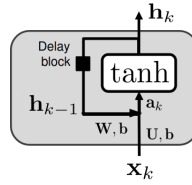


Figure 8.1: Vanilla RNN layer.

The final output is obtained as:

$$\mathbf{o}_k = V\mathbf{h}_k + \mathbf{q}, \quad \mathbf{y}_k = g(\mathbf{o}_k). \quad (8.5)$$

where g can be either the identity (for regression), softmax, etc. and V is a learnable matrix of weights.

Loops are problematic for backpropagation algorithm: by using the autodifferentiation backward-mode, we get stuck in an infinite loop. To avoid this we need to unroll the loop.

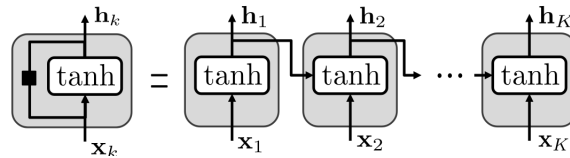


Figure 8.2: Unroll the computational graph: we get a directed acyclic graph of cells.

To summarize: the learnable weights and biases for just one layer are $\mathcal{W} = \{W, U, V, B, q\}$ and the loss function is as always $\mathcal{L}(\mathbf{y}_n, \mathbf{t}_n; \mathcal{W})$. To train a RNN we use SGD or its variants. The gradients are computed with a variant of the backpropagation called **backpropagation through time** (BTT) which is just a standard backpropagation on an unrolled version of the network graph¹.

8.2.2 Vanilla RNN problem

The main problem of vanilla RNN is that they're unable to learn long-term correlations in the input sequences. Why? Because of the **vanishing gradients problem**. Even if we have just one layer with many time steps, then the unrolled version of the graph has a huge amount of **cells**. Moreover we must apply many tanh which has a contracting derivative.

Why we use tanh? RNN are affected by instability: if we use something that it is not contracting (e.g. ReLU), then in the forward pass the magnitude of the outputs explodes. Therefore we need to use an activation function that has a contracting derivative.

8.3 Long-Short Term Memory (LSTM)

LSTM significantly modifies the architecture of the RNN. Main features:

- **Two internal states**: the usual **hidden state** $\mathbf{h}_k$ and the **cell state** $\mathbf{c}_k$ that serves to improve the **gradient flow** (mitigates the problem of the vanishing gradients).
- **Three gates** to control the information flow.
 1. **Forget gate** $\mathbf{f}_k$: helps the network forget useless past information.
 2. **Input gate** $\mathbf{i}_k$: controls how much of the current input info to let through.
 3. **Output gate** $\mathbf{o}_k$: produces the output vector of the LSTM.

8.3.1 LSTM cell

One layer is composed by many cells. The number of cells used depends on the number of past inputs that we consider.

In the LSTM cell: the sigmoid function $\sigma(\cdot)$ is used to implement a gate: controls the information flow. Indeed the output of the element-wise sigmoid function is a vector

¹"Through time" because by moving from right to left in a layer we are considering sample taken at different time

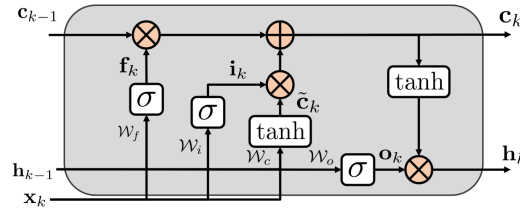
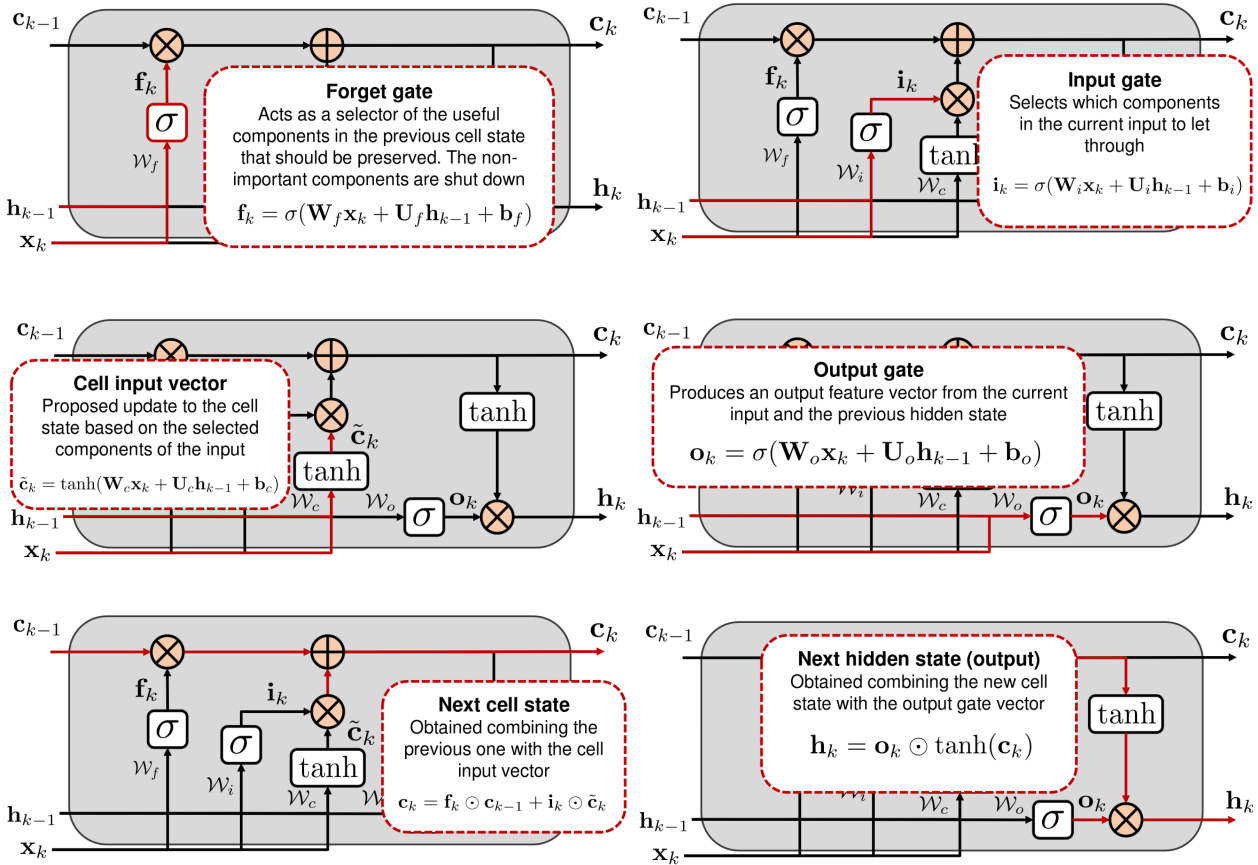


Figure 8.3: LSTM cell.

where each entry has a value in the range $[0, 1]$. For instance, by looking at the LSTM cell's figure, the output vector obtained from the element-wise product $c_{k-1} \odot f_k$ is a scaled version of c_{k-1} . If an element of f_k goes to zero, then the corresponding element in c_{k-1} is completely forgotten.

Instead the $\tanh(\cdot)$ function plays the usual role of activation function.



To summarize: the learnable parameters for a single layer are $\mathcal{W} = \{W_f, W_i, W_o, W_c, U_f, U_i, U_o, U_c, b_f, b_i, b_o, b_c\}$. The learning process in a RNN with LSTM cells is much more difficult since we have much more parameters to learn.

$$\begin{aligned}
 \mathbf{f}_k &= \sigma(\mathbf{W}_f \mathbf{x}_k + \mathbf{U}_f \mathbf{h}_{k-1} + \mathbf{b}_f) && \text{Forget gate} \\
 \mathbf{i}_k &= \sigma(\mathbf{W}_i \mathbf{x}_k + \mathbf{U}_i \mathbf{h}_{k-1} + \mathbf{b}_i) && \text{Input gate} \\
 \mathbf{o}_k &= \sigma(\mathbf{W}_o \mathbf{x}_k + \mathbf{U}_o \mathbf{h}_{k-1} + \mathbf{b}_o) && \text{Output gate} \\
 \tilde{\mathbf{c}}_k &= \tanh(\mathbf{W}_c \mathbf{x}_k + \mathbf{U}_c \mathbf{h}_{k-1} + \mathbf{b}_c) && \text{Compute the next cell state} \\
 \mathbf{c}_k &= \mathbf{f}_k \odot \mathbf{c}_{k-1} + \mathbf{i}_k \odot \tilde{\mathbf{c}}_k && \\
 \mathbf{h}_k &= \mathbf{o}_k \odot \tanh(\mathbf{c}_k) && \text{Compute next hidden state}
 \end{aligned}$$

Figure 8.4: LSTM input-output equations.

8.3.2 LSTM vs Vanilla RNN

How LSTM improves the vanishing gradients problem? Thanks to the cell state, the information flows forward in time (from first cell of the layer to the last one) without passing through many activation functions.

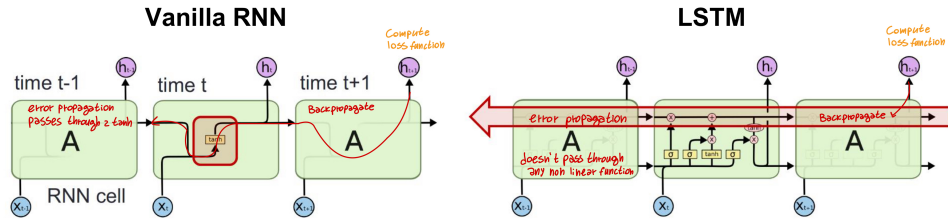
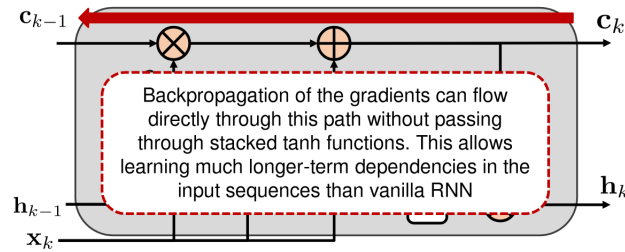


Figure 8.5: Backpropagation in vanilla RNN and in LSTM.

The idea is similar to the one that we have seen for Residual Networks: the error computed in the last cell is backpropagated towards the previous cells without passing through any non-linear function.



LSTM doesn't completely solve the vanishing gradients problem. The main reason is that the cell state $\mathbf{c}_k$ is related to $\mathbf{c}_{k-1}$ by a element-wise product operation between components that are between 0 and 1. For this reason, if we construct a RNN with many LSTM cells, our network will suffer from the vanishing gradients problem.

8.3.3 Multiple layer LSTM

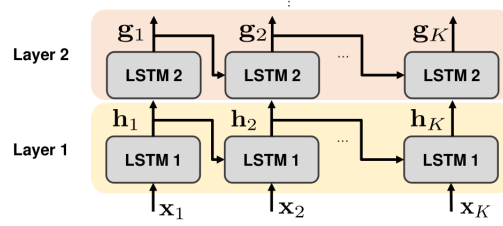


Figure 8.6: Multiple layer LSTM.

Remember that we consider the unrolled representation of the graph. All the cells from time step 1 to K that belong to the same layer share the same weights. The next layer takes as input the hidden state of the previous layer.

In practice can have a huge amount of cells (time steps considered) but the number of layers is at most 2, going above that number leads to some problems (e.g. vanishing gradients)

8.3.4 LSTM for sentimental analysis (NLP)

Description: given an input short text, classify the text into *positive* or *negative* feeling.

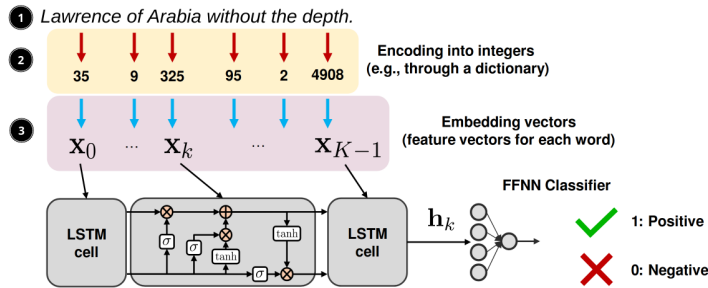


Figure 8.7: Sentiment analysis processing pipeline.

- (1) We have defined the length of the input.
- (2) Each word is associated to an integer number.
- (2) $\rightarrow$ (3) this step is a difficult task that requires many steps.

- (3) The aim is to find a meaningful representation of each word, each one is represented as a vector of features. Idea: the distance between two vectors represent the similarity between two words.

Chapter 9

Autoencoders

9.1 Basic autoencoder structure

Autoencoders are used to learn representative features of the input. This is an unsupervised learning task, therefore we don't have any label associated to the inputs. How do we evaluate the features learned without having any labels?

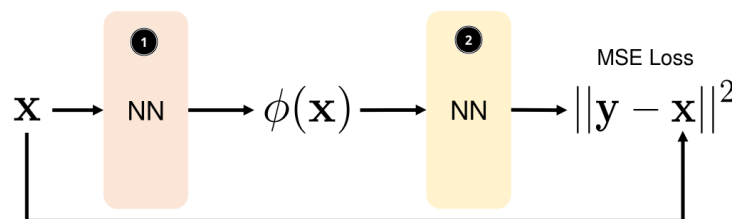


Figure 9.1

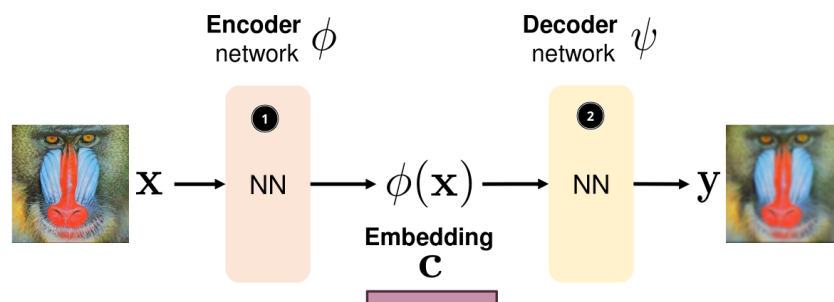


Figure 9.2: Basic autoencoder structure. $\phi(x)$ is a compressed representation with a smaller dimension w.r.t. the input $\mathbf{x}$.

The idea is simple: the first NN learns how to compress the input in a meaningful way, the second NN learns how to reconstruct the input starting from the compressed version.

The first network needs to learn the most important features, otherwise for the second network it is impossible to reconstruct the input. The loss function compute the squared error between the reconstructed image and the input image.

- **Encoder input-output relation:**

$$\mathbf{c} = \phi(\mathbf{x}, \mathcal{W}_\phi) \quad (9.1)$$

where $\mathbf{x}$ is the input, $\mathbf{c}$ is the compressed version of the input, $\mathcal{W}_\phi$ are the encoder weights.

- **Output of the decoder:**

$$\mathbf{y} = \psi(\mathbf{c}, \mathcal{W}_\psi) \quad (9.2)$$

where $\mathcal{W}_\psi$ are the decoder weights.

- **Loss function:**

$$\mathcal{L}(\mathbf{x}, \mathcal{W}_\phi, \mathcal{W}_\psi) = \|\mathbf{x} - \mathbf{y}\|^2 \quad (9.3)$$

How can we train autoencoders? Since the end-to-end structure is just a NN we can use the standard SGD and backpropagation algorithm.

9.1.1 Problem of the basic structure

The structure of an autoencoder described so far hides a huge problem. During training they can learn the **identity function**. For instance consider the extreme case in which the encoder associate each input to a unique vector and the decoder associate this vector to the input image. In the end we get a perfect reconstruction that minimizes the loss function. The autoencoder is **overfitting** the data: the compressed representation it doesn't contain meaningful features of the input.

How do we prevent this behaviour? Main strategies:

1. **Regularized** autoencoders.
2. **Denoising** autoencoders (the game changing idea).

9.2 Regularized autoencoders

We have seen that regularization is used to prevent overfitting in NN. The idea of regularized autoencoder is similar but it is not applied to the weights of the two networks, it is applied instead to the compressed representation of the input $\mathbf{c}$.

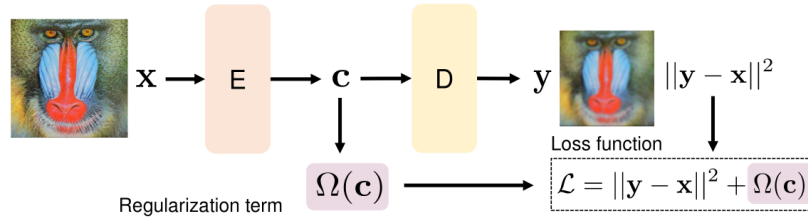


Figure 9.3: Regularized autoencoder.

Idea: add a regularization function $\Omega(\mathbf{c})$ to the loss function. In general there exist many possible regularization functions that we can use. We can consider the ones that we have introduced in the NN regularization chapter:

- L1 norm: $\Omega(\mathbf{c}) = \lambda \sum_{i=1}^v |c_i|$ for achieve sparsity (many components at zero value).
- L2 norm: $\Omega(\mathbf{c}) = \lambda \sum_{i=1}^v c_i^2$ for achieve small magnitude (many components with small values).

9.3 Denoising autoencoders

The idea is very simple: make the task more difficult by corrupting the input with **artificial noise**. The kind of artificial noise that we can use depends on the type of input data we're dealing with (e.g. gaussian noise or set some inputs to zero).

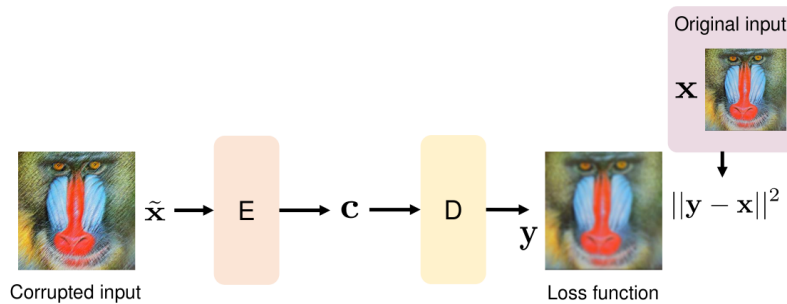


Figure 9.4: Denoising autoencoders.

Since the corrupted input $\tilde{\mathbf{x}}$ changes at every iteration, it is impossible for the network to learn the identity function. Note that, as a side effect, we learn how to remove the noise from the input data (we get this for free).

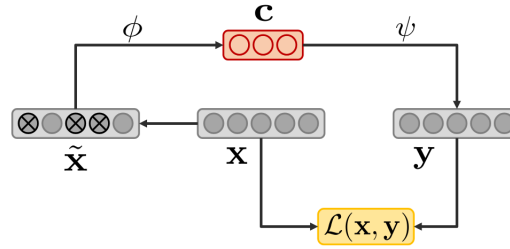


Figure 9.5: Denoising autoencoder scheme.

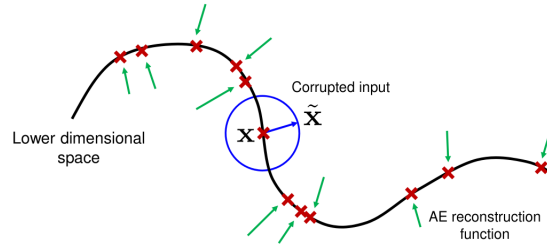
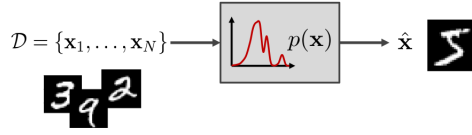


Figure 9.6: In this case our input data $\mathbf{x}$ belong to $\mathbb{R}^2$ and we want to reduce their dimensionality to $\mathbb{R}^1$. The lower dimensionality space is represented by the line that passes through all the input data, and usually is called **manifold**. By corrupting the $\mathbf{x}$ we are pushing it away from the manifold. The corrupted input cannot be compressed in the same lower dimensional space. The goal of the learner is to bring back the corrupted image to the manifold, in this way the learner is learning the shape of the manifold.

How to corrupt the image? As we have said before, usually we add Gaussian noise or pepper noise. Rule of thumb: search online what kind of noise is used for the type of data that you're dealing with.

9.4 Variational autoencoders (VAE)

VAE are generative models, so the goal is to learn the input data distribution $p(\mathbf{x})$ which is used to generate new data samples similar to those in the given dataset. The learning is based on the distribution of the training dataset. Generate new data is done by sampling from $p(\mathbf{x})$ or from a conditioned distribution on some side information $p(\mathbf{x}|I)$ (e.g. in text to image application the side information is the input text).



9.4.1 How to sample from a distribution

In general sampling from a complex distribution is hard. If we know the expression of the Cumulative Distribution Function (CDF) then sampling is performed as

$$\hat{\mathbf{x}} = F^{-1}(\mathbf{z}) \quad (9.4)$$

where $\mathbf{z}$ is sampled from a uniform distribution.

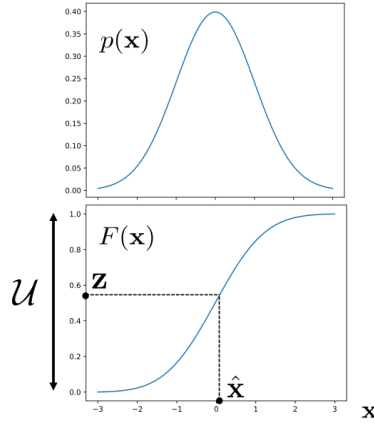


Figure 9.7: $F(\mathbf{x}) := \int_{-\infty}^{\mathbf{x}} p(\mathbf{x})$, where $p(\mathbf{x})$ is the probability density function.

Suppose that we want to sample a value $\hat{\mathbf{x}}$ from a Gaussian distribution with a defined mean and variance. Software libraries work as follows:

1. Generate a value $\mathbf{z} \in [0, 1]$ from a uniform distribution.
2. Look up the CDF of normal random variable.
3. Look up the value $\hat{\mathbf{x}}$ associated to $\mathbf{z}$ in the CDF (i.e. $\hat{\mathbf{x}} = F^{-1}(\mathbf{x})$) and return the value.

In this way the library has to implement only the uniform distribution. One can prove that, after many sampling, $\hat{\mathbf{x}}$ is distributed as $p(\mathbf{x})$.

Problem: we don't know the real data distribution (we don't have a formula) so we do not know the CDF function and therefore we cannot invert it. So the idea is to use a neural network as the generative function (it plays the role of F^{-1}).

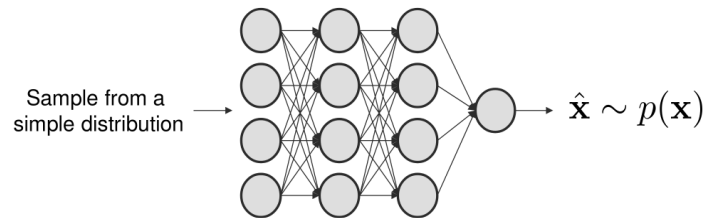


Figure 9.8: Idea of generative models. The input is the value $\mathbf{z}$ sampled from a "simple" distribution (is not always a uniform distribution). The output is the sample $\hat{\mathbf{x}}$.

9.4.2 Latent representation (LR)

The LR framework is used to model our input data.

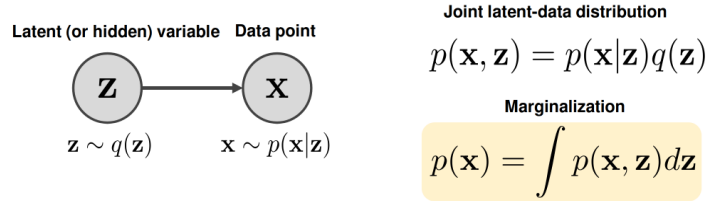


Figure 9.9: Latent representation.

We assume that our input data points $\mathbf{x}$ depend from a **latent variable** $\mathbf{z}$ which has an (unknown) distribution $q(\mathbf{z})$. The probability $p(\mathbf{x})$ is then given by the marginalization.

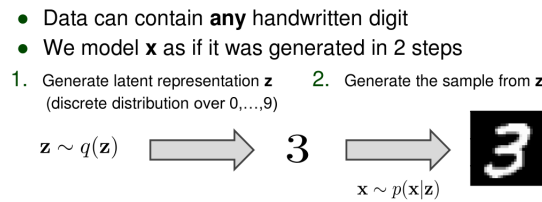


Figure 9.10: MNIST example. In this case the $\mathbf{z}$ variable is a digit between 0 to 9, each digit has an associated probability. $\mathbf{x}$ is instead the image of the digit and the probability of a certain image depends on the value of $\mathbf{z}$.

Question: how can we train this kind of model?

9.4.3 Learn $p(\mathbf{x})$

By using the latent representation, we have that

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})q(\mathbf{z})d\mathbf{z}. \quad (9.5)$$

Idea: since the training samples are true samples taken from $p(\mathbf{x})$ we can use them as target values. The NN should output values in the training set with non-zero (or high) probability. The intuition behind this framework—called “maximum likelihood”—is that if the model is likely to produce training set samples, then it is also likely to produce similar samples, and unlikely to produce dissimilar ones.

Note that the NN should generate new data so the output should be slightly different from the values in the training set (we don’t want an identity function).

Naive approach (not-good) The naive approach consider an approximation of $p(\mathbf{x})$.

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})q(\mathbf{z})d\mathbf{z} \quad \Longrightarrow \quad p(\mathbf{x}) \approx \frac{1}{n} \sum_{i=1}^n P(\mathbf{x}|\mathbf{z}_i; \mathcal{W})$$

Approximated by NN

Figure 9.11: Naive model approximation. $p(\mathbf{x})$ can be seen as $p(\mathbf{x}) = E_z[p(\mathbf{x}|\mathbf{z})]$ and the mean can be approximated as $E_z[p(\mathbf{x}|\mathbf{z})] \approx 1/n \sum_{i=1}^n P(\mathbf{x}|\mathbf{z}_i; \mathcal{W})$ (Monte Carlo approximation).

The idea is to approximate the target distribution using sampling. To this end we need to sample a large number of $\mathbf{z}$ from a simple distribution (e.g. uniform distribution), usually $\mathbf{z} \sim \mathcal{N}(0, I)$. Here we assume that the generating distribution (approximated by the NN) has a Gaussian shape.

To summarize: the $\mathbf{z}$ values are the input of our network, the target values are the input data $\mathbf{x}$ and what we want to do is to maximize $p(\mathbf{x})$ which implies $\frac{1}{n} \sum_{i=1}^n P(\mathbf{x}|\mathbf{z}_i; \mathcal{W})$.

$$\text{Maximize } \frac{1}{n} \sum_{i=1}^n P(\mathbf{x}|\mathbf{z}_i; \mathcal{W}) \quad \Longrightarrow \quad \text{Minimize } \|\hat{\mathbf{x}} - \mathbf{x}\|^2$$

The above implication is obtained by considering $\mathbf{z} \sim \mathcal{N}(0, I)$ (see section 1.3).

Main problem: the main problem of the naive approach is that we need to sample many values of $\mathbf{z}$ in order to get a good estimation of the probability. And all of this samples are used only to train on just 1 data point $\mathbf{x}$.

VAE approach VAEs approach aims to solve the problem of the naive solution.

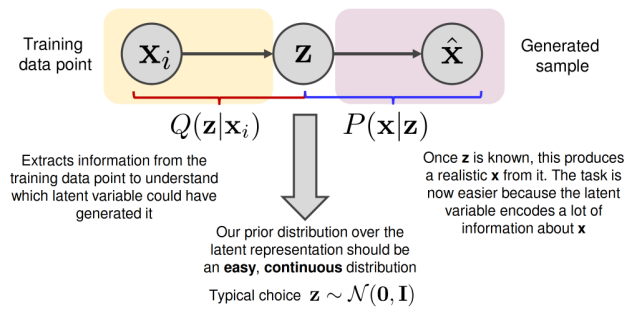
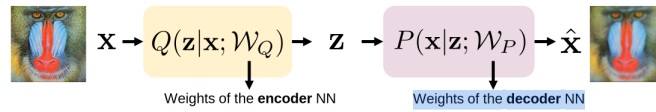


Figure 9.12: VAE approach: encode $\mathbf{x}_i$ to $\mathbf{z}$ and then decode it to $\hat{\mathbf{x}}$.

Idea: the first network, given the input $\mathbf{x}_i$ from our training set, generate a distribution $\mathbf{z}$ that is likely its latent representation. Using this approach we avoid the problem of generating a huge number of $\mathbf{z}$ values.

Why? In practice, for most $\mathbf{z}$, $P(\mathbf{x}|\mathbf{z})$ will be nearly zero, and hence contribute almost nothing to our estimate of $P(\mathbf{x})$. The key idea behind the variational autoencoder is to attempt to sample values of $\mathbf{z}$ that are likely to have produced $\mathbf{x}$, and compute $P(\mathbf{x})$ just from those.



Note that the function Q is not a deterministic mapping, it is stochastic! This means that for the same input we don't expect to get always the same output. How can we achieve such behaviour? We need to choose the shape of the distribution over $\mathbf{z}$ and then the network has to learn the parameters associated. The typical choice is the **Gaussian distribution**:

$$Q(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \mu(\mathbf{x}, \mathcal{W}_Q), \Sigma(\mathbf{x}, \mathcal{W}_Q))$$

$\Sigma(\mathbf{x}, \mathcal{W}_Q)$ is **diagonal** Parameters deterministically learned by the encoder

Figure 9.13: Covariance matrix is diagonal because the random variables in $\mathbf{z}$ are independent.

This is the trick that we use to implement a stochastic function.

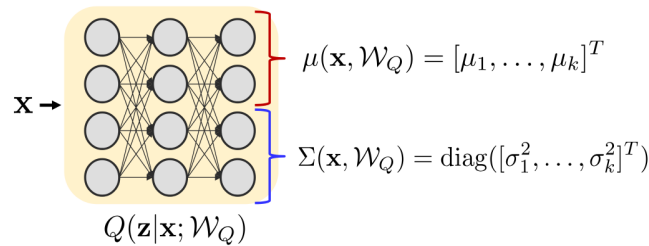


Figure 9.14: How to outputting mean and covariance.

Since the diagonal of covariance contains non-negative values, we need to use a proper activation function. One choice is the **Softplus function**:

$$\text{softplus}(x) = \frac{1}{\beta} \log(1 + \exp(\beta x)) \quad (9.6)$$

which is a smooth approximation of the ReLU.

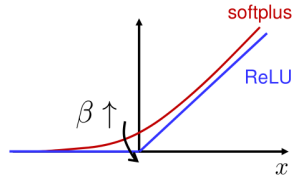


Figure 9.15: Softplus function. Note that for large negative values is ≈ 0 (just above). The bigger the β the closer to the ReLU.

9.4.4 Deriving the loss function

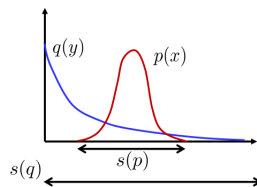
We need to define a suitable loss function. Recall that our goal is maximize $p(\mathbf{x})$ for the values in the **training data** (to increase the model's confidence in generating real data) but at the same time we want to generalize to the true full distribution (since the network needs to generate new data, not the one present in the training set).

We want to have

$$Q(\mathbf{z}|\mathbf{x}; \mathcal{W}_Q) \approx P(\mathbf{z}|\mathbf{x}) \quad (9.7)$$

where $P(\mathbf{z}|\mathbf{x})$ is the true distribution of the $\mathbf{z}$ that could have generated $\mathbf{x}$ (which is unknown). We will see that the maximization of $p(\mathbf{x})$ arises from this. We want to minimize the difference between $Q(\mathbf{z}|\mathbf{x}; \mathcal{W}_Q)$ and $P(\mathbf{z}|\mathbf{x})$ because this means that our encoder is generating good latent variables $\mathbf{z}$. To do so we need to introduce a similarity metric between two distributions.

Kullback-Leibler divergence KL-divergence is used to measure the difference between two distributions. Consider two random variables x, y with the same alphabet $\mathcal{X}$, let $p(x), p(y)$ be their probability density functions with support $s(p) \subset s(q) \subset \mathcal{X}$.



The KL-divergence is defined as:

$$\begin{aligned} D_{KL}[p||q] &:= \int_{s(p)} p(a) \log \frac{p(a)}{q(a)} da \\ &:= E_{a \sim p} \left[\log \frac{p(a)}{q(a)} \right] \end{aligned} \quad (9.8)$$

Minimizing $D_{KL}[p||q]$ means minimizing the difference between the p and q .

Two important properties of KL-divergence:

- **Asymmetry:** $D_{KL}[p||q] \neq D_{KL}[q||p]$
- **Non-negativity:** $D_{KL}[p||q] \geq 0 \forall p, q$. Also $D_{KL} = 0$ if and only if $p \equiv q$.

Intuitively, the KL-divergence, measures the "distance" between two distribution, but in practice it is not a distance function because it isn't symmetric and does not satisfy the triangular inequality.

Lets get back to our problem: minimizing the difference between $Q(\mathbf{z}|\mathbf{x}; \mathcal{W}_Q)$ and $P(\mathbf{z}|\mathbf{x})$. We use the KL-divergence:

$$D_{KL}[Q(\mathbf{z}|\mathbf{x})||P(\mathbf{z}|\mathbf{x})] := E_{\mathbf{z} \sim Q}[\log(Q(\mathbf{z}|\mathbf{x})/P(\mathbf{z}|\mathbf{x}))] \quad (9.9)$$

To expand the RHS we use the Bayes's theorem

$$p(\mathbf{z}|\mathbf{x}) := \frac{p(\mathbf{z}, \mathbf{x})}{p(\mathbf{x})} = \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})}$$

therefore:

$$\begin{aligned} E_{\mathbf{z} \sim Q}[\log(Q(\mathbf{z}|\mathbf{x})/P(\mathbf{z}|\mathbf{x}))] &= E_{\mathbf{z} \sim Q}[\log Q(\mathbf{z}|\mathbf{x}) - \log(P(\mathbf{x}|\mathbf{z})P(\mathbf{z})) + \log P(\mathbf{x})] \\ &= E_{\mathbf{z} \sim Q}[\log Q(\mathbf{z}|\mathbf{x}) - \log(P(\mathbf{x}|\mathbf{z})P(\mathbf{z}))] + \log P(\mathbf{x}) \end{aligned} \quad (9.10)$$

where $P(\mathbf{x})$ gets out of the expectation because it doesn't depend on $\mathbf{z}$. We can expand furthermore the RHS (reordering):

$$\begin{aligned} E_{\mathbf{z} \sim Q}[\log(Q(\mathbf{z}|\mathbf{x})/P(\mathbf{z}|\mathbf{x}))] &= E_{\mathbf{z} \sim Q}[-\log P(\mathbf{x}|\mathbf{z}) + \log(Q(\mathbf{z}|\mathbf{x})/P(\mathbf{z}))] + \log P(\mathbf{x}) \\ &= E_{\mathbf{z} \sim Q}[-\log P(\mathbf{x}|\mathbf{z})] + E_{\mathbf{z} \sim Q}[\log(Q(\mathbf{z}|\mathbf{x})/P(\mathbf{z}))] + \log P(\mathbf{x}) \\ &= E_{\mathbf{z} \sim Q}[-\log P(\mathbf{x}|\mathbf{z})] + D_{KL}[Q(\mathbf{z}|\mathbf{x})||P(\mathbf{z})] + \log P(\mathbf{x}) \end{aligned} \quad (9.11)$$

Note that the probability distribution $P(\mathbf{z})$ is a prior distribution! That is, we choose it.

$$\begin{aligned}
 \log P(\mathbf{x}) &= \overset{\text{(B)}}{D_{\text{KL}}[Q(\mathbf{z}|\mathbf{x})||P(\mathbf{z}|\mathbf{x})]} + \\
 &\overset{\text{(A)}}{+ E_{\mathbf{z} \sim Q}[\log P(\mathbf{x}|\mathbf{z})]} \quad \text{(C)} \\
 &\quad - \overset{\text{(D)}}{D_{\text{KL}}[Q(\mathbf{z}|\mathbf{x})||P(\mathbf{z})]}
 \end{aligned}$$

Figure 9.16: Putting everything together and reordering.

(A) $\log P(\mathbf{x})$	This is our optimization target. We want to maximize this term as it represents the probability to generate the training sample $\mathbf{x}$	(C) $E_{\mathbf{z} \sim Q}[\log P(\mathbf{x} \mathbf{z})]$	This measures how well the decoder is «decoding» the latent variable $\mathbf{z}$ to reconstruct $\mathbf{x}$. It's an expected log-likelihood.
(B) $D_{\text{KL}}[Q(\mathbf{z} \mathbf{x}) P(\mathbf{z} \mathbf{x})]$	This is an error term . It measures the «distance» between the true distribution $P(\mathbf{z} \mathbf{x})$ and our approximation using the encoder $Q(\mathbf{z} \mathbf{x})$.	(D) $D_{\text{KL}}[Q(\mathbf{z} \mathbf{x}) P(\mathbf{z})]$	This measures the «distance» between the distribution of $\mathbf{z}$ produced by the encoder and a prior distribution over $\mathbf{z}$

Observation: we want to maximize $P(\mathbf{x})$ so we need to maximize the RHS. The **problem** is that $P(\mathbf{z}|\mathbf{x})$ is unknown so we cannot compute term (B). **Solution:** since term (B) is ≥ 0 then we can neglect it and we maximize the RHS, this leads us to a lower bound of the true objective function. Indirectly we are maximizing the true objective function.

$$\log P(\mathbf{x}) \geq \underbrace{\overset{\text{(C)}}{E_{\mathbf{z} \sim Q}[\log P(\mathbf{x}|\mathbf{z})]} - \overset{\text{(D)}}{D_{\text{KL}}[Q(\mathbf{z}|\mathbf{x})||P(\mathbf{z})]}}_{\text{Evidence lower bound (ELBO)}}$$

We train our VAE by maximizing the ELBO. But how can we do that?

9.4.5 Maximizing the ELBO

First consider the (D) term in the Evidence lower bound. Recall that $Q(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \mu(\mathbf{x}), \Sigma(\mathbf{x}))$ (distribution approximated by the encoder) and we choose $P(\mathbf{z}) = \mathcal{N}(\mathbf{z}, \mathbf{0}, \mathbf{I})$ (prior distribution). The divergence of between two Gaussians is a well known derivation We skip the steps (see the slides), in the end we get the following expression:

$$\begin{aligned}
 D_{\text{KL}}[Q(\mathbf{z}|\mathbf{x})||P(\mathbf{z})] &= \frac{1}{2} [\text{tr}(\Sigma(\mathbf{x})) + \mu(\mathbf{x})^\top \mu(\mathbf{x}) - k - \log \det \Sigma(\mathbf{x})] \\
 &= \frac{1}{2} \sum_{i=1}^k (\sigma_i^2 + \mu_i^2 - 1 - \log \sigma_i^2)
 \end{aligned} \tag{9.12}$$

where $\text{tr}(\cdot)$ is the sum of the elements of the diagonal, k is the dimension of the vector $\mathbf{z}$. The formula above is very easy to compute, all the terms are given by the output of NN that implements the encoder.

Now consider the (C) term in the Evidence lower bound. We cannot solve the expected value in closed form because $P(\mathbf{x}|\mathbf{z})$ is computed by a NN. What we can do instead is to approximate the expected value using Monte Carlo approach: draw L samples of $\mathbf{z}$ and use empirical average.

$$E_{\mathbf{z} \sim Q}[\log P(\mathbf{x}|\mathbf{z})] \approx \frac{1}{L} \sum_{l=1}^L \log P(\mathbf{x}|\mathbf{z}_l) \quad (9.13)$$

We have already used this approximation for the Naive approach to VAEs and we have seen that in order to approximate the distribution we need a huge number of samples $\mathbf{z}_l$. However in this case the story is different: the value of L is much smaller thanks to the presence of the encoder which tries to approximate the distribution of $P(\mathbf{z}|\mathbf{x})$, i.e., it tries to generate meaningful latent variables. The naive approach instead was blind on the generation of $\mathbf{z}$ so the number of latent variables required was extremely high. Usually in VAEs $P(\mathbf{x}|\mathbf{z}_l)$ is assumed to be Gaussian and we know that the maximization leads to MSE loss function which is easy to handle. We will see this later.

9.4.6 Loss function

The loss function that we want to minimize is the **negative ELBO**:

$$\mathcal{L}(\mathbf{x}) = -\frac{1}{L} \sum_{l=1}^L \log P(\mathbf{x}|\mathbf{z}_l) + \frac{1}{2} \sum_{i=1}^k (\sigma_i^2 + \mu_i^2 - 1 - \log \sigma_i^2) \quad (9.14)$$

But we have a **problem**: $\mathbf{z}$ is sampled from the distribution $Q(\mathbf{z}|\mathbf{x})$ and sampling is a discontinuous operation, so we cannot backpropagate the gradient through it.

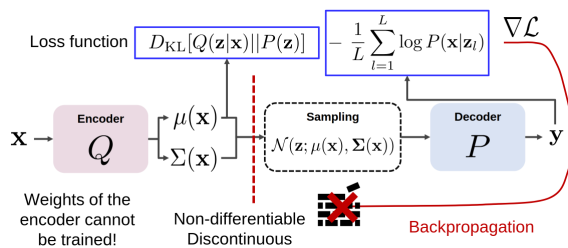


Figure 9.17: Visual explanation of the problem.

The loss contains an approximation of the expected value

$$E_{\mathbf{z} \sim Q}[\log P(\mathbf{x}|\mathbf{z})] \approx \frac{1}{L} \sum_{l=1}^L \log P(\mathbf{x}|\mathbf{z}_l)$$

If we take the gradient of this we have

$$\nabla_{\mathbf{w}_Q} E_{\mathbf{z} \sim Q(\dots, \mathbf{w}_Q)}[\log P(\mathbf{x}|\mathbf{z})]$$

The distribution used in the expectation depends on the parameters too!

Figure 9.18: Mathematical explanation of the problem: by changing the weights of the encoder we change how we sample from the distribution Q .

To avoid this problem we use a **reparametrization trick**: we move the sampling operation to an **input layer**. With this the sampling operation is independent from

the weights of the encoder.

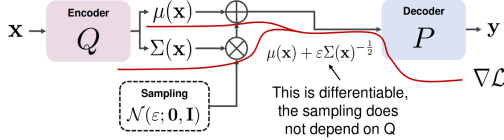


Figure 9.19: Reparametrization trick visualized.

Now the expectation in the loss is wrt the distribution of ϵ

$$\nabla_{\mathcal{W}_Q} E_{\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} [\log P(\mathbf{x}|\mathbf{z})]$$

This allows bringing the gradient inside with no complications
Using the reparametrization trick we obtain a loss function that
can be optimized using SGD-based algorithms

Figure 9.20: Mathematical reason.

Now the latent variable can be expressed as $\mathbf{z} = \mu(\mathbf{x}) + \epsilon \Sigma(\mathbf{x})^{-1/2}$ where $\epsilon \sim N(\epsilon; \mathbf{0}, \mathbf{I})$.

To summarize, by exploiting the trick, the ELBO loss function used to train the VAEs is:

$$\mathcal{L}(\mathbf{x}) = -\frac{1}{L} \sum_{l=1}^L \log P(\mathbf{x}|\mathbf{z}_l = \mu(\mathbf{x}) + \epsilon \Sigma(\mathbf{x})^{-1/2}) + \frac{1}{2} \sum_{i=1}^k (\sigma_i^2 + \mu_i^2 - 1 - \log \sigma_i^2) \quad (9.15)$$

and since $P(\mathbf{x}|\mathbf{z})$ is assumed to be a Gaussian distribution, we already know that the log of a Gaussian is proportional to the MSE. Therefore:

$$\mathcal{L}(\mathbf{x}) = \frac{1}{L} \sum_{l=1}^L \frac{\|\hat{\mathbf{x}}_l - \mathbf{x}\|^2}{2\sigma_P^2} + \frac{1}{2} \sum_{i=1}^k (\sigma_i^2 + \mu_i^2 - 1 - \log \sigma_i^2) \quad (9.16)$$

where:

- $\hat{\mathbf{x}}_l$ is the sample generated by the decoder from the latent variable $\mathbf{z}_l$.
- σ_P^2 is the variance and is used as an hyperparameter¹.

The variance σ_P^2 is used as a parameter that tune the **relative importance** of the reconstruct term and the KL term. Plays a similar role of the parameter λ that we have introduced for regularize NNs (in this case $\sigma_P^2 \propto 1/\lambda$): the higher σ_P^2 the less weight is put onto the reconstruction error, which is tolerated to be higher; if instead is small, then we ask for more fidelity to the training data samples.

¹In subsection 1.3.1 this term is not present because the variance was set to the identity matrix.

$$\mathcal{L}(\mathbf{x}) = \frac{1}{L} \sum_{l=1}^L \|\hat{\mathbf{x}}_l - \mathbf{x}\|^2 / (2\sigma_P^2) + \frac{1}{2} \sum_{i=1}^k (\sigma_i^2 + \mu_i^2 - 1 - \log \sigma_i^2)$$

RECONSTRUCTION TERM

Pushes the VAE to reconstruct the input samples as closely as possible. If this was not present, the VAE would not learn anything useful from training data, and would generate samples that do not reflect $p(\mathbf{x})$

KL-DIVERGENCE TERM

Regularizes the VAE pushing it to learn latent variables that follow an easy Gaussian distribution. If this was not present, the VAE would collapse onto learning to only reconstruct input data and not to be general enough to produce **new** samples

Figure 9.21: Meaning of the terms in the negative ELBO loss function. The KL-divergence term plays a similar role of Ω seen in section 9.2 for the traditional autoencoders.

9.4.7 Test time: generate new samples

Now that we have trained our VAE, how can we generate new data? Well, we can just get rid of the encoder since it is no longer useful and we simply fed the decoder with a sample from a Gaussian distribution.

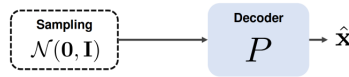
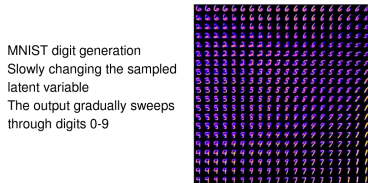


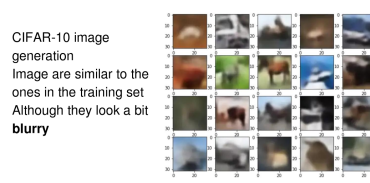
Figure 9.22: The latent variable is generated by a Gaussian distribution.

The decoder generates $\hat{\mathbf{x}}$ through a deterministic mapping. The source of randomness is given by the input Gaussian variable. Why this works? We have seen in the previous subsection that the goal of the encoder is to reduce the KL-divergence between two Gaussians distributions and we have have defined the prior distribution $P(\mathbf{z})$ as a Gaussian one (zero mean and identity covariance matrix). Therefore the decoder is getting in input very similar values as the ones that was previously given by the encoder in the training session.



MNIST digit generation
Slowly changing the sampled latent variable
The output gradually sweeps through digits 0-9

Figure 9.23: MNIST example.



CIFAR-10 image generation
Image are similar to the ones in the training set
Although they look a bit blurry

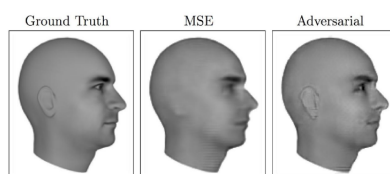
Figure 9.24: CIFAR-10 example.

Chapter 10

Generative Adversarial Networks

10.1 VAE drawbacks

GANs are generative models that was introduced to solve the problems related to VAEs. The biggest problem of VAEs is that the output generated tends to be blurry (with images, but similar results hold in general), that is, the generated image lacks in details.



But why this happen in principle? One of the reasons is the MSE output criterion which assumes a Gaussian distribution for $P(\mathbf{x}|\mathbf{z})$. VAEs assume a Gaussian distribution because otherwise the loss function (that we have derived) is intractable (no mathematical expression can be found).

Assuming a Gaussian distribution has some drawbacks:

- It cannot capture multi-modality¹.
- MSE has a preference for many small errors rather than few outliers: this means that the loss function penalizes being "too specific" (having high value). But in the example of the faces, high values could reflect the presence of a detail.

¹From wikipedia: "In statistics, a multimodal distribution is a probability distribution with more than one mode (i.e., more than one local peak of the distribution)".

The MSE tries to output something that gets small error in all cases, which means that the output is an average of many head positions that appear in the training set.

Same reasoning applies to the choice of the prior distribution $P(\mathbf{z})$: is assumed to be Gaussian because is easy and tractable. VAEs are biased by the choice of the prior $P(\mathbf{z})$ and $P(\mathbf{x}|\mathbf{z})$, and for very complex data those assumptions lead to bad results (blurry outputs) even with an infinite training set size.

10.2 GAN approach: implicit density models

VAEs model distributions explicitly: they try to construct an expression exploiting likelihood/prior distributions. Instead GANs model distribution implicitly: they encapsulate the density in a black box. The intuition behind: the less we model the distributions, the more the NN can learn by itself. The approach used by GANs is completely different from the one used by VAEs.

How can we train implicit models? The idea is to set up the learning process as a **2-player game**:

- **Player 1 - Generator:** its goal is to generate data sample; it needs to learn how to do it.
- **Player 2 - Discriminator:** its goal is to distinguish between real samples from $p(\mathbf{x})$ and fake samples generated by the generator.

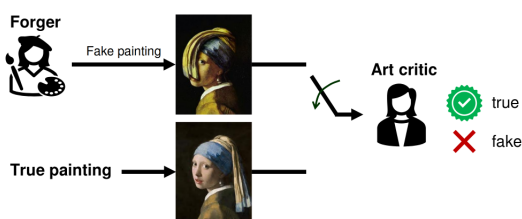


Figure 10.1: GAN idea.

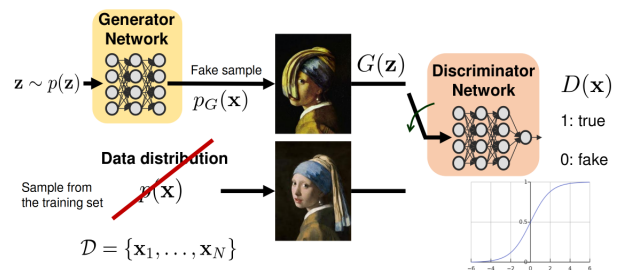


Figure 10.2: Adversarial framework.

Note that the discriminator is a tool that we use in order to train the generator, in the end what we care for is the generator. The key point is that the two are connected: the better the discriminator becomes at distinguishing real samples from fake ones, the more the generator is pushed to improve itself in order to fool the discriminator.

10.3 Adversarial framework formalized

The **discriminator** has to learn how to distinguish generated samples from the true ones. So it's a 2 class classification problem.



The discriminator has to learn the decision boundary

$$D(\mathbf{x}; \mathcal{W}_D) = p(\mathbf{x} \text{ is a sample from } p(\mathbf{x}))$$

The **generator** has to learn how to generate realistic samples. We will see that, instead of modeling the error for the generator, we will train it through the discriminator using a specific loss function. The more the discriminator makes mistakes, the better the generated samples are. Ideally we would like to have:

$$\mathbf{x} = G(\mathbf{z}) \sim p_G(\mathbf{x}) \approx p(\mathbf{x}).$$

Note that $\mathbf{z} \sim p(\mathbf{z})$ is again the latent variable that we can sample from a Gaussian distribution. Also note that in the adversarial framework, the generator is a deterministic function, the randomness is given by $\mathbf{z}$.

10.3.1 Minmax game formulation

The two players, Generator and Discriminator, have opposite goals. The value function for the game can be written as follows:

$$V(G, D) = E_{\mathbf{x} \sim p(\mathbf{x})}[\log D(\mathbf{x})] + E_{\mathbf{z} \sim p(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))] \quad (10.1)$$

where we compute one expectation over the latent distribution, one over the true distribution. $D(\mathbf{x})$ should output 1 if $\mathbf{x}$ is a true sample and 0 if it is a generated sample from $G(\mathbf{z})$.

This value function can be used as the loss function to be optimized:

- The Discriminator wants to **maximize** it $\rightarrow$ means maximizing the log-likelihood.
- The Generator wants to **minimize** it $\rightarrow$ means fooling the discriminator.

That is, they have opposite goals! The optimization problem can be written as

$$\min_G \max_D V(G, D) \quad (10.2)$$

where we optimize the value function $V(G, D)$ with respect to the weights of the generator G and the discriminator D .

10.3.2 Expected outcome of the game

In this subsection we give some theoretical results that help understand what is the result that we expect. We are going to see that *ideally* the adversarial framework converges to a point in which the generator has learned how to perfectly replicate the data distribution. In other words we want to show that the minmax formulation actually converges from a theoretical point of view. In practice we will see that those results don't hold but they are however useful to understand what should happen in the ideal case.

In the following we consider the function as «real »probability function that are not related to NN.

Proposition 2. *Assume that the generator G is fixed. Then the optimal discriminator D is obtained as*

$$D^*(G) = \frac{p(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})}. \quad (10.3)$$

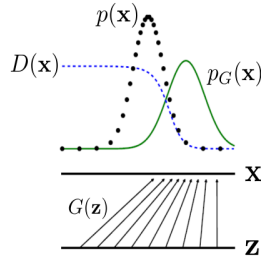


Figure 10.3: Optimal discriminator visualized. The changepoint in the decision is at the intersection between the two densities.

Before starting the proof we introduce a tool that we will need. This tool comes from **calculus of variations** and allow us to solve a maximization of a functional with respect to a function. So, consider the following functional:

$$\mathcal{F}[y(x)] = \int_a^b f(x, y(x), \dot{y}(x)) dx$$

our goal is to find the function $y(x)$ that maximize it.

Theorem 3 (Euler-Lagrange theorem). *Assume that f has continuous partial derivatives w.r.t. $y, \dot{y}, x$ and $y, \dot{y}$ have continuous partial derivatives w.r.t. x . If $y(x)$ is a function that minimizes/maximizes the functional $\mathcal{F}[y(x)]$ over $[a, b]$, then the following differential equation must be satisfied:*

$$\frac{\partial f}{\partial y} - \frac{d}{dx} \frac{\partial f}{\partial \dot{y}} = 0 \quad (10.4)$$

Note that the maximization/minimization of a functional is similar to a maximization/minimization of a function f , the difference is the second term in the equation.

Proof. Since the generator is fixed, then discriminator objective is a maximization problem: $\max_D V(D, G)$. We have:

$$V(D, G) = \int_{\mathbf{x}} \log[D(\mathbf{x})]p(\mathbf{x})d\mathbf{x} + \int_{\mathbf{z}} \log[1 - D(G(\mathbf{z}))]p(\mathbf{z})d\mathbf{z}$$

Since G is fixed, we can rewrite the second expectation as a function of $\mathbf{x}$ since $G(\mathbf{z})$ lies in the same space as the true data:

$$\int_{\mathbf{z}} \log[1 - D(G(\mathbf{z}))]p(\mathbf{z})d\mathbf{z} = \int_{\mathbf{x}} \log[1 - D(\mathbf{x})]p_G(\mathbf{x})d\mathbf{x}$$

Therefore:

$$V(D, G) = \int_{\mathbf{x}} [\log[D(\mathbf{x})]p(\mathbf{x}) + \log[1 - D(\mathbf{x})]p_G(\mathbf{x})]d\mathbf{x}$$

Maximizing $V(D, G)$ is a maximization of a **functional**, this problem can be rewritten as $\max_D V(D, G) = \max_D \int_{\mathbf{x}} f(D, \mathbf{x})d\mathbf{x}$ and the proper way to solve this is using *calculus of variations*.

Using the Euler-Lagrange theorem we have that:

$$\frac{\partial f(D, \mathbf{x})}{\partial D} = 0$$

where the second term is zero since the derivative of D doesn't appear in our functional.

$$\begin{aligned}
 \frac{\partial f(D, \mathbf{x})}{\partial D} &= 0 \\
 \Leftrightarrow \frac{\partial}{\partial D} [\log D(\mathbf{x})p(\mathbf{x}) + \log(1 - D(\mathbf{x}))p_G(\mathbf{x})] &= 0 \\
 \Leftrightarrow \frac{p(\mathbf{x})}{D(\mathbf{x})} - \frac{p_G(\mathbf{x})}{1 - D(\mathbf{x})} &= 0 \\
 \Leftrightarrow p(\mathbf{x}) - p(\mathbf{x})D(\mathbf{x}) &= p_G(\mathbf{x})D(\mathbf{x}) \\
 \Leftrightarrow D^*(\mathbf{x}) &= \frac{p(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})}
 \end{aligned}$$

□

Theorem 4. *From the optimal D^* , the global minimum of the minmax game value function is obtained if and only if $p_G(\mathbf{x}) = p(\mathbf{x})$. The minimum in such case has a value of $-\log 4$.*

Intuitively, when $p_G(\mathbf{x}) = p(\mathbf{x})$, the discriminator G cannot distinguish between real and fake samples and therefore we expect $D(\mathbf{x}) = 0.5$ for every possible sample.

Proof. Rewrite the functional by substituting D^* in place of D :

$$V(D^*, G) = \int_{\mathbf{x}} [\log[D^*(\mathbf{x})]p(\mathbf{x})d\mathbf{x} + \int_{\mathbf{x}} \log[1 - D^*(\mathbf{x})]p_G(\mathbf{x})d\mathbf{x}]$$

where we have omitted the step in which the second term of the functional is written as a function of $\mathbf{x}$ (see previous proof). Now, since D is fixed, the optimization problem is just a minimization problem.

Lets do some algebraic manipulation:

$$\begin{aligned}
 V(D^*, G) &= \int_{\mathbf{x}} \log[D^*(\mathbf{x})]p(\mathbf{x})d\mathbf{x} + \int_{\mathbf{x}} \log[1 - D^*(\mathbf{x})]p_G(\mathbf{x})d\mathbf{x} \\
 &= \int_{\mathbf{x}} \log\left[\frac{p(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})}\right]p(\mathbf{x})d\mathbf{x} + \int_{\mathbf{x}} \log\left[1 - \frac{p(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})}\right]p_G(\mathbf{x})d\mathbf{x} \\
 &= \int_{\mathbf{x}} \log\left[\frac{p(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})}\right]p(\mathbf{x})d\mathbf{x} + \int_{\mathbf{x}} \log\left[\frac{p_G(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})}\right]p_G(\mathbf{x})d\mathbf{x} \\
 &= \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} \left[\log\left(\frac{p(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})}\right) \right] + \mathbb{E}_{\mathbf{x} \sim p_G(\mathbf{x})} \left[\log\left(\frac{p_G(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})}\right) \right]
 \end{aligned} \tag{10.5}$$

If we impose $p_G(\mathbf{x}) = p(\mathbf{x})$ we get $D^*(\mathbf{x}) = 0.5$. And if we substitute in the last formulation

above we get:

$$\mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [-\log 2] + \mathbb{E}_{\mathbf{x} \sim p_G(\mathbf{x})} [-\log 2] = -\log 4 \quad (10.6)$$

Now we want to prove that the above is actually the minimum value of the functional.

To do so we subtract 10.6 to 10.5 and we get:

$$\begin{aligned} V(D^*, G) + \log 4 &= \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} \left[\log \frac{p(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})} + \log 2 \right] + \mathbb{E}_{\mathbf{x} \sim p_G(\mathbf{x})} \left[\log \frac{p_G(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})} + \log 2 \right] \\ &= \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} \left[\log \frac{2p(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})} \right] + \mathbb{E}_{\mathbf{x} \sim p_G(\mathbf{x})} \left[\log \frac{2p_G(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})} \right] \end{aligned}$$

$$\begin{aligned} V(D^*, G) &= -\log 4 + \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} \left[\log \frac{2p(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})} \right] + \mathbb{E}_{\mathbf{x} \sim p_G(\mathbf{x})} \left[\log \frac{2p_G(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})} \right] \\ &= -\log 4 + \underbrace{D_{\text{KL}} \left(p \parallel \frac{p + p_G}{2} \right)}_{\geq 0} + \underbrace{D_{\text{KL}} \left(p_G \parallel \frac{p + p_G}{2} \right)}_{\geq 0} \\ &\geq -\log 4 \end{aligned}$$

And this proves that V cannot assume values smaller than $-\log 4$ and this value is only achieved when $p_G(\mathbf{x}) = p(\mathbf{x})$. $\square$

10.4 GAN training

In the previous subsection we have seen that if we fix the generator then the best discriminator can be computed. We have also seen that, if we have the optimal discriminator, then the generator needs to perfectly approximate the real data distribution in order to minimize the value function $V(D, G)$. The above shows that the value function that we have designed behaves like we want: we get a generator that perfectly approximate the real data distribution and a generator that cannot distinguish between real and fake samples.

In theory this works, in practice it doesn't because we work with neural networks that approximate probability distribution function and not directly with distribution functions. This invalidate the theoretical results. **However** the algorithm to train GANs is derived from the two steps discussed above.

Iterative approach:

1. Fix the generator (aka do not change its weights) and **train the discriminator to convergence**. Ideally after this step we have the optimal D for the given G .
2. Fix the (optimal) discriminator found in previous step and train the generator for (e.g.) 1 step: improve the generator ability to fool the discriminator.

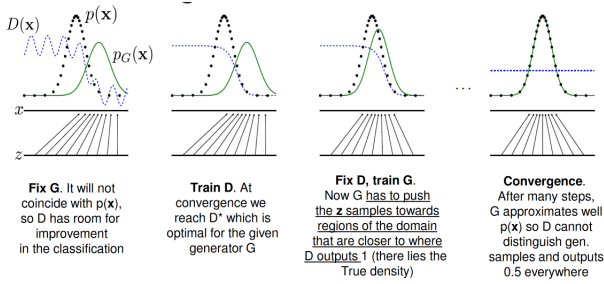


Figure 10.4: Ideal behaviour of the algorithm.

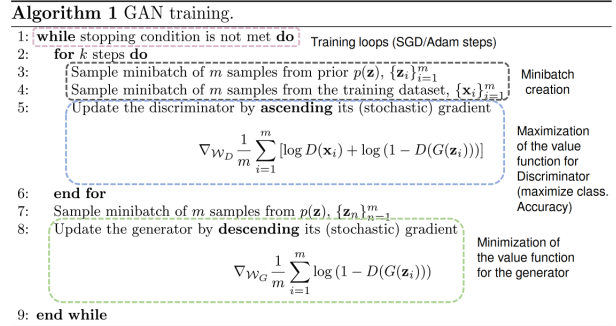


Figure 10.5: Pseudocode.

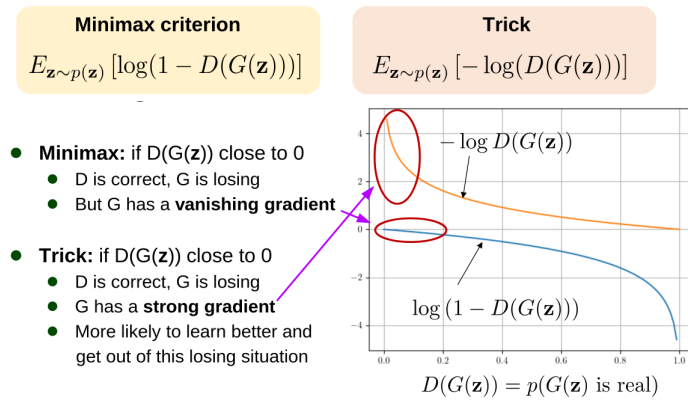
The problem of the ideal iterative algorithm is that it is infeasible because the discriminator needs to converge for each generator training step (too computationally complex). In practice the discriminator is trained for a few steps denoted by k (hyperparameter). Note also that we can add another hyperparameter that indicates for how many steps the generator is trained.

10.4.1 Resistant to overfitting

By construction, GANs are particularly resistant to overfitting. This is due to the fact that the generator G never sees the training data directly and the gradients of G are only obtained through D (which embeds information about the training data).

10.4.2 Diminishing gradients

The minmax algorithm doesn't work well in practice because of how the minimization function is defined for the generator. Instead of minimizing the log-probability of D of being correct it is better to minimize the negative log-probability of being wrong (i.e. maximizing the probability of D of being wrong). This simple trick can be easily understand if we look the plots of the objectives.



So we need to update the pseudocode used to train GANs.

Algorithm 2 GAN training.

```

1: while stopping condition is not met do
2:   for  $k$  steps do
3:     Sample minibatch of  $m$  samples from prior  $p(\mathbf{z})$ ,  $\{\mathbf{z}_i\}_{i=1}^m$ 
4:     Sample minibatch of  $m$  samples from the training dataset,  $\{\mathbf{x}_i\}_{i=1}^m$ 
5:     Update the discriminator by ascending its (stochastic) gradient

```

$$\nabla_{\mathcal{W}_D} \frac{1}{m} \sum_{i=1}^m [\log D(\mathbf{x}_i) + \log(1 - D(G(\mathbf{z}_i)))]$$

```

6:   end for
7:   Sample minibatch of  $m$  samples from  $p(\mathbf{z})$ ,  $\{\mathbf{z}_n\}_{n=1}^m$ 
8:   Update the generator by descending its (stochastic) gradient

```

$$\nabla_{\mathcal{W}_G} \frac{1}{m} \sum_{i=1}^m -\log(D(G(\mathbf{z}_i)))$$

```

9: end while

```

10.4.3 Non convergence: bouncing problem and label smoothing

In general we know that NN training doesn't converge because of the highly non-convex optimization function. But if we have convex optimization, then all the algorithm that we have introduced actually converges. GANs do not solve a simple optimization function but a **game** and this is a much more complex problem. For minmax games the convergence do not always hold even for convex problems, and in such cases the SGD keeps **bouncing between good solutions for one of the two players**.

One trick to mitigate the problem is the **one-sided label smoothing**. When training the discriminator we use the following labels:

$$\begin{aligned}
 1 - \alpha, & \quad \text{for } \mathbf{x} \sim p(\mathbf{x}) \\
 0, & \quad \text{for } \mathbf{x} \sim p_G(\mathbf{x})
 \end{aligned}$$

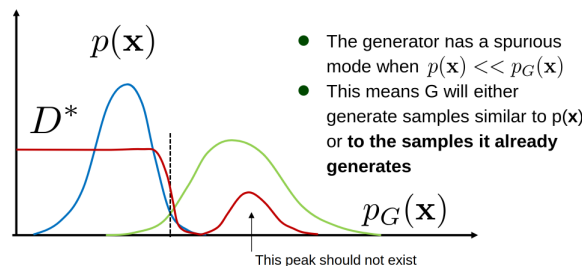
with typically $\alpha \approx 0.1$. With this smoothing trick we **penalize discriminator overconfidence**, indeed if it outputs exactly 1 then it is penalized.

Why is the smoothing useful? One of the causes of overconfidence of the discriminator is that it uses too few features to classify the input. But if this is the case then the generator can exploit this fact to fool the discriminator: it can produce samples focusing only on those few features that the discriminator considers to be important. As a result the output generated by the generator do not reflect the true data distribution. So the idea is to penalize overconfidence so that the generator is forced to generate full-shaped outputs.

Why one-sided? Why we do not smooth the 0-labels by increasing it by β factor? This option was tested but it has a big problem. By using two-sided smoothing labels the optimal discriminator has the following form:

$$D^*(G) = \frac{(1 - \alpha)p(\mathbf{x}) + \beta p_G(\mathbf{x})}{p(\mathbf{x}) + p_G(\mathbf{x})}$$

When β is zero, then smoothing by α does nothing but scale down the optimal value of the discriminator. When β is nonzero, the shape of the optimal discriminator function changes. In particular, in a region where $p(\mathbf{x})$ is very small and $p_G(\mathbf{x})$ is larger, $D^*(\mathbf{x})$ will have a peak near the spurious mode of $p_G(\mathbf{x})$.



Since G push z samples towards region of the domain that are closer to where D outputs 1, then if D has a peak near the mode of $p_G(\mathbf{x})$, the generator will be trained to either produce samples that resemble the data or to produce samples that resemble the samples it already makes.

10.4.4 Non convergence: mode collapse and minibatch discrimination

Mode collapse appears when the real data is described by a multi-modal distribution and the generator collapse to generate always the **same mode** (partial collapse) or always the **same sample** (complete collapse)². In those situation the generator is focused to a specific mode of the input distribution: it is not capable to describe the entire distribution.

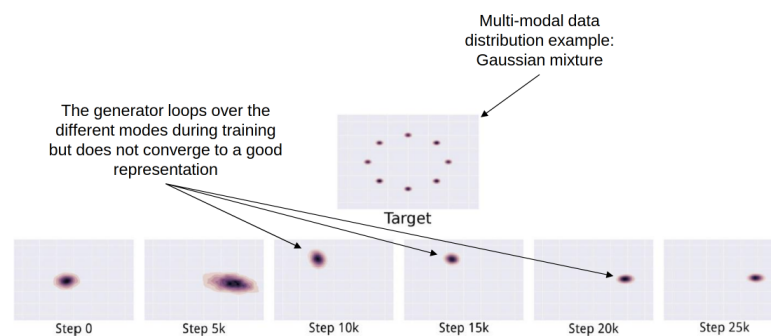


Figure 10.6: Collapse causes non convergence.

Minibatch discrimination is a technique used to mitigate the problem related to mode collapse. But what happens during mode collapse?

1. D assigns high prob. of being real to a specific fake sample
2. As a reaction, G will produce that sample more often
3. G collapses to generate only that sample
4. D realizes that that sample is fake, but the cycle restarts

Figure 10.7: During mode collapse the following steps are repeated.

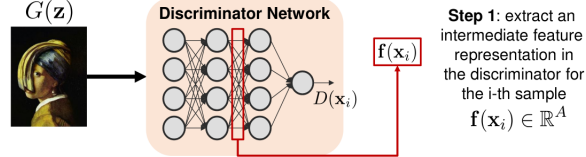
Intuitively the problem is that, if G finds out that some samples are enough for fooling D , then it keeps outputting them. But this leads the generator focusing to a specific mode and after some time the discriminator realizes that the fake samples are very similar to each other but very different from the real samples, so it starts penalizing the generator. As a result we get stuck in a loop.

One idea to mitigate this problem is to incentive G to differentiate its outputs. The idea behind minibatch discrimination is to introduce dependencies between the gradients of a minibatch. Lets see how to introduce this dependency step by step.

²Complete collapse means that the generator has learned a dirac delta like distribution function.

1. **Step 1:** extract an intermediate feature representation in the discriminator for the i -th sample $\mathbf{f}(\mathbf{x}_i) \in \mathbb{R}^A$. We do the same thing for each sample in the minibatch (say n samples):

$$\mathbf{f}(\mathbf{x}_i) \quad \forall i = 1, \dots, n$$



2. **Step 2:** transform the features through a (learned) tensor T .

$$M_i = \mathbf{f}(\mathbf{x}_i) \cdot T \quad \forall i = 1, \dots, n$$

where $T \in \mathbb{R}^{A \times B \times C}$ and therefore $M_i \in \mathbb{R}^{B \times C}$. Note that A, B, C are all hyperparameters and note that we have one matrix M_i for each sample in the minibatch.

3. **Step 3:** consider all possible pairs between the samples in the minibatch and compute the scalar scores for each pair:

$$c_b(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\|M_{i,b} - M_{j,b}\|_1), \quad \forall b = 1, \dots, B$$

where $M_{i,b}$ is the b -th row of matrix M_i .

4. **Step 4:** compute output for the i -th sample

$$o_b(\mathbf{x}_i) = \sum_{j=1}^n c_b(\mathbf{x}_i, \mathbf{x}_j) \quad \forall b = 1, \dots, B$$

then concatenate into a single vector:

$$\mathbf{o}(\mathbf{x}_i) = [o_1(\mathbf{x}_1), \dots, o_B(\mathbf{x}_i)]^T \quad \forall i = 1, \dots, n$$

5. **Step 5:** concatenate the output with the original features:

$$[\mathbf{f}(\mathbf{x}_i)^T, \mathbf{o}(\mathbf{x}_i)^T]^T$$

and the above is passed to the next layer, it is effectively the new (augmented) feature vector).

The procedure above is done separately for true samples minibatch and for fake samples minibatch. With this approach each sample of the minibatch depends in part from the features of other samples in the same minibatch, making it more difficult to collapse to a single generated sample.

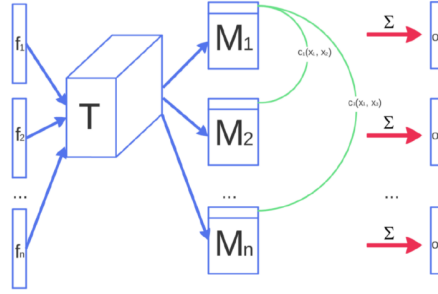


Figure 10.8: Minibatch discrimination.

10.5 Conditional GANs

After the training process is ended the discriminator is thrown away and the generator is used to generate images. In some applications is useful to control the generation process in order to generate images with specific features instead of sampling a random one from $p(\mathbf{x})$. With traditional GANs we cannot control the generation process, so for this reason Conditional GANs were introduced. In practice we observe that the CGANs perform better than standard GANs.

Idea behind: exploit side information and use adversarial learning conditioned on the information that we have. Typically we use the **class label** as side information fed in as one-hot vector, or we can use another network that extract relevant features. For instance consider the example in MNIST dataset: we can provide the label 3 and we expect to generate images that shows the digit 3 in output.

In this case the value function is defined as

$$V(G, D) = E_{\mathbf{x} \sim p(\mathbf{x})}[\log D(\mathbf{x}|\mathbf{y})] + E_{\mathbf{z} \sim p(\mathbf{z})}[\log(1 - D(G(\mathbf{z}|\mathbf{y})))] \quad (10.7)$$

but training happens in the same way.

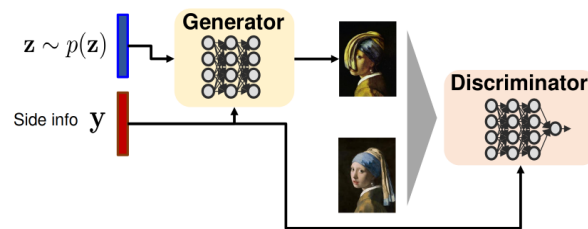


Figure 10.9: Conditional GAN structure. The label can be provided to both G and D or only to G . The discriminator can be trained considering binary classification task or multi-class classification.

Chapter 11

Continual Learning

Until now we have only considered supervised/unsupervised learning and in all the types of problems that we have addressed, the NN was trained to have good performances in a specific, single, immutable task. But it is possible to train a NN in order to make it good in multiple (sequential) tasks?

11.1 Continual learning definition

Consider two sequential learning tasks, for simplicity we consider classification for now. The first task is **Task A**, it has its associated dataset A and a probability distribution $p_A(\mathbf{x})$. The second task is **Task B**, it has its associated dataset B and a probability distribution $p_B(\mathbf{x})$. The constraint is that we do not have the access to both the datasets at the same time. We first need to train the NN on task A without having dataset B and then we need make the NN good at task B without having the (entire) dataset A .

The **ideal solution** would be to join the two datasets into a single dataset $D = \{A, B\}$ and then train a NN using D . But as we said, we cannot do that.

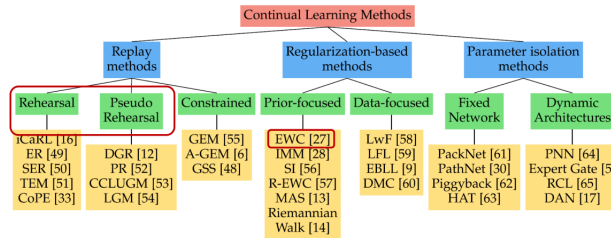


Figure 11.1: Different approaches to solve the continual learning method.

11.2 Catastrophic forgetting

The first naive solution can be: train the NN to be good at task A , then train it to be good at task B . The problem is that, this approach leads to the so called **catastrophic forgetting problem**.

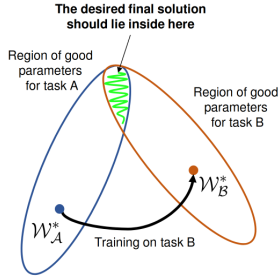


Figure 11.2: Catastrophic forgetting visually.

1. Start from a random initialization and optimize loss A
 2. Optimize loss B
 - i. Starting from the optimum for A
 - ii. Starting from a random point
- The same exact result is obtained regardless of 2.i or 2.ii

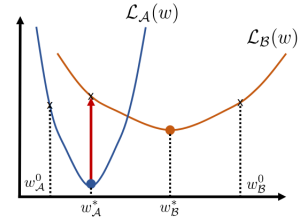


Figure 11.3: Convex loss example.

The reason behind catastrophic forgetting is not difficult to understand: when we re-train the NN on task B , then the previously optimal learned weights are used just as a simple initialization point. This is not just a theoretical result, we observe this in practical applications.

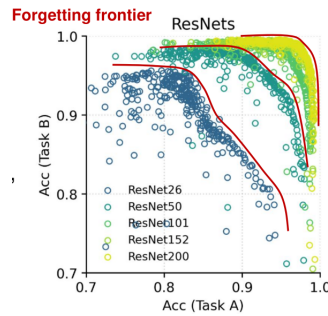


Figure 11.4: CIFAR-10 dataset. The 10 classes are separated in two classification task, each with 5 classes. A/B is the accuracy trade-off. ResNet26 is the smallest model, ResNet200 is the biggest.

It is worth noting that the larger the model, the less it "forgets" previously learned tasks. This is because a large model has a vast number of weights, which effectively serve as memory. When the network is retrained to perform well on task B , many of its weights remain unchanged, allowing it to retain its performance on task A . However, as more and more tasks are added sequentially, this property no longer holds, and the problem of catastrophic forgetting arises regardless of the network's size.

11.3 Rehearsal: replay-based method

The idea of rehearsal is very simple. For any new task that we introduce, we have a new dataset to use. Instead of training the network only on the new data, we train it on a mixture of new and old samples (to avoid storing the full old dataset). In general there exist many strategies for selecting representative old data points.

Rehearsal is a very simple idea but often it is the best choice among all the other approaches.

Formalization Lets formalize the approach proposed. First of all we need to augment the loss function with a new (rehearsal) term:

$$\begin{aligned}\mathcal{L}(\mathcal{W}) &= \mathcal{L}_B(\mathcal{W}) + \mathcal{L}_R(\mathcal{W}) \\ &= \mathcal{L}_B(\mathcal{W}) + \frac{1}{|\mathcal{R}|} \sum_{\mathbf{r} \in R} \mathcal{L}_A(\mathbf{r}, \mathcal{W})\end{aligned}\tag{11.1}$$

where R is called **memory** (or replay) **buffer** and it contains the selected samples from dataset A . The bigger $|R|$ the better we approximate the full data distribution (optimal solution: $|R| = |A|$) but much memory usage is required.

Overfitting problem This approach is very effective if we have large memory available. But if the memory is constrained, then we have a problem. If $|R|$ is very small, then, by using the previously defined loss function, the network will be good on task A only on the selected samples in R (**overfitting** problem).

Generative Replay If we have no memory at all to store data of previously learned task, then a solution is generative replay. The idea is to train a generative model along with task A , then when training on task B the dataset B is augmented with synthetic samples created by the generative model. As a result we solve the problem of overfitting task A (since we can generate many samples) but the down side is that this solution is computationally expensive (need to train and extra model).

11.4 Elastic weight consolidation (EWC)

EWC is a regularization-based method. Before introducing it, we consider the naive regularization method. In general regularization-based methods aim to solve the catastrophic forgetting by adding a suitable regularizer term to the loss function. Idea:

add a regularizer that forces the NN to preserve the information it has learned on previous tasks.

11.4.1 Naive regularizer

In the **naive approach** the regularizer penalizes an high L2 distance from previous optimal weights:

$$\mathcal{L}(\mathcal{W}) = \mathcal{L}_B(\mathcal{W}) + \frac{\lambda}{2} \|\mathcal{W} - \mathcal{W}_A^*\|^2 \quad (11.2)$$

The naive solution fails because it doesn't consider that the regions with good parameters for the two tasks is not always spherical!

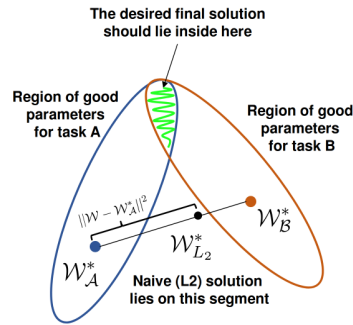


Figure 11.5: Why the naive regularizer fails. Optimizing the neural network on task B pushes the solution toward the orange point, but the regularizer pulls it back toward the black point.

11.4.2 EWC approach

The idea of EWC is to penalize more those changes that affect important weights for A and instead leave more freedom to change less important weights.

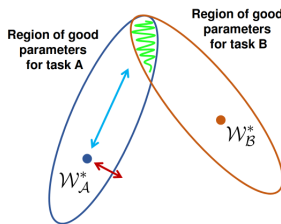


Figure 11.6: Red direction is more important than the blue.

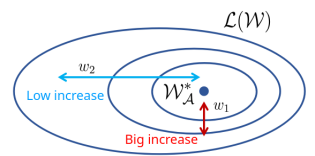


Figure 11.7: w_1 is more important than w_2 because along his direction the loss function increases.

So the idea of EWC is to use a regularizer that accounts for the importance. How can we define such metric? Instead of using the standard euclidean metric $\|\mathbf{v} - \mathbf{u}\|_2 = \sqrt{(\mathbf{v} - \mathbf{u})^T(\mathbf{v} - \mathbf{u})}$ (used by the naive approach), we add a positive semidefinite matrix $\mathbf{C}$ to get a new weighted distance function $\sqrt{(\mathbf{v} - \mathbf{u})^T \mathbf{C} (\mathbf{v} - \mathbf{u})}$.

Design the matrix $\mathbf{C}$ How to design matrix $\mathbf{C}$? We want to give more importance to the direction along which the loss has a steep increase and less importance to the direction along which the increase is smoother. To do so we can consider the curvature of the loss w.r.t. each weight.

- **Higher curvature** means loss is sharply peaked in that direction $\Rightarrow$ the weight is more important.
- **Lower curvature** means loss is more flat in that direction $\Rightarrow$ the weight is less important.

The curvature can be measured using the second derivatives/Hessian matrix but this is too expensive to compute in our case, so we need a workaround. The workaround that we consider is the **Fisher Information Matrix (FIM)**.

The **FIM measures the curvature** of the log likelihood (tool from estimation theory and statistics) and we can use it because we have seen in the first chapter that the loss of a NN is usually the negative log-likelihood.

$$\mathcal{L}(\mathcal{D}, \mathcal{W}) = -\log p(\{\mathbf{t}_n\}_{n=1}^N | \{\mathbf{x}_n\}_{n=1}^N; \mathcal{W}) = -\sum_{n=1}^N \log p(\mathbf{t}_n | \mathbf{x}_n; \mathcal{W})$$

Figure 11.8: Negative log-likelihood as loss function.

$$\mathcal{I}_{\mathcal{A}}(\mathcal{W}) = E \left[-\frac{\partial^2 \log p(\mathcal{A} | \mathcal{W})}{\partial^2 \mathcal{W}} \right] \text{Log-likelihood}$$

Depends on the task

Figure 11.9: FIM definition.

The FIM is positive semidefinite and therefore it can induce a valid metric. The problem is that, as we can see, it needs to compute the second order derivatives and we want to avoid that. To avoid this problem we exploit a very important property related to the FIM:

$$\underbrace{\mathcal{I}_{\mathcal{A}}(\mathcal{W})}_{\substack{\text{Shape} \\ |\mathcal{W}| \times |\mathcal{W}|}} = E \left[\underbrace{-\frac{\partial^2 \log p(\mathcal{A} | \mathcal{W})}{\partial^2 \mathcal{W}}}_{\substack{\text{Second-order derivative} \\ \text{(Hessian), shape } |\mathcal{W}| \times |\mathcal{W}|}} \right] = E \left[\underbrace{\left(\frac{\partial \log p(\mathcal{A} | \mathcal{W})}{\partial \mathcal{W}} \right)}_{\substack{\text{First-order derivative (Gradient)} \\ \text{Vector with } |\mathcal{W}| \text{ components} \\ \text{(column-by-row vector product)}}} \left(\frac{\partial \log p(\mathcal{A} | \mathcal{W})}{\partial \mathcal{W}} \right)^T \right]$$

By using this property we only need to compute the gradients of the loss but this is not a problem since we use backpropagation during training so this doesn't require extra effort.

Last step: replace the expectation with the empirical average over task A and also we simplify the problem by only considering the diagonal of the FIM so each element can be computed separately.

$$[\mathcal{I}_A]_{ii} \approx \frac{1}{N_A} \sum_{n=1}^{N_A} \left(\frac{\partial \log p(\mathbf{t}_n | \mathbf{x}_n, \mathcal{W})}{\partial w_i} \bigg|_{\mathcal{W}_A^*} \right)^2$$

Figure 11.10: Formula for an element in the diagonal evaluated in the optimal point $\mathcal{W}_A^*$. N_A is the size of dataset A .

$$[\mathcal{I}_A]_{ii} \approx \frac{1}{N_A} \sum_{n=1}^{N_A} \left(\frac{\partial \mathcal{L}(\mathbf{x}_n, \mathcal{W})}{\partial w_i} \bigg|_{\mathcal{W}_A^*} \right)^2$$

Figure 11.11: The log-likelihood is the loss function of our NN.

Define the EWC loss function Note that the value of $[\mathcal{I}_A]_{ii}$ is computed only once after we have trained the network on task A . So no data is stored. Finally we can define the regularized term in the loss function using the coefficients that we have derived in this section.

$$\mathcal{L}(\mathcal{W}) = \mathcal{L}_B(\mathcal{W}) + \frac{\lambda}{2} (\mathcal{W} - \mathcal{W}_A^*)^T \mathcal{I}_A (\mathcal{W} - \mathcal{W}_A^*)$$

Distance induced by the FIM

$$\mathcal{L}(\mathcal{W}) = \mathcal{L}_B(\mathcal{W}) + \frac{\lambda}{2} \sum_{i=1}^{|\mathcal{W}|} [\mathcal{I}_A]_{ii} (w_i - w_{A,i}^*)^2$$

Loss for task B
This is needed to improve performance on task B

Diagonal element of FIM
This is a weight that penalizes more the modification of important parameters for task A

Squared distance
This measures the distance from the optimal parameters for task A

The above works when we consider only two tasks but we can generalize to T tasks. The equation of the loss is obtained as follows:

$$\mathcal{L}(\mathcal{W}) = \mathcal{L}_T(\mathcal{W}) + \sum_{t=1}^{T-1} \frac{\lambda_t}{2} \sum_{i=1}^{|\mathcal{W}|} [\mathcal{I}_t]_{ii} (w_i - w_{t,i}^*)^2$$

Sum of task-specific regularizers
One for each previous task

Figure 11.12: EWC for T tasks with $T - 1$ regularizers.

As we can see from the last formula, the complexity of EWC scales linearly with the number of tasks $\Rightarrow$ it is not very scalable. This is one of the main drawbacks of this approach.

11.4.3 Results

Scenario: *permuted MNIST*. The tasks considered are multi-classification on the MNIST dataset. The dataset of the first task is the MNIST dataset, instead the dataset of task $i > 1$ is obtained by applying a fixed random permutation¹ of the pixels.

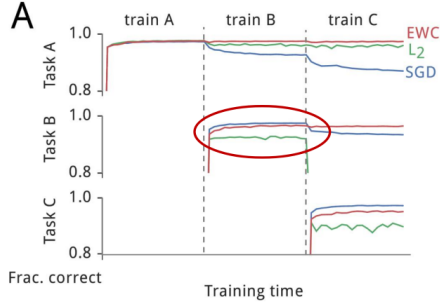


Figure 11.13: We can see that the SGD curve clearly suffers from the catastrophic forgetting. The L_2 instead preserves good accuracy on task A but prevents from learning B properly. EWC curve has good performances in all three tasks.

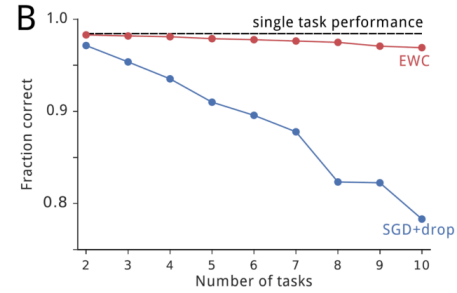


Figure 11.14: We can clearly see that EWC mitigates catastrophic forgetting but performances drop "linearly" with the increase in the number of tasks.

¹The same permutation is applied to all the image of the dataset, so that we don't lose any information.

Chapter 12

Contrastive learning

All the learning approaches (supervised/unsupervised) introduced in this entire course have a common trait: they all learn from one sample at time. This means that we do not exploit the differences among the data samples inside our dataset. In Batch Normalization we have combined somehow the samples inside the batch, but even in that case we didn't exploit the differences between them. The general idea is to exploit the differences between the data samples to achieve better results (usually contrastive learning is used for unsupervised learning and we will see one example).

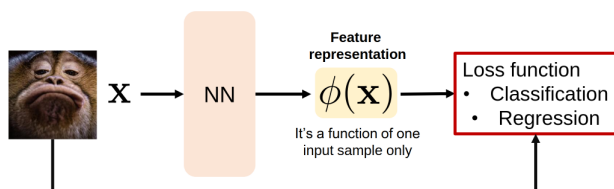


Figure 12.1: Supervised framework.

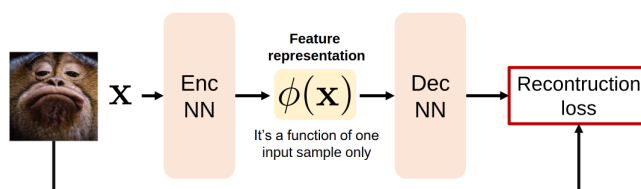


Figure 12.2: Unsupervised framework.

The problem that we want to point out is that, for both supervised and unsupervised framework, the extracted features $\phi(\mathbf{x})$ that we get after we have trained the network, are meaningful only for that specific task! They're not a good general representation of the samples in the dataset. In this chapter our goal is to define a framework that aims to find good general feature representation for our dataset and the main key behind is to leverage differences in the training samples.

Ideally we would like that different inputs should have different representations $\Rightarrow$ far from each other in feature space.

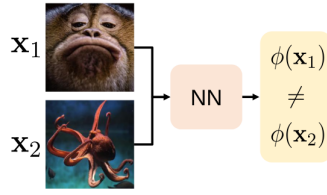


Figure 12.3: A new approach: idea behind.

12.1 Contrastive Learning framework

In this section we introduce the main components of contrastive learning.

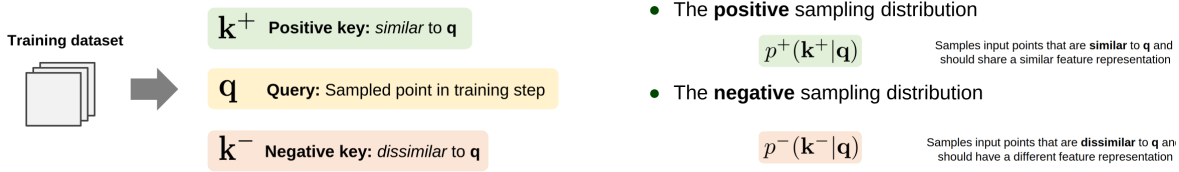


Figure 12.4: Terminology.

Figure 12.5: Two additional distribution.

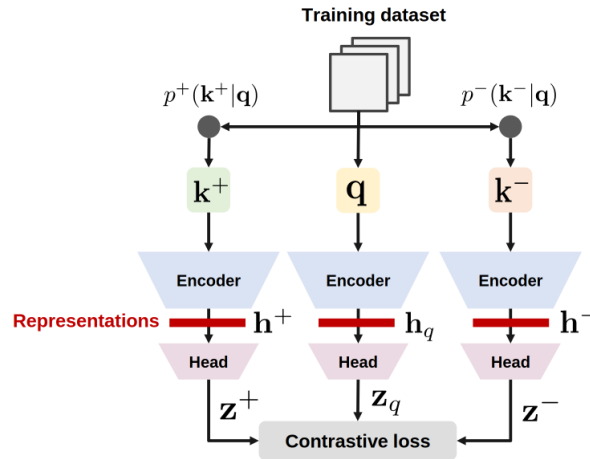


Figure 12.6: General architecture.

In the following sections, we will explain in detail each component of the general architecture shown in the figure above. The **encoder network** produces feature vectors h , which typically have lower dimensionality compared to the input query q . The main goal of the architecture is to learn a general feature representation h_q . The **(transform) head network** is the key component of the architecture: its purpose is to map the features h into new representations z that are optimized for the loss function. This means that the z

features are specifically trained to perform well with the training loss, and therefore do not necessarily provide a good general representation of the data. The inclusion of the head network is crucial, as it allows the encoder to learn more generalizable representations.

12.2 Positive and negative keys sampling

First of all we need to define what do we mean by *similar*. This depends on the task and objective that we're considering. In general we consider the following:

We consider two samples to be similar (positive pair) if they share the same semantics.

So the data representing a sample can be completely different but they must share the same semantics.

- Two augmented versions of the same image
- The same sound recorded by different microphones
- Sentences with the same meaning but different wording

Figure 12.7: Examples of positive pair.

In the following we introduce *five flavors of similarity*. Note that the notion of similarity is related to the task that we are considering, in the following we introduce only some examples.

12.2.1 Multisensory signals

Enforce similarity of the same phenomenon observed by different sensors. Example: the visual representation and the emitted sound.

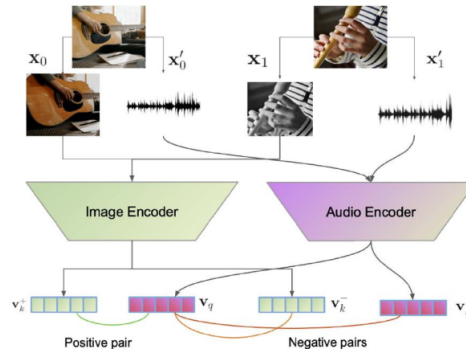


Figure 12.8: Multisensory signals example.

12.2.2 Data transformations

This is the most common and easy approach. It basically uses the techniques that we have seen for data augmentation. Different transformations of the same image are positive pairs (same semantics). Note that those transformations depend on the type of data we're dealing with!

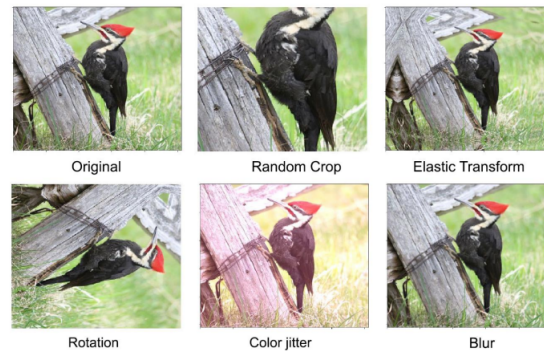


Figure 12.9: Data transformations example.

The negative key can be another image taken from the training set (we can also apply a transformation to this image).

12.2.3 Context-instance relationship

12.2.4 Sequential coherency and consistency

Similarity in this case is expressed as proximity in time or space. Examples: frames in a video, samples in audio (time), pixels in an image (space). For instance it doesn't work with words.

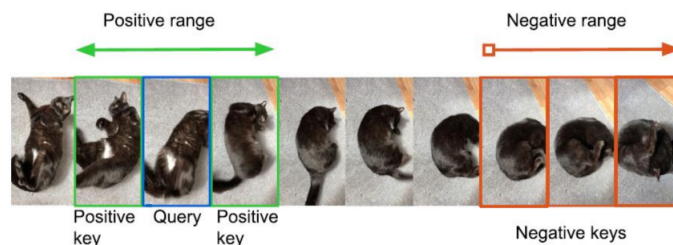


Figure 12.10: Sequential coherency and consistency.

12.2.5 Natural clustering

Clustering algorithms are used to create cluster centroids. Then we take as a positive key a sample that belongs to the same cluster of q and the negative key is taken from another cluster.

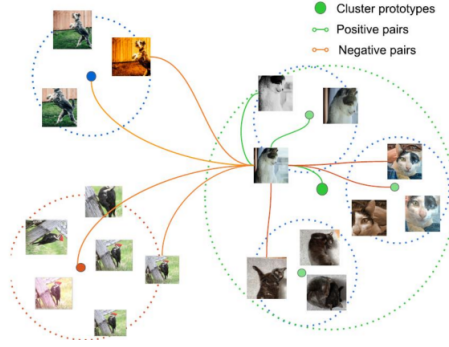


Figure 12.11: Natural clustering example.

This can be done in many ways. For instance you can use another NN or you can exploit a relation between the samples in your dataset (even if you don't have any labels). The downside of this solution is that it is time consuming at the beginning, but it can perform very well as a similarity map.

12.2.6 Dissimilarity

Picking a dissimilar sample is usually easier. One can simply sample negative keys by taking the other samples in the minibatch. Note that it could happen that we might pick a negative key $\mathbf{k}^-$ that is similar to our query $\mathbf{q}$. But this is not a problem: it means that that pair is an *hard negative* sample, that is, it is hard to distinguish between the two.

12.3 Scoring functions for similarity

A scoring function is needed in order to tell how similar are the z feature vectors (output of the transform head). Usually z is an high dimensionality vector, so it is not difficult to define a scoring function. We have many choices:

- **Euclidean distance:** $D(\mathbf{z}_i, \mathbf{z}_j) = \|\mathbf{z}_i - \mathbf{z}_j\|_2$. The lower the value, the more similar are the two vectors. Problem: When the two vectors have high dimensionality, the value tends to be too high (even if they are very similar) simply due to the large number of terms in the summation.

- **Scalar product:** $S(\mathbf{z}_i, \mathbf{z}_j) = \mathbf{z}_i^T \mathbf{z}_j = \|\mathbf{z}_i\| \cdot \|\mathbf{z}_j\| \cos \alpha$. Problem: similar to Euclidean distance.
- **Cosine similarity:** $CS(\mathbf{z}_i, \mathbf{z}_j) = \mathbf{z}_i^T \mathbf{z}_j / (\|\mathbf{z}_i\|_2 \|\mathbf{z}_j\|_2)$. By far the most used scoring function. It measures the cosine between the two vectors, so it is bounded between $[-1, 1]$. Note that with this function: the closer the value of $\cos(\alpha)$ to 1, the more similar are the two vectors.

12.4 Contrastive losses

In this section we introduce some contrastive losses.

12.4.1 "Original" Contrastive loss

It's the first proposed loss for the Contrastive framework. The loss function is defined as follows:

$$\mathcal{L}(\mathbf{q}, \mathbf{k}) = \begin{cases} D(\mathbf{z}_q, \mathbf{z})^2 & \text{if } k \sim p^+(\mathbf{k}^+|\mathbf{q}) \\ \max(0, m - D(\mathbf{z}_q, \mathbf{z})^2) & \text{if } k \sim p^-(\mathbf{k}^-|\mathbf{q}) \end{cases} \quad (12.1)$$

where $D(\cdot, \cdot)$ is the Euclidean distance, $m > 0$ is an hyperparameter that we need to fix that is called **margin**. For positive keys, we aim to minimize the Euclidean distance between the two vectors. For negative keys we aim to maximize it and to achieve zero loss the distance must be greater than or equal to the margin m .

12.4.2 Triplet loss

This loss is more powerful. It is defined as follows:

$$\mathcal{L}(\mathbf{q}, \mathbf{k}^+, \mathbf{k}^-) = \max(0, \overbrace{D(\mathbf{z}_q, \mathbf{z}^+)^2 - D(\mathbf{z}_q, \mathbf{z}^-)^2}^{\text{Relative distance}} + m) \quad (12.2)$$

where $k^+ \sim p^+(\mathbf{k}^+|\mathbf{q})$, $k^- \sim p^-(\mathbf{k}^-|\mathbf{q})$. The "original" loss works with pairs of samples and it doesn't measure the relative distance between the "positive" score ($D(\mathbf{z}_q, \mathbf{z}^+)$) and the "negative" score ($D(\mathbf{z}_q, \mathbf{z}^-)$). Instead the triplet loss pushes the difference between the two scores to be a large negative number. Indirectly it means that $D(\mathbf{z}_q, \mathbf{z}^+)$ must be small and $D(\mathbf{z}_q, \mathbf{z}^-)$ must be large and the difference between the two values must be $\leq -m$ in order to get zero loss.

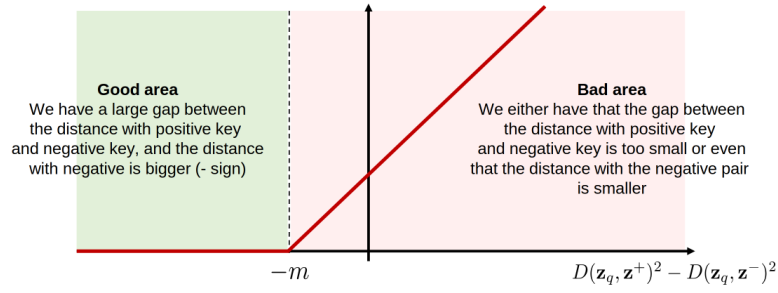


Figure 12.12: Triplet loss visualized.

12.4.3 NT-Xent

The most complicated contrastive loss function is the normalized temperature-scaled cross-entropy. It is defined for a **set of keys** (e.g. minibatch) $\mathcal{K}$:

$$\mathcal{L}(q, \mathbf{k}^+) = -\log \frac{e^{s(\mathbf{z}_q, \mathbf{z}^+)/\tau}}{\sum_{\mathbf{k} \in \mathcal{K}} e^{s(\mathbf{z}_q, \mathbf{z})/\tau}} \quad (12.3)$$

where τ is the temperature (hyperparameter), $\mathbf{z}$ is the projected representation of $\mathbf{k}$ and $s(\cdot, \cdot)$ is a generic scoring function (usually the $CS(\cdot)$ function). The argument of the log function is like a Softmax function with the additional temperature parameter τ .

Temperature in the softmax Consider the argument of the log function. The value of the temperature parameter tunes the **entropy** of the distribution. The higher the temperature, the higher the entropy $\Rightarrow$ the closer our distribution to uniform distribution.

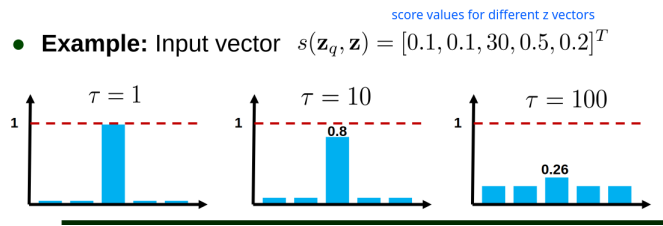


Figure 12.13: Temperature of the Softmax example. Note that the maximum remains the maximum in all three cases.

NT-Xent is hardness aware We want to show why the NT-Xent loss function is (sometimes) a better choice w.r.t. the ones introduced so far. To be hardness aware means that the negative keys are not all treated in the same way. To better understand this, consider an example in which the query q is similar to a negative key k_1^- and to

another negative key k_2^- . Let's say that it is more difficult to distinguish q from k_1^- than distinguish q from k_2^- . If we compute the gradients in such cases we will see that the harder negative sample (k_1^-) gets larger gradients w.r.t. the other one.

To summarize:

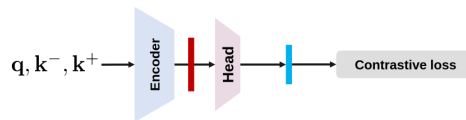
- Hardness refers to how difficult it is to distinguish between a query sample and a negative sample.
- The loss function automatically gives more attention to negative hard samples.
- Harder negatives receive larger gradients and thus more learning focus.

This automatic adjustment to focus on harder examples makes NT-Xent more effective than simpler contrastive losses that treat all negative samples equally. In this way, the model learns more from challenging examples.

Temperature in NT-Xent Temperature regulates the penalty on hard negative samples. Higher temperature leads to more uniform treatment of negatives, lower temperature increases the focus on hard negatives.

12.5 Transform head

As we have already mentioned, the head is used to map the general feature vector h into a representation z that optimizes the contrastive loss. One can even extend this approach: add multiple heads that take as input the same representation h and then train the model with multiple contrastive losses. With this approach we can get better general feature vectors.



The head network can be implemented:

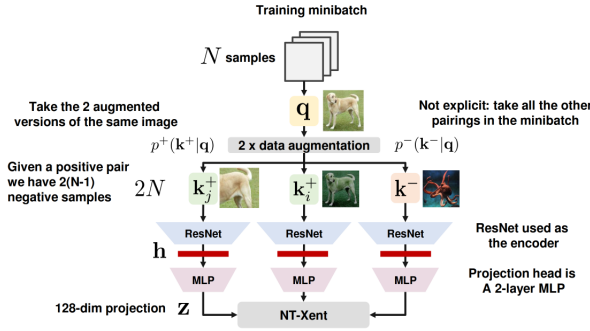
- using a **linear mapping** (just a matrix): low complexity solution but low power;
- using a small **FFNN** (1-2 layers): higher complexity and power.

Also the output vector of the head has usually a lower dimensionality w.r.t. the input.

12.6 SimCLR

12.6.1 Structure

SimCLR is a simple framework for contrastive learning of visual representations. The **objective** is to learn representations of images. The performance of the network are tested by using those representations as input to a linear classifier.



- Data augmentation: composition of **multiple** transforms

$$k_i^+ \sim t(q), k_j^+ \sim t'(q)$$

t, t'

1. Random crop
2. Color distortion
3. Gaussian blur

- Encoding step $h = \text{ResNet}(k)$

Representation vector

- Projection head $z = g(h) = \mathbf{W}^{(2)} \text{ReLU}(\mathbf{W}^{(1)} h)$

Weight matrices

- Similarity metric $s(z_i, z_j) = CS(z_i, z_j)$

Cosine similarity

The loss function for a positive pair of examples (i, j) (see in the architecture above) is defined as:

$$\ell(i, j) = -\log \frac{e^{s(z_i, z_j)/\tau}}{\sum_{m=1}^{2N} \mathbb{1}_{m \neq i} e^{s(z_i, z_m)/\tau}}. \quad (12.4)$$

The negative examples are the $2N - 2$ examples in the augmented minibatch. The final loss is computed as an average across all positive pairs, both (i, j) and (j, i) , in a mini-batch:

$$\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)] \quad (12.5)$$

where, for example, $(1, 2)$ is treated as a positive pair, as are $(3, 4)$, $(5, 6)$, and so on. where in the formula we consider that $(1, 2)$ is a positive pair, so as $(3, 4)$, $(4, 5)$, and so on.

Some key findings given by the authors of the paper:

1. A suitable combination of **data augmentation** methods is very effective in generating positive samples.
2. A **non-linear transform head** before the contrastive loss improves the quality of the representations.
3. Various findings on the **batch-size**, transform head dimension, among others.

The third point is actually intuitive: the size of the minibatch has a direct impact on the loss function. To be more precise it has a direct impact on the probabilities of mistaking

k_i^+ with k_j^+ . Indeed, bigger minibatch implies much more negative examples per positive pair.

12.6.2 Results

As we have mentioned, the results are measured by training a linear classifier (not a CNN) using the representations provided by the constrastive network. The dataset used is ImageNet $\Rightarrow$ each image in the dataset was converted into the representations provided by the contrastive network.

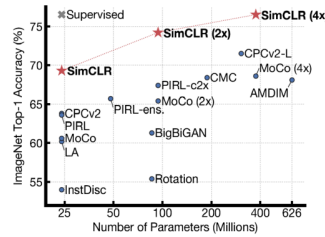


Figure 12.14: SimCLR indicates the accuracy achieved by the linear classifier trained on the representations provided by the contrastive network. The supervised approach (uses label from the beginning) is the upper bound. We can see that the contrastive approach achieve a similar accuracy to the supervised approach but the network is much more big!

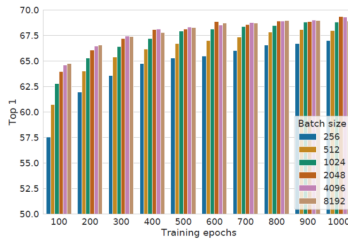


Figure 12.15: Large batch size leads to significant improvements. Now learning also depends on the relative random sampling (due to the minibatch creation) of positive/negative keys and longer training helps by showing the network more combinations.

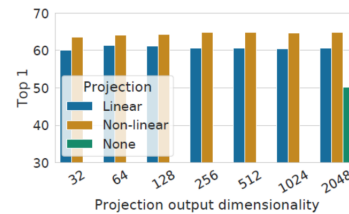


Figure 12.16: Projection head has a huge impact on the final performance: we can see (green bar) that if we do not use it, the performance drops. Note also that the dimensionality of the projection is less important.